

Safe-to-Use Speedup Techniques for VHAGaR Global-Robustness Verification

in Standard and Advanced-Standard Mode

VHAGaR / neta-s-paper — Internal technical note

Abstract

This note documents **safe-to-use** techniques that speed up VHAGaR’s global-robustness verification along two axes:

Standard mode — single network (Section 3)

Techniques that apply directly to the standard-mode MIP for a single network N_1 , with no prior solve required: perturbed-interval coupling between the clean and perturbed copies, single-network zonotope bound tightening, and per-copy triangle drop / sibling-gated refinement using only single-network state.

Advanced-standard (advstd) mode — transfer $N_1 \rightarrow N_2$ (Section 4)

Techniques that compute the global robustness bound $\delta_{N_2}^*$ for a target network N_2 by *reusing solver state* from a prior solve of a structurally similar network N_1 : weight-drift bound tightening with N_1 ’s primal, transfer-probability variable hints, and the transfer-aware extensions of the standard-mode relaxations.

“Safe-to-use” means empirically validated to produce average speedups with no severe slowdowns on any cell (geometric-mean time ratio $< 0.952\times$ and worst-cell time ratio $\leq 1.333\times$; i.e. `perf_tier` $\in \{\text{dominant}, \text{avg-win}\}$). Every technique documented here preserves the optimal global-robustness bound δ_{exact} (the relaxation-based ones preserve it as a sound upper bound).

1 Motivation and Setting

Let N_1 and N_2 be two neural networks with the same architecture — same layer sizes, same ReLU structure, same input domain — but different parameter values. A typical instance is $N_2 = N_1$ after one additional SGD epoch.

The global robustness verification problem, as formulated in VHAGaR, asks: for a given source class c_s , target class c_t , perturbation type P , and bound ε , what is the smallest δ such that no input classified with confidence at least δ as c_s can be perturbed within $\mathcal{I}_\varepsilon(\cdot)$ to be classified as c_t ? This is encoded as a mixed-integer program (MIP) and solved with Gurobi.

When N_1 and N_2 are structurally identical but numerically close, the two MIPs have identical variable names, identical constraint structure (only coefficients differ), and similar solution geometry. Advanced standard mode exploits this similarity through a set of concrete, optional techniques.

2 Background: ReLU Encoding in the MIP

The VHAGaR MIP encodes each ReLU neuron $z^+ = \max(0, \hat{z})$ using a *big-M formulation* with a binary variable $a \in \{0, 1\}$. Given pre-activation bounds $\hat{z} \in [\underline{z}, \bar{z}]$ (where $\underline{z} < 0 < \bar{z}$ for a “split”

neuron), the encoding introduces:

$$z^+ \geq 0, \tag{1}$$

$$z^+ \geq \hat{z}, \tag{2}$$

$$z^+ \leq \hat{z} - \underline{z}(1 - a), \tag{3}$$

$$z^+ \leq \bar{z}a. \tag{4}$$

This is *exact*: when $a = 1$ (neuron active), constraints force $z^+ = \hat{z}$; when $a = 0$ (neuron inactive), they force $z^+ = 0$. The binary a is what makes the problem a *mixed-integer* program — solving over all possible $\{0, 1\}$ assignments is what makes verification NP-hard.

The LP relaxation of ReLU. If we *drop* the binary a and keep only the three constraints that do not involve a , we get the *triangle LP relaxation*:

$$z^+ \geq 0, \tag{5}$$

$$z^+ \geq \hat{z}, \tag{6}$$

$$z^+ \leq \frac{\bar{z}}{\bar{z} - \underline{z}} (\hat{z} - \underline{z}). \tag{7}$$

This is a *convex relaxation*: it allows z^+ to take any value inside a triangle in the $(\hat{z}, z^+)$ plane, not just the two exact ReLU branches. There is no binary variable, so the problem becomes a linear program (LP) — much faster to solve, but the feasible set is *larger* than the exact one. Any solution to the exact encoding also satisfies the LP relaxation, but not vice versa.

The LP relaxation is used in Per-Copy Triangle Drop: selected binary variables in the final N_2 MIP are replaced with triangle relaxations to reduce the integer search space.

3 Boosting Standard Mode (single-network N_1)

This section documents techniques that apply directly to the standard-mode MIP for a single network N_1 — no prior solve of a related network is required. The same techniques (with extensions that exploit N_1 ’s primal and weight-drift information) are also used in the transfer-mode pipeline of Section 4; we describe the single-network forms here first so that the transfer extensions in Section 4 can be presented as “add N_1 -state on top of X .”

3.1 Perturbed Interval Constraints (**PerturbedIntervals**)

Flag: `--use_perturbed_intervals` $\in \{\text{false}, \text{true}\}$ (default true). **Model modification:** adds two linear inequalities per matched ReLU neuron linking the clean (“org”) and perturbed (“perturbation”) copies of the network. *After* the dual MIP is constructed; *before* `optimize!` is called.

Filename tag: `_PerturbedIntervals` appended. **Implementation:** `perturbed_interval_constraints(m, nn)` in `utils/perturbation_intervals.jl`, invoked from `run.jl` immediately after `get_model()` returns.

Perturbed interval constraints are not an *advstd* technique — the flag is read in every execution mode (standard, transfer, `advanced_standard_n2`), where in each mode it couples the two copies of the network that share weights. We describe them in detail here because they are referenced by Per-Copy Triangle Drop (the safety net that prevents the relaxed and exact copies from drifting; Section 4.4), by Sibling-Gated Refinement (the “both thin” coupling line is the pre-activation

analogue of the post-activation perturbed-interval coupling; Section 4.5), and by the unsound-combination guard at `run.jl:693--702` (which forbids `--adv_std_n2_relax_threshold > 0` unless `--use_perturbed_intervals=true`). **All safe combinations have `--use_perturbed_intervals=true`.**

What the flag adds, in one sentence. For each ReLU neuron (m, k) shared by the org and pert copies of the network, the flag adds two linear inequalities

$$z_{m,k}^{+, \text{pert}} \leq z_{m,k}^{+, \text{org}} + \Delta_{m,k}^{\uparrow}, \quad (8)$$

$$z_{m,k}^{+, \text{pert}} \geq z_{m,k}^{+, \text{org}} + \Delta_{m,k}^{\downarrow}, \quad (9)$$

where $[\Delta_{m,k}^{\downarrow}, \Delta_{m,k}^{\uparrow}]$ is a sound interval over-approximation of the post-ReLU activation difference $z_{m,k}^{+, \text{pert}} - z_{m,k}^{+, \text{org}}$ propagated layer by layer from the input perturbation $v_e \in [-\varepsilon, \varepsilon]^n$ using interval arithmetic. No binary variables are introduced; no $\hat{z}$ variable is created or modified; nothing inside either copy’s encoding changes.

Why this is informative. The dual MIP for a single network contains two copies of the same N -layer architecture: one operating on the clean input $v_{\text{in}} \in [0, 1]^n$ and one on the perturbed input $v_{x_0} = v_{\text{in}} + v_e$, with v_e bounded by the perturbation budget. *The two copies share weights:* the bias and weight coefficients at every layer are identical between the org and pert encodings. As a consequence, the per-neuron activation difference $z_{m,k}^{+, \text{pert}} - z_{m,k}^{+, \text{org}}$ depends only on the input difference v_e propagated through the network, and is bounded by an interval that shrinks proportionally with ε .

The exact MIP already constrains $v_e \in [-\varepsilon, \varepsilon]^n$, but it does so only at the *input* layer; the LP relaxations at internal nodes of Gurobi’s branch-and-bound tree are free to pick org- and pert-side activations independently subject only to the layer-wise affine and big- M constraints. The two copies can therefore drift apart in the LP relaxation in ways that the integer-feasible region forbids. Perturbed interval constraints close that gap by injecting the analytically derived per-layer drift bound directly into the constraint system, so the LP relaxation sees the same compatibility between the two copies that the integer-feasible region enforces implicitly.

This is exactly the same role that the big- M constraints play for single-copy ReLU: a sound, redundant-at-integer constraint that *tightens the LP relaxation* without changing the feasible set.

3.1.1 Interval-arithmetic recurrence for $\Delta^{\downarrow}, \Delta^{\uparrow}$

Let $\Delta_{m,k}^a \triangleq z_{m,k}^{+, \text{pert}} - z_{m,k}^{+, \text{org}}$ denote the per-neuron post-ReLU activation difference at layer m , neuron k . The recurrence below produces a sound interval $[\Delta_{m,k}^{\downarrow}, \Delta_{m,k}^{\uparrow}]$ satisfying $\Delta_{m,k}^{\downarrow} \leq \Delta_{m,k}^a \leq \Delta_{m,k}^{\uparrow}$ for every feasible joint assignment to the org and pert copies. The recurrence is implemented in `perturbed_interval_constraints(·)` (`utils/perturbation_intervals.jl:634--708`) and the per-layer type dispatch matches the network’s actual sequence of `Linear/Conv/ReLU/Flatten` layers.

Initialisation (input layer). At the input, $\Delta_0 = v_{x_0} - v_{\text{in}} = v_e$. The starting interval is the per-perturbation-type input bound on v_e :

$$[\Delta_0^{\downarrow}, \Delta_0^{\uparrow}] = [v_e^{\downarrow}, v_e^{\uparrow}].$$

The concrete numbers depend on the perturbation type (see Section 3.1.2); for ℓ_{∞} they are $[-\varepsilon \mathbf{1}, +\varepsilon \mathbf{1}]$.

Affine layer (Linear or Conv2d). Since both copies use the same affine map $z = W^\top a + b$, the difference is

$$\Delta_m^z = W^\top \Delta_{m-1}^a$$

(the bias b cancels because it is the same constant in both copies). Interval arithmetic with the constant matrix W yields:

$$\Delta_{m,k}^{z,\downarrow} = \sum_{k'} \min(W_{k,k'} \cdot \Delta_{m-1,k'}^{a,\downarrow}, W_{k,k'} \cdot \Delta_{m-1,k'}^{a,\uparrow}), \quad (10)$$

$$\Delta_{m,k}^{z,\uparrow} = \sum_{k'} \max(W_{k,k'} \cdot \Delta_{m-1,k'}^{a,\downarrow}, W_{k,k'} \cdot \Delta_{m-1,k'}^{a,\uparrow}). \quad (11)$$

These two operations are realised by `interval_matrix_vector_multiplication(W, W,  $\Delta^\downarrow, \Delta^\uparrow$ )` (Linear layers) and `interval_conv2d_bounds(F, F,  $\Delta^\downarrow, \Delta^\uparrow, 0, \text{stride}, \text{padding}$ )` (Conv2d layers), called with the network’s actual weight/filter and a zero bias. The convention $W = W^\top \cdot (\text{matrix})$ matches the MIPVerify.jl `Linear` convention noted in vagher.org/CLAUDE.md (matrix shape (input_dim, output_dim) with forward pass `transpose(matrix) · x + bias`).

ReLU layer. For each neuron at the ReLU layer, the pre-activation difference $\Delta_{m,k}^z$ has just been computed by the preceding affine step. The post-ReLU difference is clipped according to the sound rule

$$\sigma(\hat{z} + \Delta) - \sigma(\hat{z}) \in [\min(0, \Delta^\downarrow), \max(0, \Delta^\uparrow)],$$

which is exact for univariate ReLU under any fixed $\hat{z}$ and remains sound when $\hat{z}$ varies inside the layer’s feasible interval. The per-neuron clip is therefore

$$\Delta_{m,k}^{a,\uparrow} \leftarrow \max(0, \Delta_{m,k}^{z,\uparrow}), \quad (12)$$

$$\Delta_{m,k}^{a,\downarrow} \leftarrow -\max(0, -\Delta_{m,k}^{z,\downarrow}). \quad (13)$$

Equivalently, $\Delta^{a,\uparrow} = \max(0, \Delta^{z,\uparrow})$ and $\Delta^{a,\downarrow} = \min(0, \Delta^{z,\downarrow})$. Note that a neuron that is stable-inactive on both copies contributes the trivial interval $\{0\}$, and a neuron that is stable-active on both copies passes its pre-activation interval through unchanged.

Flatten layer. Permutes/reshapes the array without changing values. The interval arrays are reshaped by piping through the same `Flatten` operator, so the per-neuron correspondence with the JuMP variable naming convention is preserved.

Why the per-neuron correspondence is exact, not approximate. `interval_matrix_vector_multiplication` computes $[\Delta^{z,\downarrow}, \Delta^{z,\uparrow}]$ using the *exact* weights W that both copies use to compute their own pre-activations. There is no second-order approximation, no enclosing zonotope, and no linearisation around a base point: the only over-approximation is the elementwise loss of correlation between neurons of the same layer (interval arithmetic discards the fact that two outputs of the same linear layer share the same noise variables). Since (8)–(9) only consume the per-neuron upper/lower endpoints, the per-neuron correlation loss is exactly the information we discard and the per-neuron endpoint is what we extract.

3.1.2 Initialisation by perturbation type

The recurrence of Section 3.1.1 is generic; the only piece that varies by perturbation is the input-layer interval $[\Delta_{0,i}^\downarrow, \Delta_{0,i}^\uparrow]$ on the per-pixel difference $\Delta_{0,i} \triangleq v_{x_0,i} - v_{\text{in},i}$. Each `get_perturbation_specific_keys_*` function in `utils/perturbation_models.jl` writes this initial interval into the globals `I_pert_prev_up` and `I_pert_prev_down` together with the MIP constraints that define how the perturbed input v_{x_0} relates to the clean input v_{in} .

Throughout this subsection, $v_{\text{in},i}, v_{x_0,i} \in [0, 1]$ denote the clean and perturbed input pixels (declared bounds of the JuMP variables), e (or v_e) denotes a perturbation parameter, ε is the perturbation budget, and $\mathbf{1}_S$ is the indicator of an index set S . The mapping from each MIP constraint to its closed-form Δ formula and the resulting tight per-pixel range is given below. The same eight perturbations cover both the standard `get_model` dispatch and its `get_model_transfer` twin in the transfer pipeline.

ℓ_∞ (`--perturbation=linf`). The MIP introduces a per-pixel perturbation variable $v_{e,i} \in [-\varepsilon, +\varepsilon]$ and the constraint $v_{x_0,i} = v_{\text{in},i} + v_{e,i}$. Subtracting gives $\Delta_{0,i} = v_{e,i} \in [-\varepsilon, +\varepsilon]$, so

$$\Delta_{0,i}^\downarrow = -\varepsilon, \quad \Delta_{0,i}^\uparrow = +\varepsilon, \quad \forall i. \quad (14)$$

Example. At $\varepsilon = 0.1$ and a single hidden ReLU layer with $W_h = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}$ (the running network of Section 3.1.8), the input interval $[-0.1, +0.1]^2$ propagates to $[-0.2, +0.2]$ at each pre-activation of the first hidden layer.

Brightness (`--perturbation=brightness`). The MIP introduces a single *scalar* variable $v_e \in [0, \varepsilon]$ shared across all pixels and the constraint $v_{x_0,i} = v_{\text{in},i} + v_e$. Subtracting gives $\Delta_{0,i} = v_e$, identical for every i , so

$$\Delta_{0,i}^\downarrow = 0, \quad \Delta_{0,i}^\uparrow = +\varepsilon, \quad \forall i. \quad (15)$$

The per-pixel bound is tight (achieved at $v_e = 0$ and $v_e = \varepsilon$). The relational fact $\Delta_{0,i} = \Delta_{0,j}$ for every i, j is *not* exploited by PI, which is a per-pixel scheme; capturing that single degree of freedom would require introducing cross-pixel equality constraints on Δ , outside the scope of this technique. *Example.* At $\varepsilon = 0.05$, every input pixel has the same $\Delta_{0,i} \in [0, +0.05]$.

Contrast (`--perturbation=contrast`). The MIP introduces a scalar multiplier $e \in [1, 1 + \varepsilon]$ and the *bilinear* constraint $v_{x_0,i} = e \cdot v_{\text{in},i}$. Solving in closed form, $\Delta_{0,i} = (e - 1) \cdot v_{\text{in},i}$. With $(e - 1) \in [0, \varepsilon]$ and $v_{\text{in},i} \in [0, 1]$,

$$\Delta_{0,i}^\downarrow = 0, \quad \Delta_{0,i}^\uparrow = +\varepsilon, \quad \forall i. \quad (16)$$

Tightness: the maximum ε is realised at $e = 1 + \varepsilon$, $v_{\text{in},i} = 1$; the minimum 0 at $e = 1$. A pixel-aware sharper bound would be $[0, \varepsilon \cdot v_{\text{in},i}]$, but $v_{\text{in},i}$ is a MIP variable, not a constant, so the input interval must use the worst case $v_{\text{in},i} = 1$. *Example.* At $\varepsilon = 0.2$, every pixel has $\Delta_{0,i} \in [0, +0.2]$; the binding scenario is a fully-white pixel ($v_{\text{in},i} = 1$) under maximum contrast ($e = 1.2$).

Patch (`--perturbation=patch`). For a rectangular patch region $P \subseteq \{1, \dots, n\}$, the MIP emits $-\varepsilon \leq v_{x_0,i} - v_{\text{in},i} \leq +\varepsilon$ for $i \in P$ and $v_{x_0,i} = v_{\text{in},i}$ for $i \notin P$. Reading off $\Delta_{0,i}$ directly,

$$[\Delta_{0,i}^\downarrow, \Delta_{0,i}^\uparrow] = \begin{cases} [-\varepsilon, +\varepsilon] & i \in P, \\ [0, 0] & i \notin P. \end{cases} \quad (17)$$

The bound is tight on P (by the explicit slack) and exact off P (by the equality constraint). *Example.* A 4×4 image with a 2×2 patch at the top-left corner and $\varepsilon = 0.05$: the four patch pixels have $\Delta \in [-0.05, +0.05]$; the other twelve are pinned to $\Delta = 0$.

Occlusion (`--perturbation=occ`). For an occlusion mask $O \subseteq \{1, \dots, n\}$, the MIP forces $v_{x_0,i} = 0$ for $i \in O$ (occluded pixels go to black) and $v_{x_0,i} = v_{in,i}$ for $i \notin O$. On the mask, $\Delta_{0,i} = -v_{in,i}$, which over $v_{in,i} \in [0, 1]$ ranges over $[-1, 0]$; off the mask $\Delta_{0,i} = 0$. So

$$[\Delta_{0,i}^\downarrow, \Delta_{0,i}^\uparrow] = \begin{cases} [-1, 0] & i \in O, \\ [0, 0] & i \notin O. \end{cases} \quad (18)$$

Example. A 3×3 image with a single occluded pixel at the centre: the central pixel has $\Delta \in [-1, 0]$; all eight neighbours have $\Delta = 0$. The lower endpoint -1 is realised by a fully-white pixel becoming fully-black.

Translation (`--perturbation=translation`). For an offset (t_d, t_r) , every pixel j whose source $j - \text{off}$ lies inside the image is constrained by $v_{x_0,j+\text{off}} = v_{in,j}$ (rigid copy); every “border” pixel where the source falls outside is zero-padded, $v_{x_0,j} = 0$. For an interior destination pixel $j + \text{off}$, $\Delta_{0,j+\text{off}} = v_{in,j} - v_{in,j+\text{off}}$, the difference of two independent $[0, 1]$ values, hence $\Delta \in [-1, +1]$. For a border pixel, $\Delta_{0,j} = -v_{in,j} \in [-1, 0]$. So

$$[\Delta_{0,i}^\downarrow, \Delta_{0,i}^\uparrow] = \begin{cases} [-1, 0] & i \text{ is a border (zero-padded) pixel,} \\ [-1, +1] & i \text{ is an interior (shifted) pixel.} \end{cases} \quad (19)$$

Example. A 4×4 image shifted by $(t_d, t_r) = (1, 0)$: the top row of 4 pixels is zero-padded ($\Delta \in [-1, 0]$); the remaining 12 interior shifted pixels have $\Delta \in [-1, +1]$. This corrects the previous loose default of $[-1, +1]$ across the *entire* image (lines 337–339 of `utils/perturbation_models.jl` prior to the per-region patch; see §3.1 in code).

Rotation (`--perturbation=rotation`). For a rotation angle θ , every destination pixel i whose rotated source i_r falls inside the image is constrained by a bilinear interpolation $v_{x_0,i} = \sum_{4 \text{ src pixels}} w_{i,s} v_{in,s}$ with non-negative weights $w_{i,s} \geq 0$ summing to 1; pixels whose rotated source falls outside are zero-padded, $v_{x_0,i} = 0$. For a mapped destination, the bilinear interpolation lies in $[0, 1]$ (a convex combination of $[0, 1]$ values), so $\Delta_{0,i} = (\text{interp}) - v_{in,i} \in [-1, +1]$. For an unmapped pixel, $\Delta_{0,i} = -v_{in,i} \in [-1, 0]$. So

$$[\Delta_{0,i}^\downarrow, \Delta_{0,i}^\uparrow] = \begin{cases} [-1, 0] & i \text{ is unmapped (zero-padded),} \\ [-1, +1] & i \text{ is mapped (bilinear interp).} \end{cases} \quad (20)$$

Example. A 3×3 image rotated by 10° : the centre pixel maps to itself with weight ≈ 1 ($\Delta \approx 0$); the four corner pixels may rotate out of frame and become unmapped ($\Delta \in [-1, 0]$); the four edge-midpoint pixels stay mapped but with non-trivial mixing, $\Delta \in [-1, +1]$.

Filter-v (`--perturbation=filterv`). A *deterministic* 1×3 horizontal smoothing filter hard-coded for 28×28 MNIST: the perturbation has no free parameter; given v_{in} , v_{x_0} is uniquely determined. The MIP emits three equality blocks (using the pixel-numbering of `utils/perturbation_models.jl`):

- *Interior* (loop for `j=1:27, i=28j+1..28j+27`): $v_{x_0,i} = 0.01 v_{in,i-1} + 0.99 v_{in,i} + 0.01 v_{in,i+1}$;

- *Right edge* ($i \in 28:28:784$): $v_{x_0,i} = 0.1 v_{\text{in},i-1} + 0.8 v_{\text{in},i}$;
- *Left edge* ($i \in 1:28:756$): $v_{x_0,i} = 0.8 v_{\text{in},i} + 0.1 v_{\text{in},i+1}$;
- *Uncovered* ($i \in 2..27$, a pre-existing gap): $v_{x_0,i}$ is free in $[0, 1]$.

Substituting $\Delta_{0,i} = v_{x_0,i} - v_{\text{in},i}$ and optimising the resulting linear form over the input box $[0, 1]^n$ gives the closed-form ranges

$$[\Delta_{0,i}^\downarrow, \Delta_{0,i}^\uparrow] = \begin{cases} [-0.01, +0.02] & i \text{ interior,} \\ [-0.2, +0.1] & i \text{ on the right or left edge,} \\ [-1, +1] & i \in 2..27 \text{ (uncovered).} \end{cases} \quad (21)$$

The tight extrema are reached at 0/1-valued inputs: e.g. interior $\Delta = +0.02$ at $v_{\text{in},i-1} = v_{\text{in},i+1} = 1$, $v_{\text{in},i} = 0$; interior $\Delta = -0.01$ at the reverse. *Example.* On MNIST, this gives a per-pixel input width of 0.03 on the interior (instead of 2.0 under a trivial $[-1, +1]$ fallback), a $\sim 67\times$ tightening that propagates through the recurrence to the matched constraints.

Summary table.

Perturbation	$\Delta_{0,i}^\downarrow$	$\Delta_{0,i}^\uparrow$
ℓ_∞	$-\varepsilon$	$+\varepsilon$
brightness	0	$+\varepsilon$
contrast	0	$+\varepsilon$
patch ($i \in P / i \notin P$)	$-\varepsilon / 0$	$+\varepsilon / 0$
occlusion ($i \in O / i \notin O$)	$-1 / 0$	$0 / 0$
translation (border / interior)	$-1 / -1$	$0 / +1$
rotation (unmapped / mapped)	$-1 / -1$	$0 / +1$
filter-v (interior / edge / uncovered)	$-0.01 / -0.2 / -1$	$+0.02 / +0.1 / +1$

3.1.3 Per-perturbation soundness

The general soundness of PI (Theorem 3) reduces to a single condition: the input-layer interval must contain the actual per-pixel difference $v_{x_0,i} - v_{\text{in},i}$ in every feasible solution. We now verify this for each of the eight perturbations above, which together with Theorem 3 yields preservation of δ_{exact} for every supported P when PI is enabled.

Theorem 1 (Per-perturbation input-bound soundness). *Let $P \in \{\ell_\infty, \text{brightness}, \text{contrast}, \text{patch}, \text{occlusion}, \text{translation}, \text{rotation}, \text{filter-v}\}$ and let $[\Delta_{0,i}^\downarrow, \Delta_{0,i}^\uparrow]$ be the per-pixel input interval given by the corresponding equation of (14)–(21). For every input index i and every (v_{in}, v_{x_0}) satisfying P ’s input constraints,*

$$\Delta_{0,i}^\downarrow \leq v_{x_0,i} - v_{\text{in},i} \leq \Delta_{0,i}^\uparrow. \quad (22)$$

Proof. We handle each P in turn. In every case the proof has the same structure: the constraint set defines a closed-form linear (or affine) expression for $v_{x_0,i} - v_{\text{in},i}$ in terms of the elementary MIP variables, and the claimed interval is the elementwise min/max of that expression over the variables’ declared box.

ℓ_∞ . $v_{x_0,i} - v_{\text{in},i} = v_{e,i}$ by the input constraint, and $v_{e,i} \in [-\varepsilon, +\varepsilon]$ by the MIP variable’s declared bounds. Hence (22) holds with equality at both endpoints.

Brightness. $v_{x_0,i} - v_{\text{in},i} = v_e$, and $v_e \in [0, \varepsilon]$ by declaration. The interval is exact.

Contrast. $v_{x_0,i} - v_{\text{in},i} = (e - 1) v_{\text{in},i}$ from the bilinear constraint. With $(e - 1) \in [0, \varepsilon]$ and $v_{\text{in},i} \in [0, 1]$, the product takes values in $[\min(0, 0 \cdot 0), \max(\varepsilon \cdot 1, \varepsilon \cdot 1)] = [0, \varepsilon]$. The max is achieved at $(e, v_{\text{in},i}) = (1 + \varepsilon, 1)$; the min at $(e - 1) = 0$ or $v_{\text{in},i} = 0$.

Patch. For $i \in P$, the inequalities $-\varepsilon \leq v_{x_0,i} - v_{in,i} \leq +\varepsilon$ are emitted directly. For $i \notin P$, the equality $v_{x_0,i} = v_{in,i}$ forces $v_{x_0,i} - v_{in,i} = 0 \in [0, 0]$.

Occlusion. For $i \in O$, the equality $v_{x_0,i} = 0$ combined with $v_{in,i} \in [0, 1]$ gives $v_{x_0,i} - v_{in,i} = -v_{in,i} \in [-1, 0]$. For $i \notin O$, the equality $v_{x_0,i} = v_{in,i}$ gives $\Delta_{0,i} = 0$.

Translation. For a border pixel i (where the shift’s source falls out of frame), the MIP forces $v_{x_0,i} = 0$, so by the same step as occlusion $v_{x_0,i} - v_{in,i} \in [-1, 0]$. For an interior destination $i = j + \text{off}$, the equality $v_{x_0,j+\text{off}} = v_{in,j}$ yields $v_{x_0,j+\text{off}} - v_{in,j+\text{off}} = v_{in,j} - v_{in,j+\text{off}}$, a difference of two independent $[0, 1]$ values, hence in $[-1, +1]$.

Rotation. For an unmapped pixel i , $v_{x_0,i} = 0$ and the same argument as translation/occlusion gives $\Delta_{0,i} \in [-1, 0]$. For a mapped pixel, $v_{x_0,i} = \sum_s w_{i,s} v_{in,s}$ with weights $w_{i,s} \geq 0$ and $\sum_s w_{i,s} = 1$. By convexity, $\sum_s w_{i,s} v_{in,s} \in [\min_s v_{in,s}, \max_s v_{in,s}] \subseteq [0, 1]$. Then $\Delta_{0,i} = (\text{interp}) - v_{in,i}$ is the difference of two values in $[0, 1]$, hence in $[-1, +1]$.

Filter-v. For an interior pixel i , $\Delta_{0,i} = 0.01 v_{in,i-1} - 0.01 v_{in,i} + 0.01 v_{in,i+1}$ is linear in three independent $[0, 1]$ values, with positive coefficients on $v_{in,i-1}, v_{in,i+1}$ and negative on $v_{in,i}$. The max equals $0.01 + 0 + 0.01 = 0.02$ (at $v_{in,i-1} = v_{in,i+1} = 1, v_{in,i} = 0$); the min equals $0 - 0.01 + 0 = -0.01$ (opposite corner). For a right-edge pixel, $\Delta_{0,i} = 0.1 v_{in,i-1} - 0.2 v_{in,i}$: max = 0.1 at $v_{in,i-1} = 1, v_{in,i} = 0$; min = -0.2 at the reverse. Symmetric for the left edge. For an uncovered pixel $i \in 2..27$, no constraint pins $v_{x_0,i}$, so $v_{x_0,i} \in [0, 1]$ (its declared bound) and $\Delta_{0,i} \in [-1, +1]$ by subtraction.

In every case the listed interval contains $v_{x_0,i} - v_{in,i}$ for every feasible input. $\square$

Corollary 2 (δ_{exact} preserved by PI for every supported perturbation). *Let P be any perturbation in the list of Theorem 1. Then with PI enabled (`--use_perturbed_intervals=true`), the constraints emitted by `perturbed_interval_constraints(.)` at every ReLU layer are redundant at integer feasibility, and consequently $\mathcal{F}^{(\text{PI})} = \mathcal{F}$ and the optimum solver value under PI equals δ_{exact} .*

Proof. By Theorem 1, the input-layer interval contains $v_{x_0,i} - v_{in,i}$ for every feasible input. By induction on layer depth using the sound affine and ReLU recurrence of Section 3.1.1 (eqs. (10)–(13)), the per-neuron interval $[\Delta_{m,k}^\downarrow, \Delta_{m,k}^\uparrow]$ at every layer m and neuron k contains the post-ReLU activation difference $z_{m,k}^{+, \text{pert}} - z_{m,k}^{+, \text{org}}$ at every feasible point of $\mathcal{F}$. Hence the constraints eqs. (8)–(9) hold at every feasible point, i.e., $\mathcal{F} \subseteq \mathcal{F}^{(\text{PI})}$. The reverse inclusion holds because $\mathcal{F}^{(\text{PI})}$ is defined by strictly more constraints than $\mathcal{F}$. Therefore $\mathcal{F} = \mathcal{F}^{(\text{PI})}$ and the optima coincide. $\square$

3.1.4 Matched constraint emission

At every ReLU layer, both copies have a post-ReLU variable per neuron in the JuMP model, named according to the `{network_version}x_rect_layerCount{n}_neuronCount{n}_{layer}_{neuron}` convention (Section 2 background, and `vaghar_org/CLAUDE.md`). The constraint emitter `add_matched_interval_constraint` (`utils/perturbation_intervals.jl:127--149`) reads `JuMP.all_variables(m)`, identifies all variables whose names contain the clean and pert prefixes (default “org” and “perturbation” in standard mode; “n1_org” and “n1_pert” or “n2_org” and “n2_pert” in transfer mode), and pairs them by the structural *neuron index* (the trailing integer after the last underscore) extracted via `extract_neuron_index(.)`.

Why neuron-index matching, not positional matching. After Bound Tightening has tightened bounds, some neurons may become stable on one copy (no `x_rect` variable created) but remain split on the other. A naive positional pairing would line up the wrong neurons. The matching scheme intersects the two neuron-index sets and emits the pair of constraints (8)–(9) only on

neurons present in both copies. The `add_matched_interval_constraints!` helper returns the total number of constraints added (two per common neuron), reported as `Interval constraints added: {n}` to stdout.

Operations on the bound arrays during propagation. The local interval arrays `I_pert_up_local`, `I_pert_down` are initialised from the globals `I_pert_prev_up`, `I_pert_prev_down` at function entry (`perturbation_intervals.jl`). They are propagated through every layer in the order the network’s `nn.layers` dictates. The 4D reshape on Conv2d layers — guarded by `ndims(·) == 4` tests on `perturbation_intervals.jl:667--668` — handles the case where the perturbation initialiser returned a flat-spatial array (e.g. MNIST-style `{1, 28, 28, 1}`) and the conv operator expects a 4D tensor. The arrays are flattened with `ensure_1d(·)` just before constraint emission so that indexing by the structural neuron index works regardless of upstream shape.

3.1.5 Soundness

Theorem 3 (Perturbed interval constraints preserve δ_{exact}). *Let $\mathcal{F}$ denote the feasible set of the MIP without perturbed interval constraints, and let $\mathcal{F}^{(\text{PI})}$ denote the feasible set with them. Then $\mathcal{F}^{(\text{PI})} = \mathcal{F}$. Consequently, $\max_{x \in \mathcal{F}^{(\text{PI})}} \text{obj}(x) = \max_{x \in \mathcal{F}} \text{obj}(x) = \delta_{\text{exact}}$.*

Proof. Take any $x \in \mathcal{F}$. For each ReLU neuron (m, k) , the affine and big- M constraints already in $\mathcal{F}$ enforce

$$z_{m,k}^{+, \text{org}}(x) = \text{ReLU}(\hat{z}_{m,k}^{\text{org}}(x)), \quad z_{m,k}^{+, \text{pert}}(x) = \text{ReLU}(\hat{z}_{m,k}^{\text{pert}}(x)).$$

The pre-activations satisfy $\hat{z}_{m,k}^{\text{org}}(x), \hat{z}_{m,k}^{\text{pert}}(x) \in [l_{m,k}, u_{m,k}]$ with both copies using identical weights, so the pre-activation difference can be expressed via the affine recurrence applied to the input difference $v_e(x) \in [v_e^\downarrow, v_e^\uparrow]$. Interval arithmetic on every affine and ReLU layer in that recurrence produces sound bounds: (10)–(11) are sound by the definition of the elementwise interval product, and (12)–(13) are sound by the elementary identity $\sigma(z + \Delta) - \sigma(z) \in [\min(0, \Delta), \max(0, \Delta)]$ valid pointwise. By induction on the layer index m , the output post-ReLU difference $z_{m,k}^{+, \text{pert}}(x) - z_{m,k}^{+, \text{org}}(x)$ lies in $[\Delta_{m,k}^\downarrow, \Delta_{m,k}^\uparrow]$ for every $x \in \mathcal{F}$. Therefore x satisfies (8)–(9), hence $x \in \mathcal{F}^{(\text{PI})}$. The reverse inclusion $\mathcal{F}^{(\text{PI})} \subseteq \mathcal{F}$ is immediate because $\mathcal{F}^{(\text{PI})}$ is defined by strictly more constraints. $\square$

The theorem holds independently of the perturbation type, of which techniques are enabled in `advstd`, and of whether the run is in standard or transfer mode. In particular, the equality $\mathcal{F}^{(\text{PI})} = \mathcal{F}$ is *tighter* than the inclusion $\mathcal{F}^{(\text{PI})} \subseteq \mathcal{F}$ one might expect from “adding constraints”: the added constraints are *redundant at integer feasibility* (they are implied by the existing constraint system), but *not* redundant at the LP relaxation level (the LP relaxation does not infer the per-layer drift bound from the input bound on v_e alone). This is what makes them useful: they tighten every node’s LP without changing the optimum.

3.1.6 Why the safe combinations require it

Three observations make perturbed intervals indispensable for the safe-combination space.

(i) The LP relaxation can decouple the copies. Without (8)–(9), an LP-relaxed node of Gurobi’s branch-and-bound tree is free to choose $z_{m,k}^{+, \text{org}}, z_{m,k}^{+, \text{pert}}$ independently subject only to each copy’s own triangle/big- M envelope. Because the two copies’ triangles are individually loose (especially the pert copy’s, which is propagated from the wider input box $[-\varepsilon, 1 + \varepsilon]^n$ on a perturbation like ℓ_∞), the LP relaxation often picks joint $(z^{+, \text{org}}, z^{+, \text{pert}})$ values that are infeasible for the integer

system. Without the coupling, the LP bound is loose and Gurobi spends most of its time tightening it through branching. With the coupling, the LP bound matches the integer bound much earlier in the search.

(ii) Per-Copy Triangle Drop requires the coupling for soundness. Section 4.4 drops binaries on individually-thin neurons. When the rule fires on only *one* of the two copies (the “one thin” tier), the relaxed copy receives a triangle while the sibling keeps its exact big- M . Absent perturbed interval constraints, the LP relaxation can pick a post-ReLU value on the relaxed copy that is incompatible with *any* feasible perturbation v_e but is still inside the triangle. The exact sibling cannot “correct” the relaxed side because no constraint says the two are related; the LP feasible set then strictly exceeds the integer-feasible set in a way that the relaxation’s overshoot bound ($\delta_{\text{BTPR}} \geq \delta_{\text{exact}}$) cannot recover from in a useful finite gap. The unsound-combination guard at `run.jl:693--702` formalises this: it refuses to run with `--adv_std_n2_relax_threshold > 0` unless `--use_perturbed_intervals=true`, emitting the diagnostic

```
UNSOUND COMBINATION: --adv_std_n2_relax_threshold > 0 (Sibling-Gated Refinement, BoundTightPertRelax
requires --use_perturbed_intervals=true for soundness. Without the perturbed-interval
coupling constraints the relaxed and exact copies can drift apart and  $\delta_{\text{exact}}$  is
not guaranteed.
```

The threshold $\tau = 0$ is exempted from the guard because at $\tau = 0$ no triangle is actually emitted (the gap formula $u \cdot |l|/(2(u - l))$ is strictly positive on every *split* neuron, and stable neurons short-circuit before reaching the relaxation path; see Section 4.5’s “ $\tau = 0$ ” analysis).

(iii) Sibling-Gated Refinement’s “both thin” tier emits the pre-activation analogue. The post-activation coupling $z^{+, \text{pert}} - z^{+, \text{org}} \in [\Delta^\downarrow, \Delta^\uparrow]$ supplied by (8)–(9) is the post-ReLU sibling of the pre-activation coupling $\hat{z}^{\text{pert}} - \hat{z}^{\text{org}} \in [l_{\text{int}}, u_{\text{int}}]$ that Sibling-Gated Refinement adds in its “both thin” tier (Section 4.5, Eq. (38)). The two intervals $[\Delta^\downarrow, \Delta^\uparrow]$ and $[l_{\text{int}}, u_{\text{int}}]$ are produced by the same per-layer `pert_up/pert_down` recurrence (`compute_diff_and_comp_bounds()` stores them at the *post*-ReLU site as $\Delta^\uparrow, \Delta^\downarrow$, and `perturbed_interval_constraints()` propagates the same recurrence inline). Their composition — coupling both the pre- and the post-activations of any neuron whose binary was dropped — is what makes Sibling-Gated Refinement strictly tighter than Per-Copy Triangle Drop.

3.1.7 Performance characterisation

Cost. The flag adds $2 \cdot |\{(m, k) : \text{neuron is split on both copies}\}|$ linear constraints to the MIP. On a typical 4x10 fully-connected network with ~ 40 split neurons per copy on both sides, that is at most ~ 80 extra rows in the constraint matrix — a fraction of a percent of the total constraint count of the dual MIP. Pre-solve in Gurobi typically simplifies away the inequalities on neurons where one side is dominated; the runtime cost of the post-processing is therefore in the noise floor relative to the LP tightening it provides.

Where the flag is on by default. In `run.jl:138` the argument-parser default for `--use_perturbed_intervals` is `true`. The Python experiment drivers (`utils/run_exp.py`, `utils/run_experiment.py`, `utils/train_and_run.py`, `utils/run_3x50_exp.py`) all pass `--use_perturbed_intervals true` explicitly. The standalone reproducer in `CLAUDE.md` shows it as `false` only for the *baseline* “without-perturbed-intervals” comparison row in the no-perturbed-intervals ablation; every `advstd` safe-combination row inherits the default `true`.

Where the safe-combination ranking sits relative to the baseline. The Section 5 tables report the per-cell sp as $t_{\text{std}}/t_{\text{adv}}$, where both t_{std} and t_{adv} are measured *with* `--use_perturbed_intervals=true` on both ends of the ratio. The flag is thus part of the baseline, not part of the technique mix being compared. A separate ablation (not shown in this document) confirms that the with-perturbed-intervals baseline is itself $\gtrsim 1.3\times$ faster than the without-perturbed-intervals baseline on the safe-combo benchmark cells, so the speedups in Section 5 are *additional* to the gain perturbed intervals already provide.

3.1.8 Algorithm and worked example

Algorithm 1 Perturbed Interval Constraints

Require: Network N , perturbation type P with budget ε , dual MIP $\mathcal{M}$ with org/pert copies of N

- 1: $(\Delta_0^\downarrow, \Delta_0^\uparrow) \leftarrow$ initialise from P (Section 3.1.2)
- 2: **for** layers $m = 1, \dots, L$ in network order **do**
- 3: **if** layer is Linear/Conv2d with weight W **then**
- 4: $\Delta_m^{z,\uparrow}, \Delta_m^{z,\downarrow} \leftarrow$ elementwise max / min of $W\Delta_{m-1}^{a,\downarrow}, W\Delta_{m-1}^{a,\uparrow}$ {eqs. 10–11; bias cancels}
- 5: **else if** layer is ReLU **then**
- 6: $\Delta_{m,k}^{a,\uparrow} \leftarrow \max(0, \Delta_{m,k}^{z,\uparrow}); \Delta_{m,k}^{a,\downarrow} \leftarrow \min(0, \Delta_{m,k}^{z,\downarrow})$ {eqs. 12–13}
- 7: **for** each neuron k alive (split) on both copies of $\mathcal{M}$ **do**
- 8: add to $\mathcal{M}$: $\Delta_{m,k}^{a,\downarrow} \leq z_{m,k}^{+, \text{pert}} - z_{m,k}^{+, \text{org}} \leq \Delta_{m,k}^{a,\uparrow}$ {eqs. 8–9}
- 9: **end for**
- 10: **else if** layer is Flatten **then**
- 11: permute/reshape the interval arrays through the same operator
- 12: **end if**
- 13: **end for**

Variables / constraints added. No new variables. *Two* linear inequalities per ReLU neuron alive on both copies (eqs. (8)–(9)); on a typical 4×10 FC network with ~ 40 split neurons per copy, that is ~ 80 rows added to the constraint matrix.

Worked example (running network). We use a small running example throughout this technical note: input $x \in [0, 1]^2$, one hidden ReLU layer with weights $W_h = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}$ and bias $b_h = (-0.3, -0.1)$, scalar output $y = z_1^+ - z_2^+$, and ℓ_∞ perturbation with $\varepsilon = 0.1$.

Input interval (Section 3.1.2, ℓ_∞ row): $[\Delta_0^\downarrow, \Delta_0^\uparrow] = ([-0.1, -0.1], [+0.1, +0.1])$.

Affine recurrence at the hidden layer (eqs. 10–11):

$$\begin{aligned} [\Delta_{1,1}^{z,\downarrow}, \Delta_{1,1}^{z,\uparrow}] &= (+1) \cdot [-0.1, +0.1] + (+1) \cdot [-0.1, +0.1] = [-0.2, +0.2] \\ [\Delta_{1,2}^{z,\downarrow}, \Delta_{1,2}^{z,\uparrow}] &= (-1) \cdot [-0.1, +0.1] + (+1) \cdot [-0.1, +0.1] = [-0.2, +0.2]. \end{aligned}$$

The shared biases cancel in the difference.

Post-ReLU clip (eqs. 12–13): $\Delta_{1,k}^{a,\uparrow} = \max(0, +0.2) = 0.2$, $\Delta_{1,k}^{a,\downarrow} = \min(0, -0.2) = -0.2$ for $k \in \{1, 2\}$.

Emitted constraints. Two rows per hidden neuron, e.g. for $k = 1$:

$$-0.2 \leq z_{1,1}^{+, \text{pert}} - z_{1,1}^{+, \text{org}} \leq +0.2,$$

and identically for $k = 2$. Total: 4 inequalities (2 split neurons, 2 copies, 1 layer).

3.2 Absolute Zonotope Bound Tightening

Single-network analogue of: Source B of the advstd Bound Tightening (Section 4.2.2). **Model modification:** tightens N_1 ’s per-neuron ReLU pre-activation bounds before the MIP is built.

Goal. For each ReLU neuron (m, k) of N_1 , compute a sound interval $[l_{m,k}^{N_1}, u_{m,k}^{N_1}]$ on the pre-activation $\hat{z}_{m,k}^{N_1}$ that is tighter than what plain interval arithmetic on N_1 produces. Tighter bounds reduce big- M constants and may flip split neurons to stable, eliminating their binary entirely from the MIP.

Method. Propagate a single zonotope through N_1 whose initial hull simultaneously covers the clean input $v_{\text{in}} \in [0, 1]^n$ and the perturbed input $v_{x_0} = v_{\text{in}} + v_e \in [-\varepsilon, 1+\varepsilon]^n$:

$$c_0 = \frac{1}{2}\mathbf{1}, \quad G_0 = \text{diag}(\frac{1}{2} + \varepsilon, \dots, \frac{1}{2} + \varepsilon) \in \mathbb{R}^{n \times n}.$$

Affine layers update via $c \leftarrow Wc + b$, $G \leftarrow WG$. At each ReLU layer the standard DeepZ relaxation is applied per neuron: with hull endpoints $l_k = c_{m,k} - \sum_j |g_{m,k,j}|$ and $u_k = c_{m,k} + \sum_j |g_{m,k,j}|$, stable neurons ($u_k \leq 0$ or $l_k \geq 0$) pass through unchanged or are zeroed, while split neurons ($l_k < 0 < u_k$) are scaled by $\lambda_k = u_k / (u_k - l_k)$ with a new generator column carrying $\mu_k = -u_k l_k / (2(u_k - l_k))$ appended at row k (Section 4.2.2 gives the full recurrence). At every ReLU layer the per-neuron hull bounds

$$u_{m,k}^{N_1} = c_{m,k} + \sum_j |g_{m,k,j}|, \quad l_{m,k}^{N_1} = c_{m,k} - \sum_j |g_{m,k,j}|$$

are extracted and used to tighten the MIP’s big- M constants on neuron (m, k) .

This is exactly Source B of Section 4.2.2 applied to N_1 in standalone mode: there is no Source A diff bound to intersect against, because there is no second network. Compared to plain interval-arithmetic bound propagation through N_1 , the zonotope preserves correlations between neurons of the same layer and is therefore strictly tighter (Theorem 4 specialises to the single-network case by setting $\Delta W = 0$ throughout).

Soundness. Zonotope arithmetic is a sound abstract interpretation: at every neuron the computed $[l_{m,k}^{N_1}, u_{m,k}^{N_1}]$ contains every realisable pre-activation under any feasible (v_{in}, v_e) in their respective input boxes. Tightening the MIP’s big- M constants with these intervals therefore preserves the feasible set and the optimum δ_{exact} .

Conv layers: interval, not zonotope. The recurrence above is written for affine layers $z = Wc + b$, which matches **Linear** layers exactly. For **Conv2d** layers the implementation in `compute_n2_bounds_zonotop` (`utils/perturbation_intervals.jl`) currently falls back on *interval arithmetic* (the `else` branch at lines 1883–1894) because the standard-mode wiring in `main_standard` sets only the `use_zonotope` global, not `zonotope_conv`. The zonotope therefore “activates” at the first **Linear** layer encountered after the conv stack: the running interval state at that point is converted into a zonotope with diagonal G_0 (line 1844) and the recurrence above takes over from there. Concretely, on a typical CNN **Conv**→**ReLU**→**Conv**→**ReLU**→**Flatten**→**Linear**→**ReLU**→**Linear**, the conv-stage ReLUs receive intervals tightened by `interval_conv2d_bounds` only, while the fully-connected stage ReLUs receive the zonotope-tightened bounds described above.

The fall-back is sound (interval arithmetic is itself a sound abstract interpretation, so the per-neuron hull at every ReLU still contains all realisable pre-activations), but the conv-stage bounds are strictly looser than what a full zonotope-conv pass would produce. The bounds are therefore

comparable in tightness, layer-for-layer, to the n1ProbeLP-off Source A zonotope path used by advstd (`--adv_std_zono_bounds=true`; see Section 4.2.2), which makes the exact same trade-off. Full zonotope-conv is implemented by `conv2d_zonotope` (`utils/perturbation_intervals.jl`) and currently activated only when `zonotope_conv=true`; today that flag is set inside `main_transfer` (`run.jl`, line 1434) but not by the standard-mode or advstd-mode wirings. Lifting this restriction is a pure-precision upgrade with no soundness implication and is left as future work.

3.2.1 Algorithm and worked example

Algorithm 2 Absolute Zonotope Bound Tightening on N_1

Require: N_1 with L layers, input box $[0, 1]^n$, perturbation budget ε

```

1:  $\mathbf{c}_0 \leftarrow \frac{1}{2}\mathbf{1}_n$ ;  $G_0 \leftarrow (\frac{1}{2} + \varepsilon) I_n$  {hull covers  $[-\varepsilon, 1 + \varepsilon]^n$ }
2: for layers  $m = 1, \dots, L$  do
3:   if layer is Linear/Conv2d with  $(W, b)$  then
4:      $\mathbf{c} \leftarrow W\mathbf{c} + b$ ;  $G \leftarrow WG$  {conv falls back to interval-conv if zonotope_conv=false}
5:   else if layer is ReLU then
6:     for all neurons  $k$  do
7:        $l_k \leftarrow c_k - \sum_j |g_{k,j}|$ ;  $u_k \leftarrow c_k + \sum_j |g_{k,j}|$ 
8:       if  $u_k \leq 0$  then
9:         zero row  $k$  of  $\mathbf{c}$  and  $G$ 
10:      else if  $l_k \geq 0$  then
11:        pass row  $k$  through unchanged
12:      else
13:         $\lambda_k \leftarrow u_k / (u_k - l_k)$ ;  $\mu_k \leftarrow -u_k l_k / (2(u_k - l_k))$ 
14:         $c_k \leftarrow \lambda_k c_k + \mu_k$ ;  $g_{k,:} \leftarrow \lambda_k g_{k,:}$ 
15:        append new generator column to  $G$  with  $\mu_k$  at row  $k$ , zero elsewhere
16:      end if
17:    end for
18:    Export bounds  $(l_{m,k}^{N_1}, u_{m,k}^{N_1}) = (c_k - \sum_j |g_{k,j}|, c_k + \sum_j |g_{k,j}|)$  before the relaxation overwrite
19:  end if
20: end for
21: Inject  $\{(l_{m,k}^{N_1}, u_{m,k}^{N_1})\}$  as big- $M$  constants when emitting  $N_1$ 's MIP

```

Variables / constraints added. None added *at this stage*. The output is a table of tighter intervals that *replace* the loose big- M constants $\underline{z}, \bar{z}$ in eqs. (3)–(4) when the MIP is built. Stable neurons additionally drop their binary $a_{m,k}$ before encoding.

Worked example. Running network with $\varepsilon = 0.1$.

Initialisation. $\mathbf{c}_0 = (0.5, 0.5)$, $G_0 = \text{diag}(0.6, 0.6)$ (one column per input generator; hull = $[-0.1, 1.1]^2$).

Affine update at the hidden layer.

$$\mathbf{c}_1 = W_h \mathbf{c}_0 + b_h = (1 \cdot 0.5 + 1 \cdot 0.5 - 0.3, -1 \cdot 0.5 + 1 \cdot 0.5 - 0.1) = (0.7, -0.1),$$

$$G_1 = W_h G_0 = \begin{pmatrix} 1 \cdot 0.6 & 1 \cdot 0.6 \\ -1 \cdot 0.6 & 1 \cdot 0.6 \end{pmatrix} = \begin{pmatrix} 0.6 & 0.6 \\ -0.6 & 0.6 \end{pmatrix}.$$

Per-neuron hull at the hidden ReLU (exported before relaxation).

$$\begin{aligned}(l_{1,1}^{N_1}, u_{1,1}^{N_1}) &= (0.7 - 1.2, 0.7 + 1.2) = (-0.5, 1.9), \\(l_{1,2}^{N_1}, u_{1,2}^{N_1}) &= (-0.1 - 1.2, -0.1 + 1.2) = (-1.3, 1.1).\end{aligned}$$

Both neurons are split, so the DeepZ relaxation now scales each row of G_1 by $\lambda_k = u_k / (u_k - l_k)$ and appends a new μ_k generator column:

$$\begin{aligned}\lambda_1 &= 1.9 / 2.4 \approx 0.792, \quad \mu_1 = -1.9 \cdot (-0.5) / (2 \cdot 2.4) \approx 0.198 \\ \lambda_2 &= 1.1 / 2.4 \approx 0.458, \quad \mu_2 = -1.1 \cdot (-1.3) / (2 \cdot 2.4) \approx 0.298.\end{aligned}$$

Why no tightening over interval arithmetic here. On this single hidden layer the hull endpoints coincide with the plain-interval ones, because each generator column came from a distinct input coordinate and the rows of W_h act on each independently. The zonotope wins starting at the *second* affine layer: there, the column $W'g_{:,j}$ collapses signed contributions across k' , giving a half-width $|\sum_{k'} w'_{k,k'} g_{m-1,k',j}|$ that is $\leq$ the interval-arithmetic $\sum_{k'} |w'_{k,k'} g_{m-1,k',j}|$, with strict inequality when signs differ (Theorem 4).

3.3 Per-Copy Triangle Drop (**BoundTightPertRelax**)

Single-network analogue of: Section 4.4. **What changes vs. the advstd version:** the per-copy bounds fed to the gap formula come from Source B alone (Section 3.2) rather than the intersection of Source A (diff from N_1 to N_2) and Source B. Everything else is unchanged: the gap formula $\text{gap}(l, u) = u |l| / (2(u - l))$, the tiered per-copy decision rule (*both tiny* / *one tiny* / *neither tiny*), the triangle replacement that drops the binary, and the soundness argument $\delta_{\text{relaxed}} \geq \delta_{\text{exact}}$ are exactly as in Section 4.4.

The org/pert asymmetry that motivates the per-copy rule is present in standard mode too: N_1 encodes both copies, the pert copy’s input box $[-\varepsilon, 1+\varepsilon]^n$ is strictly wider than the clean copy’s $[0, 1]^n$, so its bounds are looser and its gap is bigger — exactly the asymmetry that makes the “one-thin” tier informative.

The combination with perturbed-interval coupling (Section 3.1) is required for soundness on the “one-thin” tier for the same reason as in advstd: without the coupling, a relaxed copy’s post-ReLU value can drift away from its exact sibling in the LP relaxation. The unsound-combination guard at `run.jl:693--702` fires regardless of mode.

3.3.1 Algorithm and worked example

Algorithm 3 Per-Copy Triangle Drop on N_1

Require: N_1 with dual (org/pert) encoding, threshold $\tau \geq 0$, per-copy bounds $(l_{m,k}^c, u_{m,k}^c)$ from Section 3.2 (or interval arithmetic)

```

1: for each ReLU neuron  $(m, k)$  split on  $\geq 1$  copy do
2:    $g^c \leftarrow \text{gap}(l_{m,k}^c, u_{m,k}^c)$  for  $c \in \{\text{org}, \text{pert}\}$  (eq. 30)
3:   if  $\max(g^{\text{org}}, g^{\text{pert}}) \leq \tau$  then
4:      $R \leftarrow \{\text{org}, \text{pert}\}$  {both-thin tier}
5:   else if  $\min(g^{\text{org}}, g^{\text{pert}}) \leq \tau$  then
6:      $R \leftarrow \{\arg \min_c g^c\}$  {one-thin tier}
7:   else
8:      $R \leftarrow \emptyset$  {neither-thin tier}
9:   end if
10:  for all  $c \in R$  do
11:    drop binary  $a_{m,k}^c$  and the two big- $M$  rows eqs. (3)–(4)
12:    emit triangle rows eqs. (5)–(7) on  $[l_{m,k}^c, u_{m,k}^c]$ 
13:  end for
14: end for

```

Variables / constraints (per flagged (m, k, c)). *Removed:* 1 binary $a_{m,k}^c$, 2 big- M rows (eqs. 3–4). *Added:* 3 triangle rows (eqs. 5–7) on the same $\hat{z}_{m,k}^c, z_{m,k}^{+,c}$ variables. No new continuous variable.

Worked example. Running network with $\varepsilon = 0.1$. Per-copy bounds (single-network setting; pert copy inherits $[-\varepsilon, 1 + \varepsilon]^2$ as its input box):

$$(l, u)_{1,1}^{\text{org}} = (-0.3, 1.7), \quad (l, u)_{1,2}^{\text{org}} = (-1.1, 0.9), \quad (l, u)_{1,1}^{\text{pert}} = (-0.5, 1.9), \quad (l, u)_{1,2}^{\text{pert}} = (-1.3, 1.1).$$

Apply $\text{gap}(l, u) = u |l| / (2(u - l))$:

neuron	g^{org}	g^{pert}	tier at $\tau = 0.15$	tier at $\tau = 0.25$
(1, 1)	0.128	0.198	one thin (org)	both thin
(1, 2)	0.248	0.298	neither	one thin (org)

Effect at $\tau = 0.15$. The MIP loses 1 binary ($a_{1,1}^{\text{org}}$), drops 2 big- M rows on the org copy of neuron (1, 1), and adds 3 triangle rows on $[-0.3, 1.7]$ to that copy. The other three (neuron, copy) combinations keep their exact big- M encoding.

Effect at $\tau = 0.25$. 3 binaries dropped ($a_{1,1}^{\text{org}}, a_{1,1}^{\text{pert}}, a_{1,2}^{\text{org}}$), 6 big- M rows removed, 9 triangle rows added. The reported δ is a sound upper bound on δ_{exact} because each emitted triangle envelopes the exact ReLU image on the same interval used to gate the binary’s encoding.

3.4 Sibling-Gated Refinement (SibGate)

Single-network analogue of: Section 4.5. **What changes vs. the advstd version:** the unconditional per-copy bounds $[l_{m,k}^{N_1,c}, u_{m,k}^{N_1,c}]$ and $[l_{m,k}^{N_1,\text{pre}}, u_{m,k}^{N_1,\text{pre}}]$ come from Source B (Section 3.2) when `--nn1_zono_bounds=true`, or from the encoder’s default interval arithmetic on N_1 otherwise — the latter is the same fall-back documented for Per-Copy Triangle Drop in Algorithm 3, and is sound but strictly looser; the pre-activation diff bound $[l_{\text{int}}, u_{\text{int}}]$ is the pure org-pert perturbation

interval — exactly the `pert_up/pert_down` recurrence described in Section 3.1 — not the composed N_1 -to- N_2 bound used in `advstd`. Both are quantities that already exist in standard mode; no N_1 -primal or weight-drift information is needed.

With those two substitutions, the conditional triangle of Eqs. (33)–(35), the “both thin” pre-activation coupling of Eq. (38), and the soundness argument carry over verbatim from Section 4.5. The single-network setting is in fact the *cleaner* setting for SibGate: the caveat noted in Section 4.5 about the comp-bound globals carrying N_1 -to- N_2 weight drift on top of the org-pert perturbation (IMPORTANT CAVEAT paragraph) does not arise, because there is no N_1 -to- N_2 diff to bundle in.

3.4.1 Algorithm and worked example

Algorithm 4 Sibling-Gated Refinement on N_1

Require: N_1 , per-copy bounds $(l_{m,k}^c, u_{m,k}^c)$ from Section 3.2 (or interval arithmetic), pre-act diff bound $(l_{\text{int}}, u_{\text{int}})_{m,k}$ from Section 3.1’s recurrence at the pre-ReLU site, tier assignment from Algorithm 3

```

1: for each ReLU neuron  $(m, k)$  do
2:   if tier is both thin then
3:     keep the two simple triangles already emitted by Algorithm 3
4:     emit pre-act coupling row eq. (38):
        $l_{\text{int}} \leq \hat{z}_{m,k}^{\text{pert}} - \hat{z}_{m,k}^{\text{org}} \leq u_{\text{int}}$ 
5:   else if tier is one thin, relaxed copy  $c$ , kept sibling pre then
6:     compute  $l_A, u_A, l_I, u_I$  via eqs. (36)–(37)
7:      $M \leftarrow u_{m,k}^c + |l_{m,k}^c|$ 
8:     replace the simple triangle on copy  $c$  with the conditional triangle eqs. (33)–(35), gated on
       existing sibling binary  $a_{m,k}^{\text{pre}}$ 
9:   else
10:    leave both copies with their exact big- $M$  encoding
11:   end if
12: end for

```

Variables / constraints added per tier. *Both thin:* 1 new linear row (pre-act coupling) on top of Per-Copy Triangle Drop’s 6 triangle rows; no new variable. *One thin:* 2 lower-bound rows + 2 conditional upper-bound rows on copy c (each gated on $a_{m,k}^{\text{pre}}$ via a big- M slack); no new variable. *Neither:* unchanged.

Worked example (one-thin tier). Same per-copy bounds as in Section 3.3.1. At $\tau = 0.15$, neuron (1, 1) is one-thin with $c = \text{org}$ relaxed and $\text{pre} = \text{pert}$ kept.

Pre-act diff bound at neuron (1, 1). From Section 3.1.8’s recurrence at the pre-activation site: $(l_{\text{int}}, u_{\text{int}}) = (-0.2, +0.2)$.

Apply eqs. (36)–(37).

$$\begin{aligned}
l_A &= \max(l_{\text{int}}, l_{1,1}^{\text{org}}) = \max(-0.2, -0.3) = -0.2 \\
u_A &= \min(u_{1,1}^{\text{pert}} + u_{\text{int}}, u_{1,1}^{\text{org}}) = \min(1.9 + 0.2, 1.7) = 1.7 \\
l_I &= \max(l_{1,1}^{\text{pert}} + l_{\text{int}}, l_{1,1}^{\text{org}}) = \max(-0.5 + (-0.2), -0.3) = -0.3 \\
u_I &= \min(u_{\text{int}}, u_{1,1}^{\text{org}}) = \min(0.2, 1.7) = 0.2.
\end{aligned}$$

$$M = u_{1,1}^{\text{org}} + |l_{1,1}^{\text{org}}| = 1.7 + 0.3 = 2.0.$$

Comparison with Per-Copy Triangle Drop’s simple triangle. On the active-sibling branch ($a_{1,1}^{\text{pert}} = 1$), SibGate’s envelope uses $[l_A, u_A] = [-0.2, 1.7]$ vs the unconditional $[-0.3, 1.7]$. On the inactive-sibling branch ($a_{1,1}^{\text{pert}} = 0$), the envelope is $[l_I, u_I] = [-0.3, 0.2]$ vs $[-0.3, 1.7]$. Both are subintervals of the Per-Copy Triangle Drop interval, so the LP relaxation is strictly tighter on every node of Gurobi’s branch tree that fixes $a_{1,1}^{\text{pert}}$.

4 Boosting Advanced Standard Mode ($N_1 \rightarrow N_2$ transfer)

4.1 Two-Phase Execution

Advanced standard mode runs in two phases, possibly in separate processes:

Phase 1 — $N1$ solve

For each (c_s, c_t) class pair, solve the standard-mode MIP for N_1 to either optimality or a user-specified timeout. Extract:

- The full primal solution $(\text{var}_i, \text{val}_i)_{i=1}^{|V|}$.
- The per-neuron interval bounds $(l_{m,k}, u_{m,k})$ on all split ReLUs (the `layers.info` dictionary).

Serialise to disk. The resulting `n1_state` directory is a self-contained snapshot of “everything we learned while solving N_1 ”.

Phase 2 — $N2$ sweep

For each (c_s, c_t) pair, build the standard-mode MIP for N_2 , load the N_1 state, apply each enabled technique, and call `optimize!`. Phase 2 never re-solves N_1 .

The safe combinations use exactly three techniques. Three techniques from the original `advstd` toolchain — MIP Start, LP Basis, and Branch Priorities — are **not** used in any safe combination and are therefore omitted from this document.

Correctness invariant

Let $\mathcal{F}_2$ denote the feasible set of N_2 ’s standard-mode MIP and let

$$\delta_{\text{exact}} \triangleq \delta_{N_2}^* = \max_{x \in \mathcal{F}_2} \text{obj}_{N_2}(x).$$

For every subset of techniques, the optimal value reported by the solver when it terminates at `OPTIMAL` is exactly δ_{exact} . The techniques only change the *path* Gurobi takes through the branch-and-bound tree, not the set of feasible solutions or the objective function.

4.2 Bound Tightening (`boundTight`)

Flag: `--adv_std_bound_tightening` (always true in safe combos). **Model modification:** tightens neuron bounds before the N_2 MIP is built.

This technique runs *before* the MIP is constructed. Its output — tighter per-neuron intervals — feeds into the MIP encoding itself (smaller big- M constants, fewer binary variables). The other techniques (branch priorities, variable hints, triangle relaxation) operate *on* the already-built MIP.

Bound Tightening is independent of the *perturbed interval constraints* that are added to the MIP after construction (see the note at the end of this subsection).

Goal. For each ReLU neuron (m, k) in N_2 , compute a sound interval $[d_{m,k}^\downarrow, d_{m,k}^\uparrow]$ on the *weight-drift difference* $\hat{z}_{m,k}^{N_2} - \hat{z}_{m,k}^{N_1}$ (how much the pre-activation changed due to N_2 having different weights than N_1). Once we have this interval, we combine it with N_1 's known bounds from Phase 1 to get N_2 bounds:

$$l_{m,k}^{N_2} \geq l_{m,k}^{N_1} + d_{m,k}^\downarrow, \quad u_{m,k}^{N_2} \leq u_{m,k}^{N_1} + d_{m,k}^\uparrow.$$

When a neuron's tightened interval lies strictly in $\mathbb{R}_{<0}$ or $\mathbb{R}_{\geq 0}$, the binary $a_{m,k}$ is fixed and removed from the MIP.

There are two methods to compute $[d_{m,k}^\downarrow, d_{m,k}^\uparrow]$. The safe combinations use one of these two, controlled by the `zono` flag.

4.2.1 Method 1: Interval arithmetic (baseline)

Used when: `zono=false`.

This is the default method. From Phase 1 we have N_1 's per-neuron bounds $(l_{m,k}^{N_1}, u_{m,k}^{N_1})$, so the post-ReLU range is $z_{m,k}^{N_1} \in [\max(0, l_{m,k}^{N_1}), \max(0, u_{m,k}^{N_1})]$.

Weight-drift difference. Because N_1 and N_2 share the same architecture, every neuron (m, k) in N_2 has a corresponding neuron in N_1 . The *difference* in their pre-activations, $z_{m,k}^{N_2} - z_{m,k}^{N_1}$, depends only on the weight difference $\Delta W = W_2 - W_1$ (the “weight drift” from training). Sound intervals on this difference are computed layer by layer using interval arithmetic.

Affine layers. Let $\Delta w_{m,k} = w_{m,k}^2 - w_{m,k}^1$ and $\Delta b_{m,k} = b_{m,k}^2 - b_{m,k}^1$ be the weight and bias differences at layer m . From Phase 1 we have N_1 's per-neuron bounds $(l_{m,k}^{N_1}, u_{m,k}^{N_1})$ for every ReLU neuron. Since the post-ReLU output is $z_{m,k} = \max(0, \hat{z}_{m,k})$, the post-ReLU range for each neuron k in layer $m-1$ is $z_{m-1,k}^{N_1} \in [\max(0, l_{m-1,k}^{N_1}), \max(0, u_{m-1,k}^{N_1})]$.

The pre-activation difference at neuron k of layer m decomposes as:

$$\hat{z}_{m,k}^{N_2} - \hat{z}_{m,k}^{N_1} = \underbrace{\sum_{k'} w_{m,k,k'}^2 (z_{m-1,k'}^{N_2} - z_{m-1,k'}^{N_1})}_{\text{propagated diff from previous layer}} + \underbrace{\sum_{k'} \Delta w_{m,k,k'} \cdot z_{m-1,k'}^{N_1} + \Delta b_{m,k}}_{\text{direct weight-drift contribution}}.$$

Interval arithmetic bounds each term per output neuron k :

$$[d_{m,k}^\downarrow, d_{m,k}^\uparrow] = \sum_{k'} w_{m,k,k'}^2 \cdot [d_{m-1,k'}^\downarrow, d_{m-1,k'}^\uparrow] + \sum_{k'} \Delta w_{m,k,k'} \cdot [\max(0, l_{m-1,k'}^{N_1}), \max(0, u_{m-1,k'}^{N_1})] + \Delta b_{m,k},$$

ReLU layers. The ReLU difference satisfies $\sigma(z+d) - \sigma(z) \in [\min(0, d^\downarrow), \max(0, d^\uparrow)]$, so the diff interval is clipped: $d^\uparrow \leftarrow \max(0, d^\uparrow)$ and $d^\downarrow \leftarrow -\max(0, -d^\downarrow)$.

The final output at each layer is a sound interval:

$$d_{m,k}^\downarrow \leq z_{m,k}^{N_2} - z_{m,k}^{N_1} \leq d_{m,k}^\uparrow.$$

These are then combined with N_1 's known bounds to get N_2 bounds:

$$l_{m,k}^{N_2} \geq l_{m,k}^{N_1} + d_{m,k}^\downarrow, \quad u_{m,k}^{N_2} \leq u_{m,k}^{N_1} + d_{m,k}^\uparrow.$$

This method is fast but loose: it treats every neuron as independent, losing all correlations between neurons in the same layer.

4.2.2 Method 2: Zonotope arithmetic (Source A + B)

Used when: `zono=true`. **Flag:** `--adv_std_zono_bounds`.

This method computes the same $[d_{m,k}^\downarrow, d_{m,k}^\uparrow]$ as Method 1, but using *zonotope arithmetic* instead of interval arithmetic. A zonotope tracks correlations between neurons by representing each neuron's value as a linear combination of shared noise variables, producing tighter bounds. Additionally, a second pass (Source B) independently bounds N_2 's absolute activations and intersects the result with the diff-based bounds.

Source A — Zonotope diff bounds. Each neuron's difference is represented as a zonotope — a centre plus generators sharing noise variables $\epsilon_j \in [-1, 1]$ across all neurons in the layer. We store a centre vector $\mathbf{c}_m \in \mathbb{R}^K$ and a generator matrix $G_m \in \mathbb{R}^{K \times p}$ (rows = K neurons, columns = p generators, where p is the number of columns accumulated so far — initially $p = K_0$ at the input layer, and each affine layer adds K' columns while each ReLU adds one column per split neuron):

$$d_{m,k} = c_{m,k} + \sum_j g_{m,k,j} \epsilon_j, \quad d_{m,k}^\downarrow = c_{m,k} - \sum_j |g_{m,k,j}|, \quad d_{m,k}^\uparrow = c_{m,k} + \sum_j |g_{m,k,j}|.$$

Shared ϵ_j variables encode correlations: interval arithmetic is the special case where G_m is diagonal (no correlations).

Initialisation. At the input layer, N_1 and N_2 receive the same input, so the diff is zero: $\mathbf{c}_0 = \mathbf{0}$, $G_0 = \mathbf{0}_{K_0 \times 0}$ (no generators yet, $p = 0$). The first affine update creates the initial generators via G'_{indep} .

Affine layer update. Let $K' =$ neurons in layer $m-1$, $K =$ neurons in layer m :

$$c_{m,k} = \sum_{k'} w_{m,k,k'}^2 c_{m-1,k'} + \sum_{k'} \Delta w_{m,k,k'} \cdot \text{mid}_{k'} + \Delta b_{m,k} \quad (23)$$

$$\underbrace{G'_{\text{corr}}}_{K \times p} = \underbrace{W_2}_{K \times K'} \underbrace{G_{m-1}}_{K' \times p} \quad (24)$$

$$\underbrace{G'_{\text{indep}}}_{K \times K'} = \underbrace{\Delta W}_{K \times K'} \cdot \underbrace{\text{diag}(\text{rad}_1, \dots, \text{rad}_{K'})}_{K' \times K'} \quad (25)$$

$$\underbrace{G_m}_{K \times (p+K')} = [G'_{\text{corr}} \mid G'_{\text{indep}}] \quad (26)$$

where $\text{mid}_{k'}$ and $\text{rad}_{k'}$ are the midpoint and half-width of N_1 's post-ReLU interval at neuron k' : $\text{mid}_{k'} = \frac{1}{2}(\max(0, u_{m-1,k'}^{N_1}) + \max(0, l_{m-1,k'}^{N_1}))$, $\text{rad}_{k'} = \frac{1}{2}(\max(0, u_{m-1,k'}^{N_1}) - \max(0, l_{m-1,k'}^{N_1}))$.

After the affine update, the column count grows from p to $p + K'$.

ReLU layer update (DeepZ). Compute hull per neuron: $l_k = c_{m,k} - \sum_j |g_{m,k,j}|$, $u_k = c_{m,k} + \sum_j |g_{m,k,j}|$.

- $u_k \leq 0$: $c_{m,k} \leftarrow 0$, $g_{m,k,\cdot} \leftarrow 0$.
- $l_k \geq 0$: unchanged.
- Split ($l_k < 0 < u_k$): $\lambda_k = \frac{u_k}{u_k - l_k}$, $\mu_k = -\frac{u_k \cdot l_k}{2(u_k - l_k)}$,

$$c_{m,k} \leftarrow \lambda_k c_{m,k} + \mu_k, \quad g_{m,k,\cdot} \leftarrow \lambda_k g_{m,k,\cdot},$$

append column to G_m : value μ_k at row k , zero elsewhere.

After the ReLU update, $p \leftarrow p + |\{k : l_k < 0 < u_k\}|$ (one new column per split neuron).

So for a network with L affine layers of widths $K_1, \dots, K_L$ and s_m split neurons at ReLU layer m :

$$p_{\text{final}} = K_0 + \sum_{m=1}^L (K_{m-1} + s_m).$$

Source B — N1-tightened absolute N2 zonotope. A separate zonotope propagates N_2 's *absolute* pre-activations (not the diff) through N_2 alone, using the same zonotope machinery as Source A but applied to N_2 's own values. Initialise with a single zonotope whose hull covers both the clean input $v_{\text{in}} \in [0, 1]^n$ and the perturbed input $v_{x_0} = v_{\text{in}} + v_e \in [-\varepsilon, 1 + \varepsilon]^n$ simultaneously:

$$\mathbf{c}_0^{\text{B}} = \frac{1}{2}\mathbf{1}, \quad G_0^{\text{B}} = \text{diag}(\frac{1}{2} + \varepsilon, \dots, \frac{1}{2} + \varepsilon) \in \mathbb{R}^{n \times n},$$

so the initial hull is $[-\varepsilon, 1 + \varepsilon]^n$ rather than $[0, 1]^n$. This widens the input box by ε on each side relative to the clean-only range, which is the price of using one zonotope to bound N_2 's pre-activations under both the clean and perturbed inputs (rather than propagating two separate zonotopes and taking their union). Propagate through N_2 's affine layers ($c \leftarrow W_2 c + b_2$, $G \leftarrow W_2 G$) and DeepZ ReLU relaxations exactly as in Source A. At each ReLU layer, the zonotope hull gives per-neuron bounds:

$$u_{m,k}^{\text{hull}} = c_{m,k}^{\text{B}} + \sum_j |g_{m,k,j}^{\text{B}}|, \quad l_{m,k}^{\text{hull}} = c_{m,k}^{\text{B}} - \sum_j |g_{m,k,j}^{\text{B}}|.$$

These are then *intersected* with Source A's diff-based bounds:

$$u_{m,k}^{N_2} = \min(u_{m,k}^{\text{hull}}, u_{m,k}^{N_1} + d_{m,k}^{\uparrow}), \quad l_{m,k}^{N_2} = \max(l_{m,k}^{\text{hull}}, l_{m,k}^{N_1} + d_{m,k}^{\downarrow}). \quad (27)$$

The first term in each min/max comes from Source B's own zonotope pass on N_2 ; the second term comes from Source A's diff bounds applied to N_1 's known bounds (the same formula as in the Goal paragraph above). Taking the tighter of the two at each neuron is sound because both are valid over-approximations.

The DeepZ ReLU relaxation is then applied using these tightened bounds, so Source B's zonotope passes through a narrower "tube" at each layer.

Conv layers: interval, not zonotope. The affine recurrence written above ($c \leftarrow Wc + b$, $G \leftarrow WG$) matches `Linear` layers exactly. For `Conv2d` layers, both Source A (`compute_diff_bounds_zonotope`) and Source B (`compute_n2_bounds_zonotope_with_n1_tighten`) in `utils/perturbation_intervals.jl` currently fall back on *interval arithmetic* (the `else` branch at lines 1883–1894) because the `advstd`-mode wiring in `main.advanced_standard` sets only the `use_zonotope` global (`run.jl`, line 807), not `zonotope_conv`. The zonotope therefore activates at the first `Linear` layer encountered after the conv stack: the running interval state at that point is converted into a zonotope with diagonal G_0 (line 1844 for Source B, analogous site for Source A) and the affine/ReLU recurrence above takes over from there. Concretely, on a CNN `Conv`→`ReLU`→`Conv`→`ReLU`→`Flatten`→`Linear`→`ReLU`→`Linear`, the conv-stage ReLUs receive Source-A/B bounds tightened by `interval_conv2d_bounds` only, while the fully-connected stage ReLUs receive the zonotope-tightened bounds described above.

The fall-back is sound: interval arithmetic is itself a sound abstract interpretation, the conv-stage hull at every ReLU contains all realisable pre-activations, and the intersection (Eq. 27) of two sound over-approximations is itself sound. But the conv-stage bounds are strictly looser than what a full zonotope-conv pass would produce, so Theorem 4 below applies layer-by-layer only from the first `Linear` layer onwards; on conv layers Source A collapses to Method 1 (interval

arithmetic), and Source B is no tighter than its interval-conv hull. Full zonotope-conv is implemented by `conv2d_zonotope` (`utils/perturbation_intervals.jl`) and currently activated only when `zonotope_conv=true`; today that flag is set inside `main.transfer` (`run.jl`, line 1434) but not by the `advstd-mode` or `standard-mode` wirings. The standard-mode analogue (Section 3.2) inherits exactly the same restriction. Lifting this restriction is a pure-precision upgrade with no soundness implication and is left as future work.

4.2.3 Precision guarantees

The two methods are ordered in precision, and Source A and Source B within Method 2 are incomparable.

Theorem 4 (Zonotope diff dominates interval diff). *Let $[d_{m,k}^{\downarrow,I}, d_{m,k}^{\uparrow,I}]$ denote the diff bound produced by Method 1 (interval arithmetic, Section 4.2.1) and $[d_{m,k}^{\downarrow,Z}, d_{m,k}^{\uparrow,Z}]$ the diff bound produced by Source A of Method 2 (zonotope, Section 4.2.2) when both start from the same N_1 state. Then for every ReLU neuron (m, k) ,*

$$d_{m,k}^{\downarrow,I} \leq d_{m,k}^{\downarrow,Z} \leq \hat{z}_{m,k}^{N_2} - \hat{z}_{m,k}^{N_1} \leq d_{m,k}^{\uparrow,Z} \leq d_{m,k}^{\uparrow,I}.$$

The inequality is strict whenever some intermediate affine layer combines two generators whose signs disagree after multiplication by the layer’s weights.

Proof sketch. Interval arithmetic is the special case of zonotope arithmetic in which the generator matrix G_m is restricted to be diagonal (each neuron has its own independent noise variable). Zonotope preserves correlations: when a subsequent affine layer computes $\sum_{k'} w_{k'} z_{m-1,k'}$, the shared noise ϵ_j combines as $(\sum_{k'} w_{k'} g_{m-1,k',j}) \epsilon_j$, contributing $|\sum_{k'} w_{k'} g_{m-1,k',j}|$ to the half-width. Interval arithmetic discards the correlation and contributes $\sum_{k'} |w_{k'} g_{m-1,k',j}|$ instead. The triangle inequality gives the former $\leq$ the latter, with equality iff all $w_{k'} g_{m-1,k',j}$ share a sign. Inducting on depth and applying the sound DeepZ ReLU relaxation at each layer yields the claim. The ReLU relaxation $\lambda_k z + \mu_k$ is affine with $\lambda_k \in [0, 1]$ and therefore preserves width ordering. $\square$

Corollary 5 (Precision gap grows with depth and drift). *Let $w_{m,k}^\bullet \triangleq d_{m,k}^{\uparrow,\bullet} - d_{m,k}^{\downarrow,\bullet}$ denote the width of the diff bound at neuron (m, k) under Method $\bullet \in \{I, Z\}$. The width gap $w_{m,k}^I - w_{m,k}^Z$ is non-decreasing in m and grows monotonically with $\|\Delta W\|_F$.*

Theorem 6 (Source A and Source B are incomparable). *Let $[l_{m,k}^A, u_{m,k}^A] \triangleq [l_{m,k}^{N_1} + d_{m,k}^{\downarrow}, u_{m,k}^{N_1} + d_{m,k}^{\uparrow}]$ be the Source-A N_2 bound and $[l_{m,k}^B, u_{m,k}^B] \triangleq [l_{m,k}^{\text{hull}}, u_{m,k}^{\text{hull}}]$ the Source-B bound.*

- (a) *There exist instances on which $u^A < u^B$ (equivalently $l^A > l^B$) strictly, and instances on which the opposite strict inequality holds.*
- (b) *The intersection bound $[\max(l^A, l^B), \min(u^A, u^B)]$ (Eq. 27) has width strictly less than $\min(w^A, w^B)$ on any neuron where the two sources disagree in orientation (i.e. $u^A < u^B$ at the upper end and $l^A > l^B$ at the lower end, or vice-versa).*

Proof sketch. (a) *A tighter than B.* Take $N_1 = N_2$ and a generic $\varepsilon > 0$. Then $\Delta W = 0$, so every Source-A generator column $G'_{\text{indep}} = 0$ and the diff zonotope remains zero, giving $[d^{\downarrow}, d^{\uparrow}] = 0$ and $l^A = l^{N_1}$, $u^A = u^{N_1}$ (i.e. N_1 ’s MIP-level bounds). Source B propagates the wider hull $[-\varepsilon, 1 + \varepsilon]^n$ through DeepZ relaxations, which discard ReLU correlations and accumulate error, so $l^B \leq l^A - c_\varepsilon$ and $u^B \geq u^A + c_\varepsilon$ for some $c_\varepsilon > 0$ on any neuron with a split ancestor.

B tighter than A. Take an N_1 whose Phase-1 solve timed out before closing the root relaxation, so its MIP bounds are vacuous: $l_{m,k}^{N_1} = -\infty$ (in practice, the big- M input bound). Source A inherits

these, giving l^A, u^A effectively vacuous. Source B is independent of N_1 's MIP state and produces finite bounds from N_2 's weights alone.

(b) Immediate from the min/max definition: when $u^A < u^B$ and $l^A > l^B$ the intersection keeps u^A and l^A (width w^A); when the sources disagree in orientation, say $u^A < u^B$ and $l^B > l^A$, the intersection is $[l^B, u^A]$ with width strictly less than both w^A and w^B . $\square$

Implication for the safe combinations. Theorem 4 explains the empirical pattern that `zono=true` never *loses* precision relative to `zono=false` but may cost more to compute. The incomparability in Theorem 6 explains why the Source A + Source B intersection is always at least as useful as either alone, and why on benchmarks where N_1 is solved to optimality the benefit of Source B over Source A alone is small, whereas on timed-out N_1 instances it is large.

4.2.4 Final bound integration

When N_2 's MIP is built, each ReLU neuron's bounds are the tightest (intersection) of all available sources:

$$u_{m,k}^{N_2} \leftarrow \min(u_{m,k}^{N_1} + d_{m,k}^\uparrow, u_{m,k}^B), \quad (28)$$

$$l_{m,k}^{N_2} \leftarrow \max(l_{m,k}^{N_1} + d_{m,k}^\downarrow, l_{m,k}^B), \quad (29)$$

Terms from disabled methods are simply omitted from the min/max. When the final interval is single-signed ($u_{m,k}^{N_2} \leq 0$ or $l_{m,k}^{N_2} \geq 0$), the associated binary is eliminated from the MIP entirely.

Soundness. Both methods produce sound over-approximations:

- Method 1 (interval arithmetic): standard interval arithmetic rules over-approximate the true range at each layer.
- Method 2 (zonotope): zonotope arithmetic is a sound abstract interpretation; Source B's intersection of two sound over-approximations is itself sound.

Every resulting bound is satisfied by every $x \in \mathcal{F}_2$, so x^* remains feasible and δ_{exact} is preserved.

Summary of what operates where:

- **Bound Tightening** (bound tightening, Sources A/B): runs *before* MIP construction. Output: tighter per-neuron intervals $\rightarrow$ smaller big- M constants, fewer binaries.
- **Perturbed interval constraints** (Section 3.1): runs *after* MIP construction. Output: additional linear constraints linking the org and perturbation copies of N_2 (Eqs. (8)–(9)).
- Both are always active in the safe combinations and complement each other.

4.2.5 Algorithm and worked example

Algorithm 5 Bound Tightening on N_2 (Method 1 and Method 2)

Require: N_1 bounds $\{(l_{m,k}^{N_1}, u_{m,k}^{N_1})\}$ from Phase 1, weights W_1, W_2 , biases b_1, b_2 , mode $\text{ZONO} \in \{\text{false}, \text{true}\}$

- 1: $\Delta W \leftarrow W_2 - W_1, \Delta b \leftarrow b_2 - b_1$
- 2: **if** $\text{ZONO} = \text{false}$ **then**
- 3: Run interval recurrence (Section 4.2.1) on the diff $\hat{z}_{m,k}^{N_2} - \hat{z}_{m,k}^{N_1}$ using $W_2, \Delta W, \Delta b$ and N_1 post-ReLU intervals; obtain $(d_{m,k}^\downarrow, d_{m,k}^\uparrow)$
- 4: **else**
- 5: **Source A:** run zonotope diff recurrence eqs. (23)–(26) with DeepZ ReLU step; extract $(d_{m,k}^\downarrow, d_{m,k}^\uparrow)$ and $(l_{m,k}^A, u_{m,k}^A) = (l_{m,k}^{N_1} + d_{m,k}^\downarrow, u_{m,k}^{N_1} + d_{m,k}^\uparrow)$
- 6: **Source B:** initialise $\mathbf{c}_0^B = \frac{1}{2}\mathbf{1}, G_0^B = (\frac{1}{2} + \varepsilon)I_n$; propagate through N_2 with W_2 ; at each ReLU layer extract hull endpoints $(l_{m,k}^B, u_{m,k}^B)$ and intersect via eq. (27) before applying DeepZ ReLU
- 7: **end if**
- 8: $u_{m,k}^{N_2} \leftarrow \min(u_{m,k}^{N_1} + d_{m,k}^\uparrow, u_{m,k}^B)$ (eq. 28; second term omitted if $\text{ZONO} = \text{false}$)
- 9: $l_{m,k}^{N_2} \leftarrow \max(l_{m,k}^{N_1} + d_{m,k}^\downarrow, l_{m,k}^B)$ (eq. 29)
- 10: Stable-neuron elimination: if $u_{m,k}^{N_2} \leq 0$ or $l_{m,k}^{N_2} \geq 0$, fix $a_{m,k}$ before emission of $\mathcal{M}_2$
- 11: Inject the tightened $(l_{m,k}^{N_2}, u_{m,k}^{N_2})$ as big- M constants in eqs. (3)–(4) of $\mathcal{M}_2$

Variables / constraints (re)written. No new rows or new variables. Bound Tightening *changes coefficients* of the existing big- M ReLU encoding: the loose $\underline{z}, \bar{z}$ in eqs. (3)–(4) on every split neuron are replaced by the tighter $(l_{m,k}^{N_2}, u_{m,k}^{N_2})$. Stable-neuron binaries $a_{m,k}$ are removed.

Worked example (Method 1 / interval). Take the running network as N_1 , and let N_2 have the same biases but $W_h^{N_2}[1, :] = (1.1, 1)$, i.e. $\Delta W = \begin{pmatrix} 0.1 & 0 \\ 0 & 0 \end{pmatrix}$ and $\Delta b = 0$.

Diff bound at the hidden ReLU. At the input layer $\hat{z}_0^{N_2} - \hat{z}_0^{N_1} = 0$ exactly, so the “propagated” term is zero. The direct weight-drift term is $\sum_{k'} \Delta w_{k,k'} z_{0,k'}^{N_1}$ with $z_{0,k'}^{N_1} \in [0, 1]$:

$$\begin{aligned} [d_{1,1}^\downarrow, d_{1,1}^\uparrow] &= 0.1 \cdot [0, 1] + 0 \cdot [0, 1] = [0, 0.1] \\ [d_{1,2}^\downarrow, d_{1,2}^\uparrow] &= 0 \cdot [0, 1] + 0 \cdot [0, 1] = [0, 0]. \end{aligned}$$

N_2 neuron bounds. Using N_1 interval bounds $(l, u)_{1,1}^{N_1} = (-0.3, 1.7)$ and $(l, u)_{1,2}^{N_1} = (-1.1, 0.9)$ (Section 3.1.8):

$$(l, u)_{1,1}^{N_2} = (-0.3 + 0, 1.7 + 0.1) = (-0.3, 1.8), \quad (l, u)_{1,2}^{N_2} = (-1.1, 0.9).$$

Big- M for $a_{1,1}^{N_2}$ becomes 1.8 on the upper rail and 0.3 on the lower rail (vs. a vacuous big- M of, e.g. 10^4 without tightening). Both neurons remain split on this 1-layer example.

Worked example (Method 2 / zonotope, abbreviated). Source A starts with $\mathbf{c}_0 = 0, G_0 = 0_{2 \times 0}$. The first affine update writes $G_1^{\text{indep}} = \Delta W \cdot \text{diag}(\text{rad}) = \begin{pmatrix} 0.1 & 0 \\ 0 & 0 \end{pmatrix} \cdot \text{diag}(0.5, 0.5) = \begin{pmatrix} 0.05 & 0 \\ 0 & 0 \end{pmatrix}$, $\mathbf{c}_1 = \Delta W \cdot \text{mid} = (0.05, 0)$. Hull at neuron 1: $(0.05 - 0.05, 0.05 + 0.05) = (0, 0.1)$; at neuron 2: $(0, 0)$. Source A reproduces Method 1 on this single hidden layer. Source B’s absolute pass on N_2

produces the hull computed in Section 3.2.1 (with $W_h^{N_2}$ instead of $W_h^{N_1}$); the intersection (eq. 27) at each ReLU keeps the tighter of the two endpoints, which on this 1-layer example is again the Source-A endpoint $(-0.3, 1.8)$ on neuron $(1, 1)$.

4.3 Variable Hints (**varHint=prev/direct/direct_pgd/prev_pgd**)

Flag: `--adv_std_var_hint` $\in \{\text{off, prev, direct, direct_pgd, prev_pgd}\}$ (five-valued; legacy true/false still accepted, mapped to prev/off with a deprecation warning). **Gurobi attributes:** `VarHintVal`, `VarHintPri` (prev/direct) or `Start` via `set_start_value` (prev_pgd/direct_pgd). **Filename tags:** `_varHintFixed` (prev), `_varHintDirect` (direct), `_varHintDirectPGD` (direct_pgd), `_varHintPrevPGD` (prev_pgd). **Retired:** the previously opt-in `--adv_std_var_hint_fix` flag has been merged into `--adv_std_var_hint`; the “fix” (filtering N_1 pseudocosts down to N_2 -surviving binaries before ranking) is now the default and only behavior. Pre-merger result files tagged `_varHint` or `_varHint_varHintFix` are reported as `vh_legacy` by the post-processing extractor and excluded from the auto-tex tables (they are not directly comparable to the merged-behavior runs).

Background: how Gurobi solves a MIP. When Gurobi solves N_2 ’s MIP, it uses *branch-and-bound*: it maintains a search tree where each node is a sub-problem with some binaries fixed and the rest relaxed to continuous $[0, 1]$. At each node, Gurobi solves an LP relaxation of that sub-problem. The LP solution gives fractional values for the unfixed binaries (e.g. $a_{m,k} = 0.37$). Gurobi then needs to decide:

1. *Which* fractional binary to branch on next.
2. *Which direction* to try first ($a_i = 0$ or $a_i = 1$).
3. Whether to run *heuristics* (RINS, diving, rounding) that try to round the fractional LP solution into a feasible integer solution (an “incumbent”).

Variable hints influence all three decisions as soft suggestions.

What the two hint attributes do. For each binary a_i in N_2 ’s MIP, we set:

- **VarHintVal**(a_i): “what value should a_i take?” — Gurobi’s heuristics prefer rounding a_i toward this value.
- **VarHintPri**(a_i): “how confident are we?” — higher priority means Gurobi follows the hint more strongly. Priority 0 means “ignore unless nothing else is available.”

Both are *soft*: they do not add constraints or change the MIP. If the LP relaxation at some node clearly disagrees with the hint, Gurobi overrides it. A wrong hint wastes some effort but cannot change the final answer.

Transfer-probability signal. The right question for the hint is not “how useful was this binary during N_1 ’s branch-and-bound” — it is “*what is the chance that N_1 ’s binary value for this neuron is still correct for N_2 ?*” We build a continuous signal $p_i \in [0, 1]$ that answers this question directly, using only state that N_1 ’s solve and Bound Tightening already produce.

Flag values. The `--adv_std_var_hint` flag takes five values. `off` sets no hint on any binary. The remaining four are the Cartesian product of two independent choices — (i) how the transfer probability p_i is computed (*prev-rule* vs. *direct-rule*), and (ii) which Gurobi channel carries the result (*soft VarHint* vs. *PGD-consensus Start*):

- **prev** — p_i from prev-rule, emitted via **VarHintVal/VarHintPri**.
- **direct** — p_i from direct-rule, emitted via **VarHintVal/VarHintPri**.
- **prev_pgd** — p_i from prev-rule, emitted via **Start** with PGD-consensus filtering (see below). No **VarHintVal/VarHintPri** is written.
- **direct_pgd** — p_i from direct-rule, emitted via **Start** with PGD-consensus filtering. No **VarHintVal/VarHintPri** is written.

The two rules for p_i :

- *prev-rule*: build a heuristic interval $\tilde{I}_i$ by shifting N_1 's pre-activation $\hat{z}_i^{N_1}$ by the diff bound $[d_i^\downarrow, d_i^\uparrow]$ and clipping to $[l_i^{N_2}, u_i^{N_2}]$, then derive p_i from interval-length ratios.
- *direct-rule*: derive p_i directly from Bound Tightening's tightened bounds $[l_i^{N_2}, u_i^{N_2}]$, without constructing $\tilde{I}_i$ or using $\hat{z}_i^{N_1}$.

All four non-off modes share the same hint-value formula (agree with $v_i^{N_1}$ when $p_i \geq 0.5$, flip otherwise); they differ only in how p_i is computed and how the resulting **hint.val** is delivered to Gurobi.

Ingredients. For each ReLU neuron $i = (m, k)$ that survives Bound Tightening's elimination (i.e., $l_{m,k}^{N_2} < 0 < u_{m,k}^{N_2}$):

- $v_i^{N_1} \in \{0, 1\}$ — N_1 's optimal binary (from N_1 's primal). Used by both modes to select N_1 's supporting side of zero.
- $[l_i^{N_2}, u_i^{N_2}]$ — N_2 's final tightened bounds after Bound Tightening's integration step (Section 4.2.4). Used by both modes.
- (*prev only*) $\hat{z}_i^{N_1}$ — N_1 's pre-activation scalar. When $v_i^{N_1} = 1$ this equals N_1 's post-ReLU value z_i^{+, N_1} and is read directly from the primal. When $v_i^{N_1} = 0$ the big- M encoding only constrains $\hat{z}_i^{N_1} \in [l_i^{N_1}, 0]$; the **prev** rule takes its midpoint $l_i^{N_1}/2$ as a proxy.
- (*prev only*) $[d_i^\downarrow, d_i^\uparrow]$ — the weight-drift diff bound from Bound Tightening.

Mode prev — previous rule. Shift N_1 's achieved pre-activation by the diff bound, clip to N_2 's sound bounds, and take p_i as the fraction of the clipped interval on N_1 's supporting side of zero:

$$I_i = [\hat{z}_i^{N_1} + d_i^\downarrow, \hat{z}_i^{N_1} + d_i^\uparrow], \quad \tilde{I}_i = I_i \cap [l_i^{N_2}, u_i^{N_2}],$$

$$p_i = \begin{cases} \frac{\text{length}(\tilde{I}_i \cap (-\infty, 0])}{\text{length}(\tilde{I}_i)} & \text{if } v_i^{N_1} = 0, \\ \frac{\text{length}(\tilde{I}_i \cap [0, +\infty))}{\text{length}(\tilde{I}_i)} & \text{if } v_i^{N_1} = 1. \end{cases}$$

$\tilde{I}_i$ is a heuristic prior on where $\hat{z}_i^{N_2}$ lands under the assumption that N_2 's optimal input is close to N_1 's. It is not a sound over-approximation of $\hat{z}_i^{N_2}$ at N_2 's own optimum (which is allowed to use a different input), but for an advisory hint this is the right quantity.

Mode direct — new rule. Skip the $\tilde{I}_i$ construction entirely. p_i is the fraction of N_2 's tightened pre-activation range on N_1 's supporting side of zero:

$$p_i = \begin{cases} \frac{-l_i^{N_2}}{u_i^{N_2} - l_i^{N_2}} & \text{if } v_i^{N_1} = 0, \\ \frac{u_i^{N_2}}{u_i^{N_2} - l_i^{N_2}} & \text{if } v_i^{N_1} = 1. \end{cases}$$

Bound Tightening’s integration step already folds N_1 ’s ReLU bounds and the weight drift into $[l_i^{N_2}, u_i^{N_2}]$, so re-deriving an interval by shifting $\hat{z}_i^{N_1}$ only re-introduces the midpoint-proxy error (when $v_i^{N_1} = 0$) and the diff-width noise (when $[d_i^\downarrow, d_i^\uparrow]$ is wide). Using $[l_i^{N_2}, u_i^{N_2}]$ directly self-calibrates: a wider diff bound flows through Bound Tightening to produce a wider $[l_i^{N_2}, u_i^{N_2}]$, which pulls p_i toward 0.5 and lowers the priority automatically.

Three regimes of p_i . Under either mode:

- $p_i = 1$: the source range lies entirely on N_1 ’s side — N_1 ’s choice almost certainly transfers.
- $0 < p_i < 1$: the source range straddles zero; p_i measures how much of its length still backs N_1 .
- $p_i = 0$: the source range lies entirely on the opposite side — the drift has likely flipped the neuron; N_1 ’s raw choice is wrong for N_2 .

The “source range” is $\tilde{I}_i$ under **prev** and $[l_i^{N_2}, u_i^{N_2}]$ under **direct**.

Continuous hint rule (all four non-off modes). Agree with N_1 when $p_i \geq 0.5$, flip otherwise:

$$\text{hint_val}(a_i) \leftarrow \begin{cases} v_i^{N_1} & \text{if } p_i \geq 0.5, \\ 1 - v_i^{N_1} & \text{if } p_i < 0.5. \end{cases}$$

Under **prev/direct** this is paired with a V-shaped priority, maximal at both extremes and minimal at the 50/50 line:

$$\text{VarHintVal}(a_i) \leftarrow \text{hint_val}(a_i), \quad \text{VarHintPri}(a_i) \leftarrow \max(1, \text{round}(100 \cdot |2p_i - 1|)).$$

Under **prev_pgd/direct_pgd** the same **hint_val** is instead routed through **Start** with a consensus filter against the PGD warm-start (see below); no **VarHintVal/VarHintPri** is written.

Anchor points, for intuition:

p_i	VarHintVal	VarHintPri	Interpretation
1.00	$v_i^{N_1}$	100	Source range fully on N_1 ’s side
0.75	$v_i^{N_1}$	50	Majority still backs N_1
0.50	either	1	No usable signal
0.25	$1 - v_i^{N_1}$	50	Majority opposes N_1
0.00	$1 - v_i^{N_1}$	100	Source range fully opposite to N_1 ; drift flipped the neuron

The hint value flips exactly at the $p_i = 0.5$ crossing. The priority is continuous in p_i and never requires a hand-tuned threshold beyond the floor at 1 (priority 0 is reserved by Gurobi for “ignore this hint”).

PGD-consensus Start channel (prev_pgd/direct_pgd). The “_pgd” modes use the same **hint_val**(a_i) as their non-pgd siblings but deliver it via **Start** (**set_start_value** in JuMP) instead of **VarHintVal/VarHintPri**. Before Variable Hints runs, PGD has already populated **Start** on some binaries via **hyper_attack_hints()**; for each surviving binary a_i we read **JuMP.start_value**(a_i) and apply a three-way consensus filter:

- **PGD silent** (**start_value** = **nothing**): N_1 ’s transfer hint is the only signal available — call **set_start_value**(a_i , **hint_val**) (“*fill*”).
- **PGD agrees** (**start_value** = **hint_val**): PGD and the N_1 transfer point at the same branch; leave PGD’s **Start** in place, no update (“*leave*”).

- **PGD disagrees** (`start_value` $\neq$ `hint_val`): the two signals contradict — call `set_start_value( $a_i$ , nothing)` to withdraw PGD’s value, leaving the binary unstated for Gurobi to decide (“*withdraw*”).

The intent is to keep only the warmstart bits both sources agree on (or that only one source filled in), instead of fighting between two incomplete MIP starts. If PGD failed entirely (all starts are `nothing`), the modes degrade to “variable hints fill every gap via `Start`” — equivalent to unconditional N_1 -transfer warmstart.

Relation to prev/direct. The p_i formulas, the flip rule, and the handling of eliminated binaries are unchanged; only the Gurobi channel differs. Priority information is discarded (Gurobi’s `Start` API has no analogue of `VarHintPri`) — every `hint_val` that survives the consensus filter has equal weight.

Optimal-hint characterization of direct. The p_i formula for `direct` is not a heuristic — it is a posterior probability under a canonical prior derived from Bound Tightening’s output. Specifically:

Theorem 7 (Bayes-optimal hint under max-entropy prior). *Fix a surviving binary a_i (so $l_i^{N_2} < 0 < u_i^{N_2}$). Let π_i be the maximum-entropy prior on $\hat{z}_i^{N_2}$ whose support is the Bound-Tightening output, i.e. $\hat{z}_i^{N_2} \sim \text{Unif}([l_i^{N_2}, u_i^{N_2}])$, and let $v_i^{N_2,*} \in \{0, 1\}$ be the (unknown) value taken by a_i at N_2 ’s optimum. Then:*

- (a) *The transfer probability of the `direct` rule equals the posterior probability of agreement with N_1 :*

$$\Pr_{\pi_i}[v_i^{N_2,*} = v_i^{N_1} \mid l_i^{N_2}, u_i^{N_2}] = p_i^{\text{direct}}.$$

- (b) *The hint value $\text{VarHintVal}(a_i) = (p_i \geq 0.5) ? v_i^{N_1} : 1 - v_i^{N_1}$ is the maximum a posteriori estimate of $v_i^{N_2,*}$ under π_i .*
- (c) *The hint priority $\text{VarHintPri}(a_i) = \max(1, \text{round}(100 \cdot |2p_i - 1|))$ is monotone in the Bayes-risk reduction $\mathbb{E}_{\pi_i}[\mathbf{1}\{v_i^{N_2,*} \neq \hat{v}\}] - \mathbb{E}_{\pi_i}[\mathbf{1}\{v_i^{N_2,*} \neq \text{MAP}\}]$ over the uniform random hint $\hat{v}$.*

Proof. The big- M encoding ties $a_i = \mathbf{1}\{\hat{z}_i^{N_2} \geq 0\}$ (up to tie-breaking on the measure-zero set $\hat{z}_i^{N_2} = 0$). Under π_i ,

$$\Pr_{\pi_i}[\hat{z}_i^{N_2} > 0] = \frac{u_i^{N_2}}{u_i^{N_2} - l_i^{N_2}}, \quad \Pr_{\pi_i}[\hat{z}_i^{N_2} < 0] = \frac{-l_i^{N_2}}{u_i^{N_2} - l_i^{N_2}}.$$

Substituting into the `direct` case split gives (a). Part (b) follows because the MAP estimate under a Bernoulli posterior with parameter q is $\mathbf{1}\{q \geq 0.5\}$; the `direct` rule assigns $v_i^{N_1}$ when $p_i \geq 0.5$ and $1 - v_i^{N_1}$ otherwise, which is exactly the MAP value. Part (c) is immediate from $|2p_i - 1|$ being the total-variation distance between the posterior and the uniform distribution. $\square$

Why prev is biased. The `prev` rule computes p_i on the interval $\tilde{I}_i = [\hat{z}_i^{N_1} + d_i^\downarrow, \hat{z}_i^{N_1} + d_i^\uparrow] \cap [l_i^{N_2}, u_i^{N_2}]$, which is a strict subinterval of $[l_i^{N_2}, u_i^{N_2}]$ whenever the diff bound is narrower than the tightened bound. $\tilde{I}_i$ is not the maximum-entropy support of $\hat{z}_i^{N_2}$: it presumes that N_2 ’s optimum will evaluate $\hat{z}_i^{N_2}$ near $\hat{z}_i^{N_1}$, a commitment that the big- M encoding does not license (the two MIPs can pick different inputs). Consequently `prev` is a biased estimator of $\Pr[v_i^{N_2,*} = v_i^{N_1}]$, with bias proportional to the diff-bound width $d_i^\uparrow - d_i^\downarrow$. This explains the empirical self-calibration observation already noted above: on instances where the diff bound is wide relative to $[l_i^{N_2}, u_i^{N_2}]$, `direct`’s Bayes-optimal pull toward $p_i = 0.5$ correctly suppresses the priority, whereas `prev` emits a confidently-wrong hint.

Applicability. The rule runs only for binaries that survive Bound Tightening’s elimination. When $l_i^{N_2} \geq 0$ or $u_i^{N_2} \leq 0$ the binary is already fixed and removed from the MIP, so there is nothing to hint.

Hints vs. candidate solution — and why we use both channels. Gurobi exposes two advisory channels for biasing the search. Soft *variable hints* (`VarHintVal/VarHintPri`) nudge each binary independently with a confidence weight; Gurobi is free to ignore any individual hint the LP relaxation disagrees with. A full *MIP Start* (`Start`) submits a candidate assignment that Gurobi’s start heuristic tries to turn into a feasible incumbent. Start is a stronger commitment: if N_1 ’s solution is infeasible for N_2 , Gurobi spends effort repairing (or discarding) it. Blindly transferring the entire N_1 primal as a Start was harmful in all tested combinations (“MIP Start mode”, all LOSER) — which motivated first the soft-hint modes (`prev`, `direct`), and then the PGD-consensus Start modes (`prev_pgd`, `direct_pgd`) that only emit a Start bit when the N_1 -transfer hint and the PGD warmstart agree (or when only one of them has an opinion). The consensus filter replaces the “repair-everything” failure mode with a “warmstart only the bits both signals trust” policy.

Soundness: δ_{exact} is preserved under all five modes. We claim $\delta_{\text{varHint}} = \delta_{\text{exact}}$ for every mode $\in \{\text{off}, \text{prev}, \text{direct}, \text{prev_pgd}, \text{direct_pgd}\}$ whenever Gurobi terminates at OPTIMAL. The argument is uniform:

- **No constraint, coefficient, or bound is modified by any mode.** `prev/direct` only set the advisory Gurobi attributes `VarHintVal/VarHintPri`; `prev_pgd/direct_pgd` only call `set_start_value` (which maps to Gurobi’s `Start` attribute) or withdraw it (`set_start_value(.,nothing)`). None of `VarHintVal`, `VarHintPri`, `Start` participates in the constraint system; they are inputs to Gurobi’s heuristics and branching priority, not to feasibility. In particular, `Start` does *not* bound the variable — the feasible set is whatever the big- M /triangle constraints define, independent of any Start value.
- **Feasible-set invariance.** Consequently:

$$\mathcal{F}_2^{(\text{off})} = \mathcal{F}_2^{(\text{prev})} = \mathcal{F}_2^{(\text{direct})} = \mathcal{F}_2^{(\text{prev_pgd})} = \mathcal{F}_2^{(\text{direct_pgd})} = \mathcal{F}_2.$$

Since $\delta_{\text{exact}} \triangleq \max_{x \in \mathcal{F}_2} \text{obj}_{N_2}(x)$ is defined purely in terms of $\mathcal{F}_2$ and the unchanged objective, δ_{exact} is identical across all five modes.

- **Consequence at termination.** At OPTIMAL, Gurobi returns $\max_{x \in \mathcal{F}_2^{(\text{mode})}} \text{obj}_{N_2}(x) = \delta_{\text{exact}}$ in every mode. At TIME_LIMIT the reported incumbent is a sound lower bound on δ_{exact} whose numeric value may differ across modes (hints shift the branching path and which incumbents are found first), but the underlying δ_{exact} is fixed.

This covers both the soft-hint modes and the PGD-consensus Start modes in a single argument — neither channel writes a constraint, so the optimum is locked to δ_{exact} regardless of whether the hint is right, wrong, or contradictory. The only thing a bad hint costs is search time.

4.3.1 Algorithm and worked example

Algorithm 6 Variable Hints on N_2 ($\text{MODE} \in \{\text{prev}, \text{direct}, \text{prev_pgd}, \text{direct_pgd}\}$)

Require: $\mathcal{M}_2$ for N_2 , N_1 primal $(v_i^{N_1}, \hat{z}_i^{N_1})$, N_2 tightened bounds $(l_i^{N_2}, u_i^{N_2})$ from Section 4.2, diff bounds $(d_i^\downarrow, d_i^\uparrow)$ (prev variants only)

- 1: **for** each surviving binary a_i in $\mathcal{M}_2$ (i.e. $l_i^{N_2} < 0 < u_i^{N_2}$) **do**
- 2: **if** $\text{MODE} \in \{\text{prev}, \text{prev_pgd}\}$ **then**
- 3: **if** $v_i^{N_1} = 1$: $\hat{z}_i^{N_1} \leftarrow z_i^{+,N_1}$; **else** $\hat{z}_i^{N_1} \leftarrow l_i^{N_1}/2$
- 4: $I_i \leftarrow [\hat{z}_i^{N_1} + d_i^\downarrow, \hat{z}_i^{N_1} + d_i^\uparrow]$; $\tilde{I}_i \leftarrow I_i \cap [l_i^{N_2}, u_i^{N_2}]$
- 5: $p_i \leftarrow \text{length}(\tilde{I}_i \cap S_i) / \text{length}(\tilde{I}_i)$ where $S_i = (-\infty, 0]$ if $v_i^{N_1} = 0$ **else** $[0, +\infty)$
- 6: **else**
- 7: *direct / direct_pgd*: **if** $v_i^{N_1} = 0$: $p_i \leftarrow (-l_i^{N_2}) / (u_i^{N_2} - l_i^{N_2})$; **else** $p_i \leftarrow u_i^{N_2} / (u_i^{N_2} - l_i^{N_2})$
- 8: **end if**
- 9: **if** $p_i \geq 0.5$: $\text{hint_val} \leftarrow v_i^{N_1}$; **else** $\text{hint_val} \leftarrow 1 - v_i^{N_1}$
- 10: **if** $\text{MODE} \in \{\text{prev}, \text{direct}\}$ **then**
- 11: $\text{VarHintVal}(a_i) \leftarrow \text{hint_val}$
- 12: $\text{VarHintPri}(a_i) \leftarrow \max(1, \text{round}(100 \cdot |2p_i - 1|))$
- 13: **else**
- 14: *prev_pgd / direct_pgd*: PGD-consensus **Start**; $s \leftarrow \text{JuMP.start_value}(a_i)$
- 15: **if** $s = \text{nothing}$ **then**
- 16: $\text{set_start_value}(a_i, \text{hint_val})$ {fill}
- 17: **else if** $\text{round}(s) = \text{hint_val}$ **then**
- 18: leave PGD's start in place {agree}
- 19: **else**
- 20: $\text{set_start_value}(a_i, \text{nothing})$ {withdraw}
- 21: **end if**
- 22: **end if**
- 23: **end for**

Variables / constraints added. *None.* The technique writes only to the Gurobi attributes `VarHintVal`, `VarHintPri` (soft-hint modes) or `Start` (PGD-consensus modes). None of these participates in the constraint matrix; the feasible set, objective, and δ_{exact} are identical to $\text{MODE} = \text{off}$.

Worked example. Take a surviving binary a_i on neuron $(1, 1)$ of N_2 from Section 4.2.5's example $((l_i^{N_2}, u_i^{N_2}) = (-0.3, 1.8), (d_i^\downarrow, d_i^\uparrow) = (0, 0.1))$. Suppose N_1 's Phase-1 primal achieved $v_i^{N_1} = 1$ with $\hat{z}_i^{N_1} = z_i^{+,N_1} = 0.4$.

prev rule. $I_i = [0.4 + 0, 0.4 + 0.1] = [0.4, 0.5]$, $\tilde{I}_i = I_i \cap [-0.3, 1.8] = [0.4, 0.5]$. $S_i = [0, +\infty)$, so $\tilde{I}_i \cap S_i = [0.4, 0.5]$, length 0.1. Then $p_i = 0.1/0.1 = 1.0$.

direct rule. $p_i = u_i^{N_2} / (u_i^{N_2} - l_i^{N_2}) = 1.8/2.1 \approx 0.857$.

Emitted hint. Both modes pick $\text{hint_val} = v_i^{N_1} = 1$ (since $p_i \geq 0.5$). Under *prev*, $\text{VarHintPri} = 100$ (maximal). Under *direct*, $\text{VarHintPri} = \max(1, \text{round}(71.4)) = 71$. The wider Bound Tightening interval self-calibrates the priority downward without any threshold tuning, because p_i regresses to 0.5 as $|l_i^{N_2}|, u_i^{N_2}$ both grow.

prev_pgd variant. Suppose PGD has already written $\text{start_value}(a_i) = 1$. Since $\text{round}(1) = 1 = \text{hint_val}$, the algorithm *leaves* PGD's start untouched. If PGD had written 0, the algorithm

would *withdraw* (set `nothing`), because the two signals contradict. If PGD had stayed silent (`start_value = nothing`), the algorithm would *fill* the start with `hint_val = 1`.

4.4 Per-Copy Triangle Drop (`BoundTightPertRelax`)

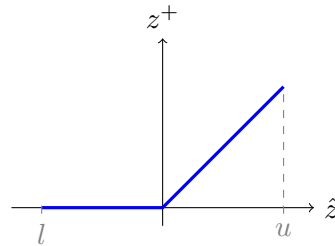
Flag: `--adv_std_n2_relax_threshold` $\in \{\text{off}, 0.01, 0.1, 0.5, 1.0\}$ (in safe combos; `off` means Per-Copy Triangle Drop is disabled). **Model modification:** replaces selected N_2 binary variables with LP relaxations. **Filename tag:** runs with $\tau \geq 0$ emit `_BoundTightPertRelax $\{\tau\}$` in place of `_boundTight` (the tag implies bound-tightening is active). The retired `_relaxT $\{\tau\}$` tag is not accepted by the post-processing pipeline.

What is relaxation, and why would we want it? The MIP we hand to Gurobi contains two kinds of variables: *continuous* variables (real numbers, e.g. the pre-activation $\hat{z}$ of a neuron) and *binary* variables $a \in \{0, 1\}$ (one per split ReLU neuron, indicating whether the neuron is active or inactive). Binaries are what make the problem hard: Gurobi has to branch over every possible 0/1 assignment in the worst case, so each binary in principle doubles the search tree. Our networks have hundreds of split ReLUs, which is why verification is slow.

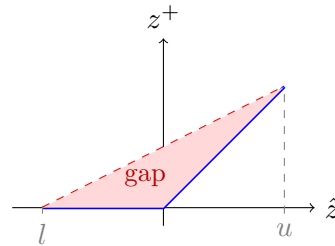
Relaxation means: replace the hard $\{0, 1\}$ choice on a binary with a looser, purely continuous set of constraints that still over-covers the exact ReLU behavior. The continuous version is a linear program (LP) — no branching needed. The price is that the feasible set becomes larger (it now includes some non-ReLU points), so the reported δ may come out slightly higher than the true optimum. If we relax only neurons that are “almost decided anyway,” the price is small and the speedup is large.

This subsection explains: (i) what exactly the relaxation replaces, (ii) how we measure “almost decided,” (iii) what $\tau = 0.5$ means concretely, and (iv) which binaries get dropped.

Exact vs. relaxed ReLU encoding. Recall from Section 2 that a split ReLU neuron (pre-activation bounds $l < 0 < u$) can be encoded in two ways:



(a) **Exact encoding**
4 constraints + binary a
 z^+ on the blue line only



(b) **Triangle relaxation**
3 constraints, no binary
 z^+ anywhere in shaded region

- **Exact** (a): uses a binary variable $a \in \{0, 1\}$ and 4 constraints (Eqs. (1)–(4)). $a = 1$ means “neuron active, $z^+ = \hat{z}$ ”; $a = 0$ means “neuron inactive, $z^+ = 0$ ”. The post-ReLU value z^+ is forced onto the exact ReLU curve (blue line in diagram (a)). This is precise but expensive: having to branch on a is what makes verification NP-hard.
- **Relaxed** (b): drops the binary a and uses 3 linear constraints (Eqs. (5)–(7)). z^+ is now allowed to take any value inside the shaded triangle in diagram (b). Every point on the true ReLU curve is still inside this triangle (so the relaxation *over-covers* reality), but the triangle also admits some points that are not on the curve — these extra admissible points are the

price we pay. The shaded region between the blue line and the red dashed envelope is the *gap*: the set of $(\hat{z}, z^+)$ pairs that the relaxation allows but the exact encoding would not.

The smaller the triangle, the less the relaxation loses. If l is very close to zero (the neuron “wants” to be active nearly always) or u is very close to zero (the neuron wants to be inactive nearly always), the triangle is a thin sliver and the relaxation is almost indistinguishable from the exact ReLU. Relaxing such a neuron costs almost nothing and saves one binary. When $|l| \approx u$ the triangle is fat — the neuron is genuinely ambiguous and we do not want to relax it.

Measuring “almost decided”: the triangle’s gap area. We quantify how thin the triangle is by its area. For a ReLU neuron with pre-activation bounds $[l, u]$:

$$\text{gap}(l, u) = \begin{cases} 0 & \text{if } l \geq 0 \text{ or } u \leq 0 \quad (\text{neuron is } \textit{stable} \text{ — not split at all}), \\ \frac{u \cdot |l|}{2(u - l)} & \text{otherwise (neuron is split; compute triangle area).} \end{cases} \quad (30)$$

The second case is just the base-times-height formula for the triangle in diagram (b) (base $u - l$ along the $\hat{z}$ -axis, peak offset $u \cdot |l|/(u - l)$ above the ReLU line at $\hat{z} = 0$). Some concrete numbers to build intuition:

l	u	$\text{gap}(l, u)$	Interpretation
−0.1	2.0	0.095	sliver: almost always active; cheap to relax
−2.0	0.1	0.095	sliver: almost always inactive; cheap to relax
−0.5	0.5	0.125	still fairly thin
−1.0	1.0	0.25	moderate
−2.0	2.0	0.5	big triangle; relaxation is noticeably loose
−5.0	5.0	1.25	very fat; ambiguous neuron, leave exact

A threshold τ drawn in the same units of (pre-activation area) lets us say: “relax any neuron whose triangle has area $\leq \tau$, keep the rest exact.” Concretely:

- $\tau = 0.01$: relax only extremely thin slivers ($|l| < 0.02$ or $u < 0.02$ in the table above).
- $\tau = 0.1$: relax most slivers but keep anything more than moderately split.
- $\tau = 0.5$: relax everything with $\text{gap} \leq 0.5$, including the $[-2, 2]$ case above; genuinely ambiguous neurons ($\text{gap} > 0.5$) still keep their binary. This is aggressive but still below the fully-symmetric case.
- $\tau = 1.0$: relax almost every split neuron; fat triangles get replaced too. Strongest speedup, loosest answer.
- $\tau = \text{off}$: relax nothing — every split neuron keeps its binary and its exact big- M encoding.

So the “0.5” in `--adv_std_n2_relax_threshold=0.5` is literally the maximum triangle-area any neuron can have and still be converted from exact-with-binary to relaxed-without-binary.

Which binaries actually get dropped? Because VHAGaR encodes the network twice (see the next paragraph), every split ReLU neuron (m, k) owns *two* binaries in the MIP: $a_{m,k}^{\text{org}} \in \{0, 1\}$ on the clean-input copy and $a_{m,k}^{\text{pert}} \in \{0, 1\}$ on the perturbed-input copy. The relaxation decision is made per copy: for each copy $c \in \{\text{org}, \text{pert}\}$ whose gap $g_{m,k}^c$ satisfies the tiered rule below, we remove $a_{m,k}^c$ together with the two big- M constraints that mention it (Eqs. (3)–(4)) and replace them with the three triangle inequalities (Eqs. (5)–(7)) on that copy’s interval $[l_{m,k}^{N_2,c}, u_{m,k}^{N_2,c}]$. Concretely, three outcomes are possible for a single neuron:

- drop *both* $a_{m,k}^{\text{org}}$ and $a_{m,k}^{\text{pert}}$ (relax both copies);
- drop exactly one of them (relax only that copy; the other copy keeps its binary and its four big- M constraints);
- keep both (no relaxation on this neuron).

Nothing else is touched in any of the three outcomes: the pre-activations $\hat{z}_{m,k}^{\text{org}}, \hat{z}_{m,k}^{\text{pert}}$, the post-ReLUs $z_{m,k}^{+, \text{org}}, z_{m,k}^{+, \text{pert}}$, the neurons in adjacent layers, the inter-layer linear constraints, and the perturbation-coupling constraints that link the two copies all remain exactly as they were.

Why there are *two* bounds per neuron (the org/pert twist). VHAGaR encodes each network twice inside one MIP: once for the clean input v_{in} (the “org” copy) and once for the perturbed input $v_{x_0} = v_{\text{in}} + v_e$ (the “pert” copy). Every ReLU neuron therefore has *two* instances in the MIP: one on the org copy with pre-activation bounds $[l_{m,k}^{N_2, \text{org}}, u_{m,k}^{N_2, \text{org}}]$, and one on the pert copy with (generally looser, because the pert input box is wider) bounds $[l_{m,k}^{N_2, \text{pert}}, u_{m,k}^{N_2, \text{pert}}]$. Each instance has its own binary and its own triangle gap. This lets us make the relaxation decision *independently* per copy: we can relax the org-copy binary, the pert-copy binary, both, or neither.

Decision rule (tiered, per-copy). Put everything together. For each ReLU neuron (m, k) we compute two gap numbers, one per copy:

$$g_{m,k}^{\text{org}} = \text{gap}(l_{m,k}^{N_2, \text{org}}, u_{m,k}^{N_2, \text{org}}), \quad g_{m,k}^{\text{pert}} = \text{gap}(l_{m,k}^{N_2, \text{pert}}, u_{m,k}^{N_2, \text{pert}}),$$

then compare each to the threshold τ . There are three cases (hence the word “tiered”):

1. **Both tiny** — if $\max(g_{m,k}^{\text{org}}, g_{m,k}^{\text{pert}}) \leq \tau$, both copies have thin triangles, so we drop *both* binaries $a_{m,k}^{\text{org}}$ and $a_{m,k}^{\text{pert}}$ and put the triangle inequalities on both copies instead.
2. **Only one tiny** — if exactly one of the two gaps is $\leq \tau$, we drop the binary for the smaller-gap copy only. The other copy keeps its binary and exact big- M encoding. This is the case that motivates the name **BoundTightPertRelax**: under a non-trivial perturbation the pert copy’s bounds are looser (its triangle is bigger), so a neuron can easily be tight on the org side and still split on the pert side.
3. **Neither tiny** — if both gaps exceed τ , the neuron is genuinely ambiguous on both sides. Keep the binary on both copies, do not relax.

The per-copy bounds $l_{m,k}^{N_2, \text{copy}}, u_{m,k}^{N_2, \text{copy}}$ that feed $\text{gap}(\cdot)$ are the ones Bound Tightening already computed — specifically, they are the intersection of Source A (N_1 ’s per-copy bound plus the diff) and Source B (the shared absolute- N_2 zonotope hull):

$$u_{m,k}^{N_2, \text{copy}} = \min \left(\underbrace{u_{m,k}^{N_1, \text{copy}} + d_{m,k}^{\uparrow}}_{\text{Source A: } N_1 \text{ bound + diff}}, \underbrace{u_{m,k}^{\text{B}}}_{\text{Source B: } N_2 \text{ zonotope hull}} \right), \quad (31)$$

$$l_{m,k}^{N_2, \text{copy}} = \max \left(l_{m,k}^{N_1, \text{copy}} + d_{m,k}^{\downarrow}, l_{m,k}^{\text{B}} \right). \quad (32)$$

The org/pert asymmetry flows entirely through Source A: N_1 ’s Phase-1 MIP is itself dual-copy, so its stored bound table $\{(l_{m,k}^{N_1, \text{copy}}, u_{m,k}^{N_1, \text{copy}})\}$ distinguishes the two copies (the pert copy inherits a wider input box $[-\varepsilon, 1+\varepsilon]^n$, so its bounds are typically looser). The diff interval $[d_{m,k}^{\downarrow}, d_{m,k}^{\uparrow}]$ is shared between copies. Source B is also shared: its zonotope is initialised to cover both input boxes at once, so a single hull is valid for either copy. Therefore, the only reason $g_{m,k}^{\text{org}}$ and $g_{m,k}^{\text{pert}}$ ever disagree is N_1 ’s own per-copy bounds; when those happen to be identical at a neuron, both gaps collapse to the same value and Tier 2 degenerates into Tier 1 or Tier 3 there.

Safety net: the two copies are still linked. Even when we relax one copy and leave the other exact, the two are not free to drift apart. VHAGaR’s perturbed-interval coupling constraints (always active in advstd safe combinations) link them via $z_{\text{pert}}^+ - z_{\text{org}}^+ \in [\Delta^\downarrow, \Delta^\uparrow]$, where $[\Delta^\downarrow, \Delta^\uparrow]$ is a sound interval on how much the pert post-ReLU value can differ from the org one under the perturbation budget. So relaxing a binary on one side still indirectly constrains the other side.

Why N_2 ’s per-copy bounds (not N_1 ’s)? We judge a neuron’s gap using N_2 ’s tightened bounds, not N_1 ’s, for three reasons. (i) The triangle constraints we emit use exactly those N_2 bounds — deciding on the same interval we later write into the MIP is what makes the soundness argument work (see “Consistency requirement” and “Soundness” below). (ii) Under non-trivial perturbations, N_2 ’s org- and pert-copy bounds can differ substantially (the pert copy sees a wider input box), so a single-gap rule built on N_1 would miss the asymmetric opportunities Tier 2 exploits. (iii) The flag name itself — `BoundTightPertRelax` — advertises the dependency on Bound Tightening’s per-copy bound tightening.

Consistency requirement. The decision-time gaps $g_{m,k}^{\text{org}}$ and $g_{m,k}^{\text{pert}}$ must be computed from the *same* intersected bounds that get baked into the emitted triangle constraints. In the code, both sites call the shared helper `intersect_per_copy_bounds` in `core_ops.jl`, so there is no drift between “the interval we decided was thin” and “the interval of the triangle we wrote into the MIP.” This invariant is what the soundness argument below rests on.

What does this cost, and what do we gain? Each neuron we relax removes one binary variable and two big- M constraints from the MIP (and adds three linear inequalities), so on a network with hundreds of split ReLUs we typically save dozens of binaries. Since each binary in principle doubles Gurobi’s branch-and-bound tree, removing even a handful of binaries can cut solve time substantially.

The price is that the reported δ is no longer guaranteed equal to the exact optimum δ_{exact} ; it is now a *sound upper bound*:

$$\delta_{\text{relaxed}} \geq \delta_{\text{exact}}.$$

In other words, if the relaxed MIP certifies robustness at some δ , then the exact MIP would also certify robustness at (at worst) that same δ . The overshoot grows with τ , but since we only relax neurons with gap $\leq \tau$, each relaxation contributes at most a small slice of looseness. In practice the observed overshoot is often in the third decimal digit even at $\tau = 0.5$.

Soundness. Let $R \subseteq \{(m, k, \text{copy})\}$ be the set of (neuron, copy) pairs selected for relaxation by the tiered rule, and let $\mathcal{F}_2^{\{\{6\}\}}$ denote the feasible set of the resulting N2 MIP. We claim $\mathcal{F}_2 \subseteq \mathcal{F}_2^{\{\{6\}\}}$, hence $\delta_{\text{BTPR}} \geq \delta_{\text{exact}}$.

- For $(m, k, \text{copy}) \notin R$ the exact big- M encoding is emitted, identical to the baseline; every $x \in \mathcal{F}_2$ trivially satisfies the same constraints.
- For $(m, k, \text{copy}) \in R$ the binary $a_{m,k,\text{copy}}$ and its four big- M constraints are dropped; the three triangle inequalities are emitted on the *same* per-copy bound $[l_{m,k}^{N_2,\text{copy}}, u_{m,k}^{N_2,\text{copy}}]$ that would have gated the exact encoding (the consistency requirement above). By the triangle-relaxation lemma (Eqs. (5)–(7)), every $(\hat{z}, z^+)$ satisfying the exact ReLU encoding on $[l_{m,k}^{N_2,\text{copy}}, u_{m,k}^{N_2,\text{copy}}]$ lies inside the triangle on the same interval. So every $x \in \mathcal{F}_2$ satisfies the relaxed constraints.

All other constraints (perturbed-interval coupling, VHAGaR dependencies, inter-layer linking, warmstart hints as non-constraining hints) are unchanged. Therefore $\mathcal{F}_2 \subseteq \mathcal{F}_2^{\{6\}}$, and taking the max of the unchanged objective over a superset can only raise the value: $\delta_{\text{BTPR}} = \max_{x \in \mathcal{F}_2^{\{6\}}} \text{obj}_{N_2}(x) \geq \max_{x \in \mathcal{F}_2} \text{obj}_{N_2}(x) = \delta_{\text{exact}}$.

4.4.1 Algorithm and worked example

Algorithm 7 Per-Copy Triangle Drop on N_2 (`compute_n2_relax_decision!`)

Require: N_2 with dual encoding (org/pert copies), threshold τ , per-copy N_2 bounds $(l_{m,k}^{N_2,c}, u_{m,k}^{N_2,c})$ from Bound Tightening (Section 4.2.4, eqs. 31–32)

```

1: n2_relax_decision  $\leftarrow \{\}$ 
2: for each ReLU neuron  $(m, k)$  in  $N_2$  do
3:   skip if stable on both copies {relu()} short-circuits before the binary is allocated}
4:    $g^c \leftarrow \text{gap}(l_{m,k}^{N_2,c}, u_{m,k}^{N_2,c})$  for  $c \in \{\text{org}, \text{pert}\}$  (eq. 30)
5:   if  $\max(g^{\text{org}}, g^{\text{pert}}) \leq \tau$  then
6:      $(\text{relax\_org}, \text{relax\_pert}) \leftarrow (\text{true}, \text{true})$  {both thin}
7:   else if  $\min(g^{\text{org}}, g^{\text{pert}}) \leq \tau$  then
8:      $(\text{relax\_org}, \text{relax\_pert}) \leftarrow \text{flag only arg min}_c g^c$  {one thin}
9:   else
10:     $(\text{relax\_org}, \text{relax\_pert}) \leftarrow (\text{false}, \text{false})$  {neither}
11:   end if
12:   record the joint flag pair under each pass's MIP key so relu() can dispatch on it at emission time
13: end for
14: During  $\mathcal{M}_2$  build: for each  $(m, k, c)$  with flag true, drop binary  $a_{m,k}^c$  and its two big- $M$  rows (eqs. 3–4); emit the three triangle rows (eqs. 5–7) on  $[l_{m,k}^{N_2,c}, u_{m,k}^{N_2,c}]$ 

```

Variables / constraints per relaxed (m, k, c) . *Removed:* 1 binary $a_{m,k}^c$ and 2 big- M rows (eqs. 3–4). *Added:* 3 triangle rows (eqs. 5–7) on the same $\hat{z}_{m,k}^c, z_{m,k}^{+,c}$ variables. No new continuous variable.

Worked example. Same N_1, N_2 pair as Section 4.2.5. Per-copy N_2 bounds for the hidden ReLU layer (taking Method-1 outputs and extending the pert copy's N_1 bounds by the same diff):

neuron	$(l, u)^{N_2, \text{org}}$	$(l, u)^{N_2, \text{pert}}$	$(g^{\text{org}}, g^{\text{pert}})$
(1, 1)	(−0.3, 1.8)	(−0.5, 2.0)	(0.129, 0.200)
(1, 2)	(−1.1, 0.9)	(−1.3, 1.1)	(0.248, 0.298)

Tier at $\tau = 0.15$. Neuron (1, 1): $g^{\text{org}} = 0.129 \leq 0.15 < g^{\text{pert}}$, so *one-thin (org)* — drop $a_{1,1}^{N_2, \text{org}}$, keep $a_{1,1}^{N_2, \text{pert}}$. Neuron (1, 2): both gaps > 0.15 , so *neither-thin*.

Emitted on the relaxed org-copy of neuron (1, 1). Three rows (eqs. 5–7) on $[l_{1,1}^{N_2, \text{org}}, u_{1,1}^{N_2, \text{org}}] = [-0.3, 1.8]$:

$$z_{1,1}^{+, \text{org}} \geq 0, \quad z_{1,1}^{+, \text{org}} \geq \hat{z}_{1,1}^{\text{org}}, \quad z_{1,1}^{+, \text{org}} \leq \frac{1.8}{1.8 - (-0.3)} (\hat{z}_{1,1}^{\text{org}} - (-0.3)) \approx 0.857 (\hat{z}_{1,1}^{\text{org}} + 0.3).$$

The pert copy retains its four exact big- M rows (eqs. 1–4) and its binary $a_{1,1}^{N_2,\text{pert}}$. The two copies remain linked by the perturbed-interval row (Section 3.1’s $z_{1,1}^{+, \text{pert}} - z_{1,1}^{+, \text{org}} \in [-0.2, 0.2]$), which is what makes the one-thin tier sound.

4.5 Sibling-Gated Refinement (SibGate)

Flag: `--adv_std_n2_sibling_gate` $\in \{\text{false}, \text{true}\}$. **Requires:** `--adv_std_n2_relax_threshold` (this technique augments Per-Copy Triangle Drop’s tiered decision rule; it does not introduce one). **Model modification:** replaces the simple Wong–Kolter triangle with a conditional triangle gated on the surviving sibling binary in the “one thin” tier, and adds a pre-activation coupling constraint linking the two copies in the “both thin” tier. **Filename tag:** `_SibGate` appended to the existing `_BoundTightPertRelax $\{\tau\}$` tag.

What is changed. Per-Copy Triangle Drop’s tiered per-copy decision rule (Section 4.4) is inherited verbatim. Only the constraints emitted on each tier change:

Tier	Per-Copy Triangle Drop (today)	Sibling-Gated Refinement (proposed)
both thin	simple triangle on each copy	simple triangle on each copy <i>plus</i> a single pre-activation coupling line linking them
one thin	simple triangle on the relaxed copy	conditional triangle on the relaxed copy, gated on the kept sibling’s binary, with int
neither	exact big- M on both	unchanged

Tier “one thin” — conditional triangle

Let $c \in \{\text{org}, \text{pert}\}$ be the relaxed copy and pre its sibling (the copy whose binary survives in the MIP): $\text{pre} = \text{pert}$ when $c = \text{org}$, and $\text{pre} = \text{org}$ when $c = \text{pert}$. Write $a_{m,k}^{\text{pre}}$ for the surviving sibling binary. Instead of the three inequalities (5)–(7) the MIP receives four:

$$z_{m,k}^c \geq 0, \quad z_{m,k}^c \geq \hat{z}_{m,k}^c, \quad (33)$$

$$z_{m,k}^c \leq \frac{u_A}{u_A - l_A} (\hat{z}_{m,k}^c - l_A) + M(1 - a_{m,k}^{\text{pre}}), \quad (34)$$

$$z_{m,k}^c \leq \frac{u_I}{u_I - l_I} (\hat{z}_{m,k}^c - l_I) + M a_{m,k}^{\text{pre}}, \quad (35)$$

with $M = u_{m,k}^{N_2,c} + |l_{m,k}^{N_2,c}|$ chosen large enough to make the “off” upper bound vacuous over the feasible region. (For $c = \text{org}$ this is exactly the conditional triangle of the existing `--use_relaxations` encoding; the $c = \text{pert}$ case is the symmetric mirror, gated on $a_{m,k}^{\text{org}}$ instead.)

The four conditional bounds l_A, u_A, l_I, u_I are built from three sound interval inputs — two of them produced by Bound Tightening (`--adv_std_bound_tightening`) and one of them produced by the perturbation-coupling pipeline that backs `--use_relaxations`:

- $[l_{m,k}^{N_2,c}, u_{m,k}^{N_2,c}]$ — the unconditional pre-activation bound on the *relaxed* copy c , produced by Bound Tightening’s `intersect_per_copy_bounds()` routine (Section 4.2.4).
- $[l_{m,k}^{N_2,\text{pre}}, u_{m,k}^{N_2,\text{pre}}]$ — the same routine applied to the *sibling* (kept-exact) copy pre .
- $[l_{\text{int}}, u_{\text{int}}]$ — the *intended* input is a sound interval bound on the perturbation-induced pre-activation difference $\hat{z}_{m,k}^c - \hat{z}_{m,k}^{\text{pre}}$ in N_2 . The formula derivation in (36)–(37) and the soundness proof below assume exactly that quantity, which is the pre-activation analogue of the post-activation coupling bound $[\Delta^\downarrow, \Delta^\uparrow]$ already used by the perturbed-interval constraints. **Caveat on today’s code:** the routines that populate the global `relu_comp_*_bounds` actually store $\hat{z}_{N_2,\text{pert}} - \hat{z}_{N_1,\text{org}}$, which bundles the org–pert perturbation diff with the $N_1 - N_2$

weight drift; the dispatch is gated on `--use_relaxations` or `--adv_std_zono_bounds`, *not* on `--adv_std_bound_tightening`, and the diff component (but not the pert component) becomes zonotope-tightened when `--adv_std_zono_bounds=true`. The next paragraph spells out the recurrence the code uses and what changes are needed to expose the pure pert bound that this derivation needs.

How $[l_{\text{int}}, u_{\text{int}}]$ is computed in the actual code. The quantity Sibling-Gated Refinement conceptually wants is the pre-activation *org-pert* diff $\hat{z}_{m,k}^{\text{pert}} - \hat{z}_{m,k}^{\text{org}}$ in N_2 . Inside both `compute_diff_and_comp_bounds()` and its zonotope sibling `compute_diff_bounds_zonotope()` (`utils/perturbation_intervals.jl`), this diff lives in two 4D arrays `pert_up`, `pert_down`. The recurrence that propagates them through N_2 alone is verbatim:

Initialisation (input layer). `pert_up` $\leftarrow +\varepsilon \cdot \mathbf{1}$, `pert_down` $\leftarrow -\varepsilon \cdot \mathbf{1}$.

Linear layer (with weights W_2). Plain interval arithmetic; the bias cancels in the difference:

$$[\text{pert_down}', \text{pert_up}'] = W_2 \cdot [\text{pert_down}, \text{pert_up}].$$

Conv layer (with filter F_2). Same idea, via `interval_conv2d_bounds( $F_2, F_2, \cdot$ )` with zero bias.

ReLU layer. Per-neuron clipping: `pert_up` $\leftarrow \max(0, \text{pert_up})$, `pert_down` $\leftarrow -\max(0, -\text{pert_down})$.

The two values stored at the input to each ReLU are then exactly the quantity Sibling-Gated Refinement wants, and they would be the natural source for $[l_{\text{int},m,k}, u_{\text{int},m,k}]$. The pert recurrence is *always interval arithmetic*, in both compute routines; the zono flag does not switch it. (What the zono flag switches is the *diff* state that runs alongside, propagating $\hat{z}_{m,k}^{N_2} - \hat{z}_{m,k}^{N_1}$ as a zonotope — a separate quantity.)

What the code actually exposes today (Important caveat). At each ReLU layer the code does not push `pert_up`, `pert_down` to a global. It only pushes the *composed* bound

$$\text{relu_comp_up}_{m,k} = \text{diff_up}_{m,k} + \text{pert_up}_{m,k}, \quad \text{relu_comp_down}_{m,k} = \text{diff_down}_{m,k} + \text{pert_down}_{m,k},$$

which bounds $\hat{z}_{m,k}^{N_2, \text{pert}} - \hat{z}_{m,k}^{N_1, \text{org}}$ (weight drift *and* input perturbation combined). These are the `relu_comp_*_bounds` globals that today's `--use_relaxations` code reads. Two consequences:

1. If Sibling-Gated Refinement reads `relu_comp_*_bounds` verbatim, its $[l_{\text{int}}, u_{\text{int}}]$ inherits the N_1 – N_2 diff component on top of the org-pert perturbation — that is *not* what the conditional triangle's derivation in (36)–(37) assumes, and the diff component does become zonotope-tightened when `--adv_std_zono_bounds=true`.
2. To match the derivation, the code change for Sibling-Gated Refinement needs to expose `pert_up`, `pert_down` at each ReLU as new globals (call them `relu_pert_*_bounds`) and have Sibling-Gated Refinement read those instead of `relu_comp_*`. With that change, $[l_{\text{int}}, u_{\text{int}}]$ would always be interval, regardless of `--adv_std_zono_bounds`, as the derivation requires.

Flags that today trigger the routines (and what `run_relaxation_sweep.py` sets). Both `compute_diff_and_comp_bounds()` and `compute_diff_bounds_zonotope()` are dispatched from `utils/perturbation_intervals.jl`. The dispatch fires only when at least one of these flags is true:

- `--use_relaxations=true` (with N_1 binaries kept and N_1 encoded), *or*
- `--adv_std_zono_bounds=true` (`bound_n2_relu_using_zonotope`), *or*
- one of the niche flags `bound_n2_xp_using_composed`, `bound_by_zonotope_n2_hidden_neurons_which_are_not_no_n2_xp_encoding`, `no_n1_encoding_at_all`, or `n1_stability_relax_threshold  $\geq 0$` .

When the dispatch does fire, the choice between the two routines is `--adv_std_zono_bounds: false → compute_diff_and_comp_bounds()`, `true → compute_diff_bounds_zonotope()`.

The Phase-2 invocation built by `run_relaxation_sweep.py` (`run_relaxation_sweep.py:2486-2513`) does *not* pass `--use_relaxations`, and Sibling-Gated Refinement’s flag `--adv_std_n2_sibling_gate` is not yet wired up. So in the sweep the routines only fire on the rows where `--adv_std_zono_bounds=true`. Concretely:

- Sweep row with `--adv_std_zono_bounds=true`: `relu_comp.*_bounds` are populated by the zonotope variant. The diff side is zonotope-tightened, the pert side is interval — so the comp bound is partially zonotope.
- Sweep row with `--adv_std_zono_bounds=false`: `relu_comp.*_bounds` are not populated at all. A Sibling-Gated Refinement emission that reads them would see empty arrays and have nothing to gate on; it would degrade to Per-Copy Triangle Drop.

The minimum code change to make Sibling-Gated Refinement active in the sweep is to broaden the dispatch condition to also fire on `--adv_std_n2_sibling_gate=true`, and to expose the pert state as its own global so $[l_{\text{int}}, u_{\text{int}}]$ matches the formula derivation rather than carrying weight drift on top.

With those inputs in hand, the conditional bounds are obtained by intersecting Source-A-style conditional intervals (the first argument of each max/min) with the unconditional per-copy bound on the relaxed copy (the second argument):

$$l_A \triangleq \max(l_{\text{int}}, l_{m,k}^{N_2,c}), \quad u_A \triangleq \min(u_{m,k}^{N_2,\text{pre}} + u_{\text{int}}, u_{m,k}^{N_2,c}), \quad (36)$$

$$l_I \triangleq \max(l_{m,k}^{N_2,\text{pre}} + l_{\text{int}}, l_{m,k}^{N_2,c}), \quad u_I \triangleq \min(u_{\text{int}}, u_{m,k}^{N_2,c}). \quad (37)$$

The first argument of each max/min is what today’s `--use_relaxations` code uses on its own (computed by `compute_diff_and_comp_bounds()`, currently gated on `--use_relaxations`, *not* on `--adv_std_bound_tightening`). The second argument is the new contribution: the per-copy unconditional bound $[l_{m,k}^{N_2,\cdot}, u_{m,k}^{N_2,\cdot}]$ that Bound Tightening (`--adv_std_bound_tightening`) produces via `intersect_per_copy_bounds()`. So the conditional bounds genuinely need *both* pipelines to be active: the comp-bound machinery to give the first argument, and Bound Tightening’s per-copy tightening to give the second. With either pipeline missing, the intersection collapses to whichever side is still finite and Sibling-Gated Refinement degrades — in the limit, all the way back to Per-Copy Triangle Drop.

Tier “both thin” — pre-activation coupling

Both binaries are dropped, both copies receive the simple triangle exactly as in Per-Copy Triangle Drop. We additionally emit *one* linear constraint per neuron:

$$l_{\text{int}} \leq \hat{z}_{m,k}^{\text{pert}} - \hat{z}_{m,k}^{\text{org}} \leq u_{\text{int}}. \quad (38)$$

This constraint involves only existing pre-activation variables; no binary, no big- M . Its purpose is to prevent the LP relaxation from choosing the two copies’ pre-activations independently. (At the post-activation level the analogous coupling $z^{+,\text{pert}} - z^{+,\text{org}} \in [\Delta^\downarrow, \Delta^\uparrow]$ is already contributed by the perturbed-interval coupling described after Section 4.2.4; the new line is its missing pre-activation analogue.)

Soundness — “one thin”. Let $\mathcal{F}_2$ be the feasible set of N_2 ’s exact MIP and let $\mathcal{F}_2^{(4)}$ be the feasible set after Sibling-Gated Refinement emission. We claim $\mathcal{F}_2 \subseteq \mathcal{F}_2^{(4)}$.

Take any $x \in \mathcal{F}_2$ and any neuron (m, k) in tier “one thin”; let $b = a_{m,k}^{\text{pre}}(x) \in \{0, 1\}$ be the realised value of the kept sibling binary at x . We verify each of (33)–(35) at x .

The two lower bounds in (33) appear in the exact ReLU encoding too, so x satisfies them.

For the upper bounds we split on b .

(i) $b = 1$. The exact big- M on the sibling enforces $\hat{z}^{\text{pre}}(x) \in [0, u_{m,k}^{N_2, \text{pre}}]$. Bound Tightening’s diff bound gives $\hat{z}^c(x) - \hat{z}^{\text{pre}}(x) \in [l_{\text{int}}, u_{\text{int}}]$, so

$$\hat{z}^c(x) \in [l_{\text{int}}, u_{m,k}^{N_2, \text{pre}} + u_{\text{int}}].$$

Combining with Bound Tightening’s unconditional bound $\hat{z}^c(x) \in [l_{m,k}^{N_2, c}, u_{m,k}^{N_2, c}]$ and the definition in (36) yields $\hat{z}^c(x) \in [l_A, u_A]$. The Wong–Kolter envelope on $[l_A, u_A]$ over-approximates $\text{ReLU}(\hat{z})$ on that interval (the standard triangle-relaxation lemma applied with endpoints l_A, u_A). Since $z^c(x) = \text{ReLU}(\hat{z}^c(x))$ under the exact encoding,

$$z^c(x) \leq \frac{u_A}{u_A - l_A} (\hat{z}^c(x) - l_A),$$

which together with $M(1 - b) = 0$ is exactly (34). Constraint (35) has slack $M \cdot b = M$ on its right-hand side; by the choice of M this is at least $u_{m,k}^{N_2, c} \geq z^c(x)$, so the constraint is vacuous.

(ii) $b = 0$. Symmetric: the exact big- M on the sibling enforces $\hat{z}^{\text{pre}}(x) \in [l_{m,k}^{N_2, \text{pre}}, 0]$, hence $\hat{z}^c(x) \in [l_{m,k}^{N_2, \text{pre}} + l_{\text{int}}, u_{\text{int}}] \cap [l_{m,k}^{N_2, c}, u_{m,k}^{N_2, c}] = [l_I, u_I]$ by (37). The triangle envelope on $[l_I, u_I]$ over-approximates ReLU, giving (35) with $M \cdot b = 0$; (34) is vacuous by the same big- M argument with $M(1 - b) = M$.

So x satisfies all four constraints, in either case of b .

Soundness — “both thin”. The added constraint (38) uses $[l_{\text{int}}, u_{\text{int}}]$, which is derived by sound interval-arithmetic propagation of the input perturbation through N_2 in `compute_diff_and_comp_bounds()`. Therefore for every $x \in \mathcal{F}_2$, $\hat{z}^{\text{pert}}(x) - \hat{z}^{\text{org}}(x) \in [l_{\text{int}}, u_{\text{int}}]$ holds by construction.

Why δ_{exact} remains in the search space. Combining the two soundness arguments above with the unchanged tier “neither” and the unchanged simple-triangle / big- M encodings everywhere else gives

$$\mathcal{F}_2 \subseteq \mathcal{F}_2^{(4)}.$$

Let $x^* \in \arg \max_{x \in \mathcal{F}_2} \text{obj}_{N_2}(x)$, so by definition $\text{obj}_{N_2}(x^*) = \delta_{\text{exact}}$. Since $x^* \in \mathcal{F}_2 \subseteq \mathcal{F}_2^{(4)}$, the point x^* is also feasible under Sibling-Gated Refinement. The objective function is unchanged, so

$$\delta_{\text{SibGate}} \triangleq \max_{x \in \mathcal{F}_2^{(4)}} \text{obj}_{N_2}(x) \geq \text{obj}_{N_2}(x^*) = \delta_{\text{exact}}.$$

Taking the max over a superset can only raise the value, so δ_{exact} is in the SibGate search space and the reported δ_{SibGate} at OPTIMAL is a sound upper bound on δ_{exact} .

When the relaxation is genuinely tighter than Per-Copy Triangle Drop. The two conditional intervals $[l_A, u_A], [l_I, u_I]$ are subintervals of $[l_{m,k}^{N_2, c}, u_{m,k}^{N_2, c}]$, so each conditional triangle is no fatter than the single triangle Per-Copy Triangle Drop emits. The intersection step (36)–(37) shrinks them strictly whenever Source A’s diff-derived bound and Source B’s absolute bound disagree at a given endpoint (Theorem 6). The “both thin” coupling line shrinks the joint LP feasible region of $(\hat{z}^{\text{org}}, \hat{z}^{\text{pert}})$ from a rectangle to a slab. In each case the relaxation is at least as tight as Per-Copy Triangle Drop and strictly tighter on neurons where the additional information binds.

Summary of SibGate’s added constraints. On top of Per-Copy Triangle Drop’s tiered emission, SibGate writes at most three extra linear constraints per relaxed neuron — and *only* on neurons that Per-Copy Triangle Drop already classified as “thin enough” for some level of relaxation:

- *Both-thin tier* (both binaries already dropped by Per-Copy Triangle Drop): SibGate adds the single pre-activation coupling line (38), $l_{\text{int}} \leq \hat{z}_{m,k}^{\text{pert}} - \hat{z}_{m,k}^{\text{org}} \leq u_{\text{int}}$. No binary, no big- M — just one slab tying the two preacts.
- *One-thin tier* (binary dropped on copy c , kept on sibling pre): SibGate *replaces* the simple Wong–Kolter triangle on copy c with the two big- M conditional upper bounds (34)–(35), each gated on the surviving sibling binary $a_{m,k}^{\text{pre}}$. The two lower bounds (33) are unchanged.
- *Neither-thin tier*: SibGate touches nothing; Per-Copy Triangle Drop leaves both copies with their exact big- M encoding.

Why $\delta_{\text{SibGate}} = \delta_{\text{exact}}$ at $\tau = 0$. At threshold $\tau = 0$ Per-Copy Triangle Drop’s tiered rule collapses significantly. Recall the gap formula (30): for any *split* neuron ($l < 0 < u$) the triangle area $\text{gap}(l, u) > 0$ strictly; for any *stable* neuron ($l \geq 0$ or $u \leq 0$) it is exactly 0. So the only way for a gap to satisfy $g \leq \tau = 0$ is for the neuron to be stable on that copy. Walking the three tiers under this condition:

- *Both-thin* requires $\max(g^{\text{org}}, g^{\text{pert}}) \leq 0$, i.e. both copies stable. But a neuron stable on both copies has no binary on either side (the big- M encoding short-circuits on $l \geq 0$ or $u \leq 0$ before ever allocating a), so the decision-precompute step (`compute_n2_relax_decision!`, line 85 of `utils/n2_relax_decision.jl`) skips it via an early `continue`. **The both-thin tier is empty.**
- *One-thin* requires $\min(g^{\text{org}}, g^{\text{pert}}) \leq 0$ with the other > 0 . The relaxed copy c is the stable side; the sibling is genuinely split and keeps its binary. But on the stable side `relu()` short-circuits at `core_ops.jl:304/316` (returns 0 for $u \leq 0$ or x for $l \geq 0$) *before* the Per-Copy Triangle Drop triangle path at line 562. No triangle is emitted on the stable copy — and correspondingly `n2_relu_state[(..., c)]` stays `nothing`. `apply_sibgate_constraints!` sees `st_c === nothing` at line 64 and `continues` without writing any constraint. **The one-thin tier emits nothing too.**
- *Neither-thin* is the default for every genuinely split neuron ($\text{gap} > 0 > \tau$), and the standard exact big- M encoding is used.

Every neuron of the N_2 MIP therefore receives the exact big- M encoding it would have received without any of Per-Copy Triangle Drop or Sibling-Gated Refinement active. The bound globals SibGate populates (`relu_n2pert_*_bounds`, `n2_preact_*_bounds` via `compute_n2_pert_relaxation_bounds` at `run.jl:971-973`) are consulted only by SibGate’s own post-encoding pass (which we just established is a no-op at $\tau = 0$) and by unrelated niche flags (`--no_n1_binaries_and_relaxations_only_on_n2`, `--constrain_n2_xp_via_n1_zonotope`); `intersect_per_copy_bounds` (Sources A/B/C) never reads them. Consequently

$$\mathcal{F}_2^{(\text{SibGate}, \tau=0)} = \mathcal{F}_2^{(\tau=0)} = \mathcal{F}_2,$$

and at termination `OPTIMAL`

$$\delta_{\text{SibGate}}|_{\tau=0} = \delta_{\text{BTPR}}|_{\tau=0} = \delta_{\text{exact}}.$$

Equality is *constraint-level*: the same MIP is handed to Gurobi in both modes. Reported wall-clock t_{adv} and the incumbent returned at `TIME.LIMIT` may still differ between the two because Gurobi’s

parallel branch-and-bound is nondeterministic at the default settings used here (**Threads**=32, **MIPFocus**=3, no fixed **Seed**); two re-runs of either configuration alone exhibit the same wallclock variance.

4.5.1 Algorithm and worked example

Algorithm 8 Sibling-Gated Refinement on N_2

Require: N_2 with dual encoding, threshold τ , per-copy bounds $(l_{m,k}^{N_2,c}, u_{m,k}^{N_2,c})$ from Bound Tightening, pre-act diff bound $(l_{\text{int}}, u_{\text{int}})_{m,k}$ propagated by the interval recurrence of Section 3.1 at the pre-ReLU site, tier assignment from Algorithm 7

- 1: **for** each ReLU neuron (m, k) in N_2 **do**
- 2: **if** tier is *both thin* **then**
- 3: keep the two simple triangles already emitted by Algorithm 7 on each copy
- 4: emit pre-act coupling row eq. (38):

$$l_{\text{int}} \leq \hat{z}_{m,k}^{\text{pert}} - \hat{z}_{m,k}^{\text{org}} \leq u_{\text{int}}$$
- 5: **else if** tier is *one thin*, relaxed copy c , kept sibling pre **then**
- 6: compute (l_A, u_A, l_I, u_I) via eqs. (36)–(37)
- 7: $M \leftarrow u_{m,k}^{N_2,c} + |l_{m,k}^{N_2,c}|$
- 8: replace the simple triangle on copy c with the conditional triangle rows eqs. (33)–(35), each upper bound gated on existing sibling binary $a_{m,k}^{\text{pre}}$
- 9: **else**
- 10: leave both copies with their exact big- M encoding
- 11: **end if**
- 12: **end for**

Variables / constraints added (per tier). *Both thin:* 1 new row (pre-act coupling) on top of Per-Copy Triangle Drop’s 6 triangle rows; no new variable. *One thin:* 2 lower-bound rows (eq. 33) + 2 conditional upper-bound rows (eqs. 34–35) on copy c , each upper bound multiplied by the existing sibling binary $a_{m,k}^{\text{pre}}$ via a big- M slack; no new variable. *Neither thin:* unchanged.

Worked example (one-thin tier). Same N_2 pair and bounds as Section 4.4.1. At $\tau = 0.15$, neuron (1, 1) is one-thin with $c = \text{org}$, pre = pert.

Pre-act diff bound at neuron (1, 1). Using the *interval*-arithmetic recurrence of Section 3.1 at the pre-ReLU site (“How $[l_{\text{int}}, u_{\text{int}}]$ is computed in the actual code” paragraph): $(l_{\text{int}}, u_{\text{int}}) = (-0.2, +0.2)$.

Apply eqs. (36)–(37).

$$\begin{aligned}
l_A &= \max(l_{\text{int}}, l_{1,1}^{N_2,\text{org}}) = \max(-0.2, -0.3) = -0.2 \\
u_A &= \min(u_{1,1}^{N_2,\text{pert}} + u_{\text{int}}, u_{1,1}^{N_2,\text{org}}) = \min(2.0 + 0.2, 1.8) = 1.8 \\
l_I &= \max(l_{1,1}^{N_2,\text{pert}} + l_{\text{int}}, l_{1,1}^{N_2,\text{org}}) = \max(-0.5 + (-0.2), -0.3) = -0.3 \\
u_I &= \min(u_{\text{int}}, u_{1,1}^{N_2,\text{org}}) = \min(0.2, 1.8) = 0.2 \\
M &= u_{1,1}^{N_2,\text{org}} + |l_{1,1}^{N_2,\text{org}}| = 2.1.
\end{aligned}$$

Emitted on the relaxed org-copy. The two lower bounds (eq. 33) plus the two conditional uppers

(eqs. 34–35):

$$\begin{aligned}
z_{1,1}^{\text{org}} &\geq 0, & z_{1,1}^{\text{org}} &\geq \hat{z}_{1,1}^{\text{org}}, \\
z_{1,1}^{\text{org}} &\leq \frac{1.8}{1.8 - (-0.2)} (\hat{z}_{1,1}^{\text{org}} - (-0.2)) + 2.1 (1 - a_{1,1}^{\text{pert}}) = 0.9 (\hat{z}_{1,1}^{\text{org}} + 0.2) + 2.1 (1 - a_{1,1}^{\text{pert}}), \\
z_{1,1}^{\text{org}} &\leq \frac{0.2}{0.2 - (-0.3)} (\hat{z}_{1,1}^{\text{org}} - (-0.3)) + 2.1 a_{1,1}^{\text{pert}} = 0.4 (\hat{z}_{1,1}^{\text{org}} + 0.3) + 2.1 a_{1,1}^{\text{pert}}.
\end{aligned}$$

When $a_{1,1}^{\text{pert}} = 1$ the second row is active and the third is slack; vice versa for $a_{1,1}^{\text{pert}} = 0$. On the active branch SibGate’s envelope is over $[-0.2, 1.8]$ (vs the unconditional $[-0.3, 1.8]$ from Per-Copy Triangle Drop); on the inactive branch it is over $[-0.3, 0.2]$ (vs $[-0.3, 1.8]$).

Worked example (both-thin tier). At $\tau = 0.5$ neuron $(1, 1)$ enters the both-thin tier (both gaps ≤ 0.5). Per-Copy Triangle Drop now emits two simple triangles, one on each copy ($[-0.3, 1.8]$ and $[-0.5, 2.0]$). SibGate *additionally* emits the coupling row

$$-0.2 \leq \hat{z}_{1,1}^{\text{pert}} - \hat{z}_{1,1}^{\text{org}} \leq +0.2$$

on the existing pre-activation variables, with no binary and no big- M . This forbids the LP relaxation from choosing the two pre-acts independently — a constraint the perturbed-interval post-act coupling already enforces post-ReLU but not pre-ReLU (Section 3.1.6, item (iii)).

5 Safe-to-Use Combinations

Tables below list *every* recorded advstd combination at all seeds, all τ values, evaluated per-architecture and broken out by perturbation type and size. Each row corresponds to a single $(c_{\text{src}}, c_{\text{tgt}})$ cell of one combo (rows for the same combo are grouped, with the flag columns shown only on the first cell of the combo). We report wall-clock `t_std` and `t_adv` per cell, the per-cell speedup `sp = t_std/t_adv` (higher is better; > 1 means advstd finished faster than the with-perturbed-intervals baseline), $\overline{\text{sp}}_{c_{\text{src}}}$ (mean `sp` over all tested c_{tgt} for the same c_{src} within a combo, shown once per c_{src} group), and `n_relax` (per-cell number of N2 binary variables removed by Technique 3). The `bound_tight` column labels which Technique 1 variant is used (`interval` / `zono` / `none`; `n1Probe` is always off), and the τ column is the Technique 3 threshold. Within each (perturbation, size) block combos are sorted by descending mean `sp` (fastest advstd first); cells within a combo are sorted by $(c_{\text{src}}, c_{\text{tgt}})$. `mipStart` and `lpBasis` are always off and omitted for brevity. Cells where both modes hit Gurobi’s `TIME_LIMIT` are shown with their wall-clock times; for those cells, a separate per-arch gap-comparison table reports how often advstd reached a tighter $u - l$ bound on the objective than standard, alongside the mean gap of each method.

Note: every table below — the overall combination ranking, the per-arch perturbation tables, and the per-arch `TIME_LIMIT` gap-comparison tables — is restricted to the 2 combinations `zono:prev_pgd:0.0+sg`, `zono:prev_pgd:0.5+sg` (via `--combination_table`).

#	bound_tight	varHint	τ	$\overline{sp}$	n_wins	n_cells	win_rate	$\langle \text{arch}, \text{pert}, \text{p_size} \rangle$ where standard beats advstd on every c_{src}	$\langle \text{arch}, c_{src} \rangle$ where, on <i>every</i> tested $(\text{pert}, \text{p_size})$, advstd was either faster on average ($\overline{sp} > 1$), or both methods averaged at the wall-clock cap ($\overline{t_{std}} \approx \overline{t_{adv}} \approx T$, so the $\overline{sp} \leq 1$ verdict is a timeout artefact) AND advstd's $u-l$ gap on the timeout cells was no worse than standard's (entries needing this timeout-tied clause for at least one $(\text{pert}, \text{p_size})$ are tagged <i>(tied at TIME_LIMIT)</i>)
1	zono	prev_pgd	0.5+sg	1.330	352	548	64.23%	<cn1,translation,3,3>	<3x50,1> <i>(tied at TIME_LIMIT)</i> ; <3x50,2> <i>(tied at TIME_LIMIT)</i> ; <3x50,0> <i>(tied at TIME_LIMIT)</i>
2	zono	prev_pgd	0.0+sg	1.268	340	540	62.96%	none	<3x10,1>; <3x50,2> <i>(tied at TIME_LIMIT)</i> ; <3x50,0> <i>(tied at TIME_LIMIT)</i>

Table 1: Overall advstd combination ranking at all seeds, all τ values — one row per flag combination, aggregated over every $(\text{arch}, \text{pert}, \text{size}, c_{src}, c_{tgt})$ cell in the per-arch tables below. $\overline{sp}$: mean per-cell $sp = t_{std}/t_{adv}$; n_wins counts cells with $sp > 1$; win_rate=n_wins/n_cells (bold when $> 50\%$). losers: $\langle \text{arch}, \text{pert}, \text{p_size} \rangle$ triples where every tested c_{src} averaged $sp < 1$. winners: $\langle \text{arch}, c_{src} \rangle$ pairs where every tested $(\text{pert}, \text{p_size})$ averaged $sp > 1$, or $\overline{t_{std}}$ and $\overline{t_{adv}}$ both sat within 60 s of the observed TIME_LIMIT cap *and* the gap rule ($\overline{gap}_{adv} \leq \overline{gap}_{std}$ or $|\overline{gap}_{std} - \overline{gap}_{adv}| < 0.05$) holds on the timeout cells (such entries are tagged *(tied at TIME_LIMIT)*); * flags partial coverage (some $(\text{pert}, \text{p_size})$ for this arch were not exercised by this combo). Sorted by descending win_rate, then $\overline{sp}$; $\overline{sp} > 1$ in **bold**. Rows are shaded blue.

Per-perturbation histograms: $\overline{sp}$ paired with win-rate

The figures below answer “which advstd combination should I pick on this perturbation $\times$ architecture?” from *two* angles at once. Each row of paired charts shows a single $\langle \text{arch}, \text{pert} \rangle$ slice twice: the **left** chart reports the combo’s mean speed-up (how fast it is on average), the **right** chart reports its win rate (how often it actually beats standard). Combos occupy a **stable x position** across both charts of a row *and* across every row of an arch (alphabetical on the combo abbreviation), so a reader can scan one column down the page and watch a combo’s behaviour change as the perturbation type changes in either metric.

Left chart — bar height = the arithmetic mean of $\overline{sp}$ across every $c_{\text{src}} \times c_{\text{tgt}} \times \text{seed}$ cell across all p_sizes of the perturbation type. **Each bar is vertically split** into a *solid bottom* and a *faded + diagonally hatched top* in the same combo colour: the solid fraction equals $\text{wins}/\text{p_sizes}$, where a p_size’s win is defined by a *hybrid* rule that mirrors the orange gap-comparison table’s treatment of the two cell populations — a p_size in which every cell finished (no `TIME LIMIT`) wins iff $\overline{sp} > 1$; a p_size in which every cell hit `TIME LIMIT` wins iff $\overline{\text{gap}}_{\text{adv}} < \overline{\text{gap}}_{\text{std}}$ across its timeout cells; a p_size with cells of both kinds is judged *per cell* and wins iff a majority of cells win on their own type (each non-timeout cell votes *yes* when $sp > 1$, each timeout cell votes *yes* when its $u - l$ gap on advstd is tighter than on standard). Bars above 3.0 are clipped at that height and the true value is printed in **bold** above the bar; the dashed line marks the $\overline{sp} = 1$ breakeven.

Right chart — each column encodes three quantities by nested rectangles on an absolute-count y-axis (scale $0 \rightarrow n_{\text{max}}$, where n_{max} is the largest n_cells of any combo on the chart, marked by a dashed reference line): a faint *envelope* of height n_{max} (maximum coverage anywhere on the chart); a mid-grey *tests* rectangle of height n_cells (this combo’s actual coverage on the slice); and a coloured *wins* bar inside, split into a *solid bottom* = finished wins (cells where the run finished and advstd was faster, $sp > 1$) and a *faded + hatched top* = timeout wins (cells where both modes hit `TIME LIMIT` and advstd’s $\overline{\text{gap}}_{\text{adv}} < \overline{\text{gap}}_{\text{std}}$). The combo’s **win rate** is therefore the fraction of the mid-grey rectangle that is coloured; an under-tested combo’s column is visibly shorter than its envelope. A combo with no timeout cells shows a fully solid coloured bar, and a combo whose wins all came from tighter-gap timeout cells shows only a hatched top.

The ★ above each chart marks the best combo on that chart (highest $\overline{sp}$ on the left, highest win rate on the right — the two stars may pick different combos when the orderings disagree). A best-to-worst ranking summary line sits above each chart with the per-combo metric value and the relevant breakdown ($\text{wins}/\text{p_sizes}$ for sp , $(\text{finished} + \text{timeout})/n$ for win rate). Cells with mismatched `--timeout` caps between the two modes are excluded (same rule as the per-arch perturbation tables and the blue overall ranking).

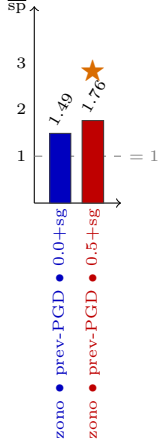
Note: these histograms are restricted to the 2 combinations `zono:prev_pgd:0.0+sg`, `zono:prev_pgd:0.5+sg` (via `--combination_table`).

Architecture **cnn1** — $\overline{sp}$ paired with win-rate

This page shows 4 paired rows, one per **perturbation type** measured on **cnn1**. Each row repeats the same $\langle \text{arch}, \text{pert} \rangle$ slice twice: the *left* chart is the mean-speedup view (section intro above); the *right* chart is the win-rate view (same intro, different bar definition). Combo columns occupy the same x position in both charts of a row, so a column can be scanned down the page to follow one combo across perturbation types in both metrics.

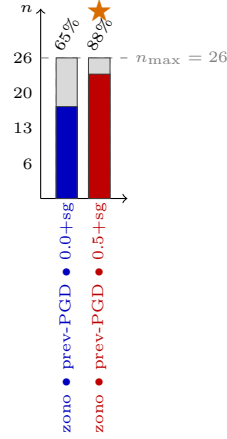
sp **cnn1** | pert **patch** (p.sizes merged: 1,14,14,3) | 2
combos, total $n = 52$ cells.

Ranking (best $\rightarrow$ worst): ★ 1.76 (1/1) ■ 1.49 (1/1)



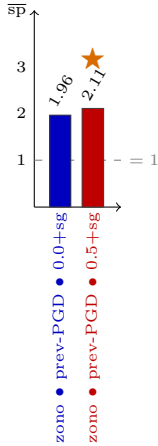
win **cnn1** | pert **patch** (p.sizes merged: 1,14,14,3) | 2
combos, total $n = 52$ cells, $n_{\max} = 26$.

Ranking (best $\rightarrow$ worst): ★ 88% (23f+0t/26)
■ 65% (17f+0t/26)



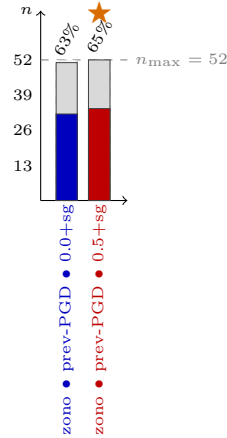
sp **cnn1** | pert **occ** (p.sizes merged: 3,3,5, 5,5,5) | 2
combos, total $n = 103$ cells.

Ranking (best $\rightarrow$ worst): ★ 2.11 (2/2) ■ 1.96 (2/2)



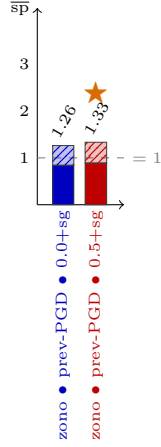
win **cnn1** | pert **occ** (p.sizes merged: 3,3,5, 5,5,5) | 2
combos, total $n = 103$ cells, $n_{\max} = 52$.

Ranking (best $\rightarrow$ worst): ★ 65% (34f+0t/52)
■ 63% (32f+0t/51)



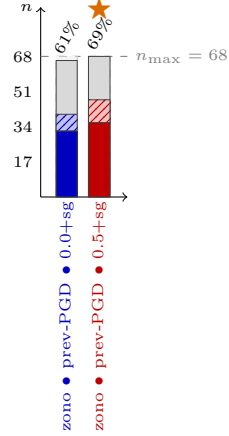
sp cnn1 | pert **translation** (p-sizes merged: 1,1, 3,1, 3,3) | 2 combos, total $n = 134$ cells.

Ranking (best $\rightarrow$ worst): ★■ 1.33 (2/3) ■ 1.26 (2/3)



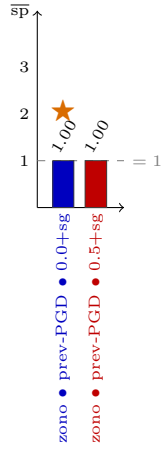
win cnn1 | pert **translation** (p-sizes merged: 1,1, 3,1, 3,3) | 2 combos, total $n = 134$ cells, $n_{\max} = 68$.

Ranking (best $\rightarrow$ worst): ★■ 69% (36f+11t/68) ■ 61% (32f+8t/66)



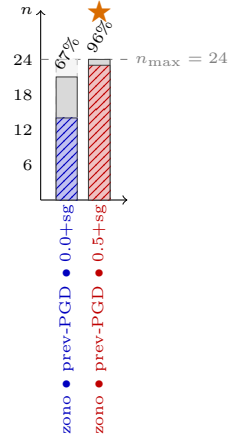
sp cnn1 | pert **linf** (p-sizes merged: 0.1) | 2 combos, total $n = 45$ cells.

Ranking (best $\rightarrow$ worst): ★■ 1.00 (1/1) ■ 1.00 (1/1)



win cnn1 | pert **linf** (p-sizes merged: 0.1) | 2 combos, total $n = 45$ cells, $n_{\max} = 24$.

Ranking (best $\rightarrow$ worst): ★■ 96% (0f+23t/24) ■ 67% (0f+14t/21)

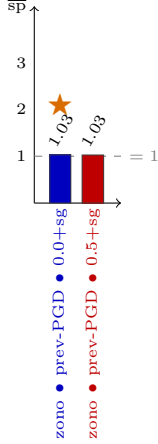


Architecture 3x50 — $\overline{sp}$ paired with win-rate

This page shows 4 paired rows, one per **perturbation type** measured on **3x50**. Each row repeats the same $\langle \text{arch}, \text{pert} \rangle$ slice twice: the *left* chart is the mean-speedup view (section intro above); the *right* chart is the win-rate view (same intro, different bar definition). Combo columns occupy the same x position in both charts of a row, so a column can be scanned down the page to follow one combo across perturbation types in both metrics.

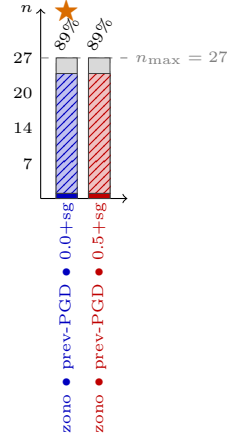
sp 3x50 | pert **patch** (p.sizes merged: 1,14,14,3) | 2
combos, total $n = 54$ cells.

Ranking (best $\rightarrow$ worst): ★ 1.03 (1/1) ■ 1.03 (1/1)



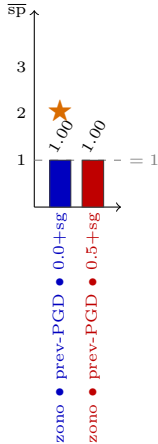
win 3x50 | pert **patch** (p.sizes merged: 1,14,14,3) | 2
combos, total $n = 54$ cells, $n_{\max} = 27$.

Ranking (best $\rightarrow$ worst): ★ 89% (1f+23t/27)
■ 89% (1f+23t/27)



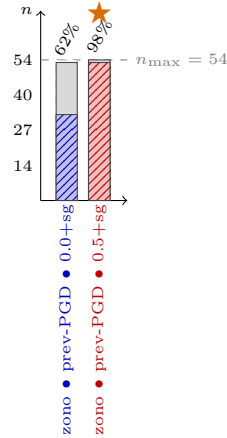
sp 3x50 | pert **occ** (p.sizes merged: 3,3,5, 5,5,5) | 2
combos, total $n = 107$ cells.

Ranking (best $\rightarrow$ worst): ★ 1.00 (2/2) ■ 1.00 (2/2)

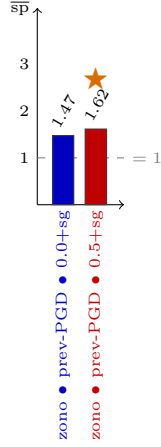


win 3x50 | pert **occ** (p.sizes merged: 3,3,5, 5,5,5) | 2
combos, total $n = 107$ cells, $n_{\max} = 54$.

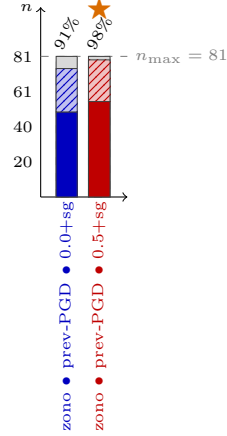
Ranking (best $\rightarrow$ worst): ★ 98% (0f+53t/54)
■ 62% (0f+33t/53)



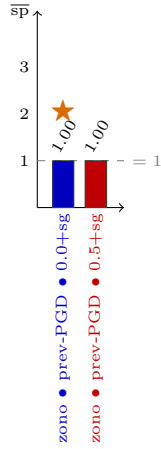
sp 3x50 | pert **translation** (p-sizes merged: 1,1, 3,1, 3,3) | 2 combos, total $n = 162$ cells.
Ranking (best → worst): ★■ 1.62 (3/3) ■ 1.47 (3/3)



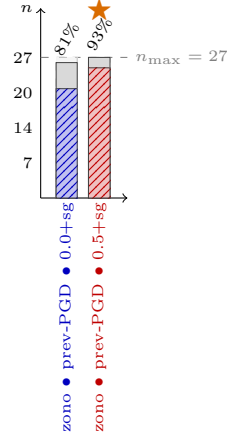
win 3x50 | pert **translation** (p-sizes merged: 1,1, 3,1, 3,3) | 2 combos, total $n = 162$ cells, $n_{\max} = 81$.
Ranking (best → worst): ★■ 98% (55f+24t/81) ■ 91% (49f+25t/81)



sp 3x50 | pert **linf** (p-sizes merged: 0.1) | 2 combos, total $n = 53$ cells.
Ranking (best → worst): ★■ 1.00 (1/1) ■ 1.00 (1/1)



win 3x50 | pert **linf** (p-sizes merged: 0.1) | 2 combos, total $n = 53$ cells, $n_{\max} = 27$.
Ranking (best → worst): ★■ 93% (0f+25t/27) ■ 81% (0f+21t/26)

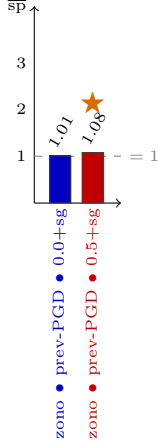


Architecture 3x10 — $\overline{sp}$ paired with win-rate

This page shows 4 paired rows, one per **perturbation type** measured on **3x10**. Each row repeats the same $\langle \text{arch}, \text{pert} \rangle$ slice twice: the *left* chart is the mean-speedup view (section intro above); the *right* chart is the win-rate view (same intro, different bar definition). Combo columns occupy the same x position in both charts of a row, so a column can be scanned down the page to follow one combo across perturbation types in both metrics.

sp 3x10 | pert **patch** (p.sizes merged: 1,14,14,3) | 2
combos, total $n = 54$ cells.

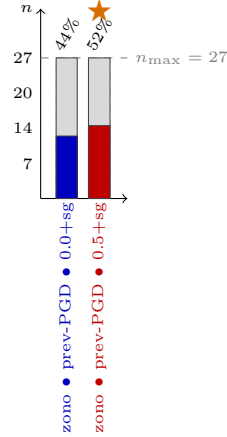
Ranking (best $\rightarrow$ worst): ★ 1.08 (1/1) ■ 1.01 (1/1)



win 3x10 | pert **patch** (p.sizes merged: 1,14,14,3) | 2
combos, total $n = 54$ cells, $n_{\max} = 27$.

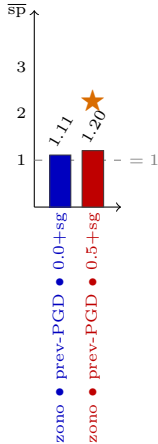
Ranking (best $\rightarrow$ worst): ★ 52% (14f+0t/27)

■ 44% (12f+0t/27)



sp 3x10 | pert **occ** (p.sizes merged: 3,3,5, 5,5,5) | 2
combos, total $n = 108$ cells.

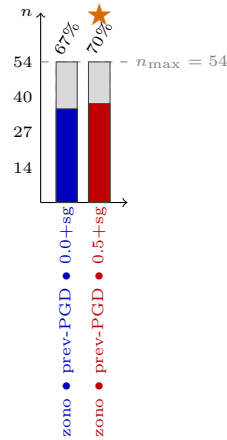
Ranking (best $\rightarrow$ worst): ★ 1.20 (2/2) ■ 1.11 (2/2)



win 3x10 | pert **occ** (p.sizes merged: 3,3,5, 5,5,5) | 2
combos, total $n = 108$ cells, $n_{\max} = 54$.

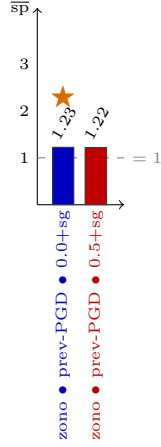
Ranking (best $\rightarrow$ worst): ★ 70% (38f+0t/54)

■ 67% (36f+0t/54)



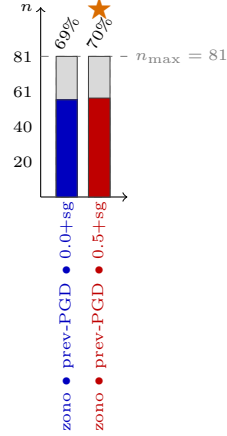
sp 3x10 | pert **translation** (p-sizes merged: 1,1, 3,1, 3,3) | 2 combos, total $n = 162$ cells.

Ranking (best $\rightarrow$ worst): ★■ 1.23 (3/3) ■ 1.22 (3/3)



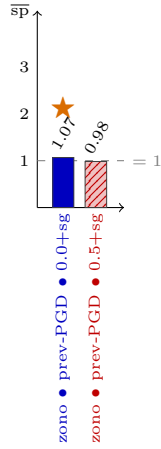
win 3x10 | pert **translation** (p-sizes merged: 1,1, 3,1, 3,3) | 2 combos, total $n = 162$ cells, $n_{\max} = 81$.

Ranking (best $\rightarrow$ worst): ★■ 70% (57f+0t/81) ■ 69% (56f+0t/81)



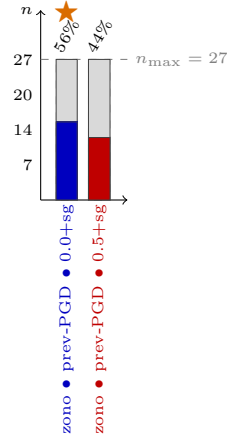
sp 3x10 | pert **linf** (p-sizes merged: 0.1) | 2 combos, total $n = 54$ cells.

Ranking (best $\rightarrow$ worst): ★■ 1.07 (1/1) ■ 0.98 (0/1)



win 3x10 | pert **linf** (p-sizes merged: 0.1) | 2 combos, total $n = 54$ cells, $n_{\max} = 27$.

Ranking (best $\rightarrow$ worst): ★■ 56% (15f+0t/27) ■ 44% (12f+0t/27)

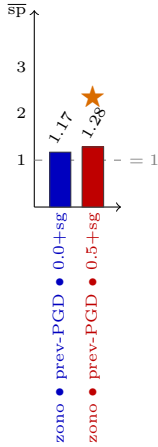


Per-perturbation histograms across all architectures: $\overline{sp}$ paired with win-rate

This section pools every $(c_{src}, c_{tgt}, seed)$ cell across *all* measured architectures, so each paired row reports a combo’s behaviour on one perturbation type aggregated over every arch we ran. Bar definitions, the $\star$ best-combo marker, and the rules used to split each bar into solid + hatched segments are exactly the same as in the per-arch section above. Architecture is shown as **ALL** in each chart’s caption. **Note:** these histograms are restricted to the 2 combinations **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (via `--combination-table`).

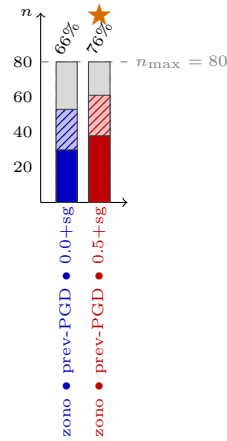
sp **ALL** | pert **patch** (p.sizes merged: 1,14,14,3) | 2 combos, total $n = 160$ cells.

Ranking (best $\rightarrow$ worst): $\star \blacksquare 1.28$ (1/1) $\blacksquare 1.17$ (1/1)



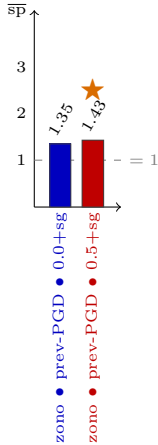
win **ALL** | pert **patch** (p.sizes merged: 1,14,14,3) | 2 combos, total $n = 160$ cells, $n_{max} = 80$.

Ranking (best $\rightarrow$ worst): $\star \blacksquare 76\%$ (38f+23t/80) $\blacksquare 66\%$ (30f+23t/80)



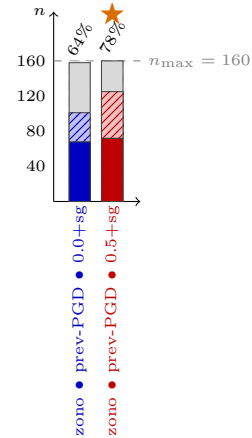
sp **ALL** | pert **occ** (p.sizes merged: 3,3,5, 5,5,5) | 2 combos, total $n = 318$ cells.

Ranking (best $\rightarrow$ worst): $\star \blacksquare 1.43$ (2/2) $\blacksquare 1.35$ (2/2)



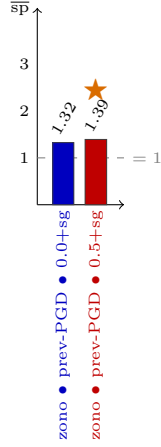
win **ALL** | pert **occ** (p.sizes merged: 3,3,5, 5,5,5) | 2 combos, total $n = 318$ cells, $n_{max} = 160$.

Ranking (best $\rightarrow$ worst): $\star \blacksquare 78\%$ (72f+53t/160) $\blacksquare 64\%$ (68f+33t/158)



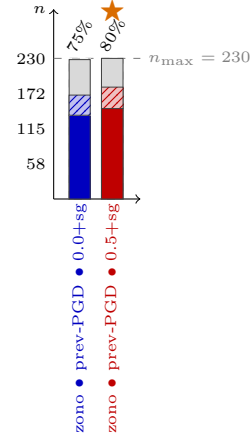
sp ALL | pert **translation** (p.sizes merged: 1,1, 3,1, 3,3) | 2 combos, total $n = 458$ cells.

Ranking (best $\rightarrow$ worst): ★■ 1.39 (3/3) ■ 1.32 (3/3)



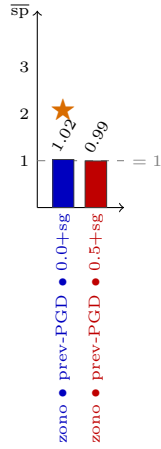
win ALL | pert **translation** (p.sizes merged: 1,1, 3,1, 3,3) | 2 combos, total $n = 458$ cells, $n_{\max} = 230$.

Ranking (best $\rightarrow$ worst): ★■ 80% (148f+35t/230) ■ 75% (137f+33t/228)



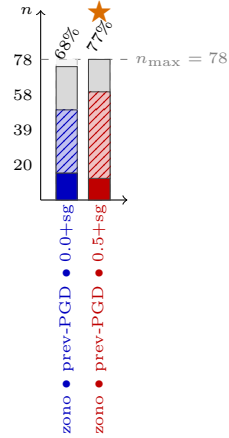
sp ALL | pert **linf** (p.sizes merged: 0.1) | 2 combos, total $n = 152$ cells.

Ranking (best $\rightarrow$ worst): ★■ 1.02 (1/1) ■ 0.99 (1/1)



win ALL | pert **linf** (p.sizes merged: 0.1) | 2 combos, total $n = 152$ cells, $n_{\max} = 78$.

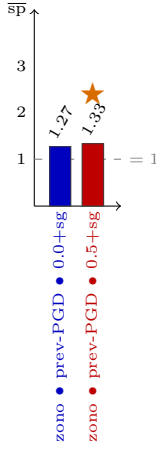
Ranking (best $\rightarrow$ worst): ★■ 77% (12f+48t/78) ■ 68% (15f+35t/74)



Overall histograms across all architectures and perturbations: $\overline{sp}$ paired with win-rate

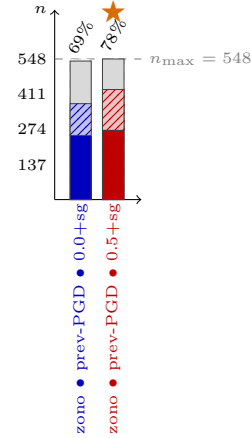
This section pools every $(c_{src}, c_{tgt}, seed)$ cell across *all* architectures *and* *all* perturbation types into a single paired row—one bar per combo summarising every cell on record. Bar definitions are the same as in the sections above; both the architecture and the perturbation are shown as **ALL** in each chart’s caption. **Note:** these histograms are restricted to the 2 combinations `zono:prev_pgd:0.0+sg`, `zono:prev_pgd:0.5+sg` (via `--combination_table`).

sp **ALL** | pert **ALL** (p.sizes merged: 0.1, 1, 1, 1, 14, 14, 3, 3, 1, 3, 3, 3, 3, 5, 5, 5, 5) | 2 combos, total $n = 1088$ cells.
Ranking (best $\rightarrow$ worst): ★ 1.33 (7/7) ■ 1.27 (7/7)



win **ALL** | pert **ALL** (p.sizes merged: 0.1, 1, 1, 1, 14, 14, 3, 3, 1, 3, 3, 3, 3, 5, 5, 5, 5) | 2 combos, total $n = 1088$ cells, $n_{max} = 548$.

Ranking (best $\rightarrow$ worst): ★ 78% (270f+159t/548) ■ 69% (250f+124t/540)



6 Safe-to-Use Combinations: nn1 Boosting

This section mirrors the structure of Section 5 but for **Boosting Standard Mode** (single-network N_1 , Section 3) rather than the $N_1 \rightarrow N_2$ transfer mode. Each combination corresponds to a choice of the three flags introduced in Section 3: `nn1.zono.bound` (Absolute Zonotope Bound Tightening, §3.2), `nn1.relax.threshold` (Per-Copy Triangle Drop, §3.3), and `nn1.sibling.gate` (Sibling-Gated Refinement, §3.4). The baseline against which speedups are reported is the with-perturbed-intervals standard run (no boosts beyond §3.1). Files generated by the standard-mode boosts carry the `_stdBoost_` filename prefix so they never collide with the `_N2.advStd_` files used in Section 5.

The content between the AUTO markers below is regenerated by

```
python3 run_relaxation_sweep.py \
  --find_advstd_faster_than_standard \
  --compare_to_with_perturbed \
  --combo_ranking_seeds 4 \
  --arch_models cnn1=. 3x50=. 3x10=.
```

When the sweep finishes it overwrites the block in place; everything outside the markers (including this paragraph) is preserved.

Tables below summarise every recorded standard-mode boost combination across all sweep seeds, broken out by architecture. `sp = t_base/t_boost` is the per-cell speedup over the with-perturbed-intervals baseline (higher is better; > 1 means the boost was faster). The boost “combo”

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
patch	1,14,14,3	zono	prev_pgd	0.5+sg	0	1	164	81.52	65.97	1.236	93.38	72.80	1.283	0.02
					0	2	164	45.49	36.31	1.253				
					0	3	164	96.17	56.34	1.707				
					0	4	164	328.21	182.41	1.799				
					0	5	164	62.33	39.28	1.587				
					0	6	164	34.38	28.45	1.208				
					0	7	164	71.72	125.19	0.573				
					0	8	164	55.30	54.00	1.024				
					0	9	164	65.33	67.28	0.971				
		zono	prev_pgd	0.0+sg	0	1	0	81.52	43.94	1.855	93.38	81.04	1.152	0.03
					0	2	0	45.49	38.96	1.168				
					0	3	0	96.17	46.80	2.055				
					0	4	0	328.21	288.53	1.138				
					0	5	0	62.33	57.72	1.080				
					0	6	0	34.38	54.04	0.636				
					0	7	0	71.72	77.27	0.928				
					0	8	0	55.30	51.10	1.082				
					0	9	0	65.33	71.03	0.920				
		zono	prev_pgd	0.5+sg	1	0	164	56.34	46.50	1.212	101.81	50.29	2.025	0.02
					1	2	164	54.24	36.01	1.506				
					1	3	164	94.60	26.15	3.618				
					1	4	164	113.19	49.23	2.299				
					1	5	164	138.25	67.20	2.057				
					1	6	164	206.61	122.51	1.686				
					1	7	164	83.24	33.22	2.506				
					1	8	164	67.32	44.88	1.500				
					1	9	164	102.50	26.89	3.812				
		zono	prev_pgd	0.0+sg	1	0	0	56.34	41.71	1.351	101.81	360.41	0.282	0.02
					1	2	0	54.24	66.38	0.817				
					1	3	0	94.60	49.35	1.917				
					1	4	0	113.19	45.67	2.478				
					1	5	0	138.25	48.83	2.831				
					1	6	0	206.61	2654.58	0.078				
					1	7	0	83.24	189.58	0.439				
					1	8	0	67.32	60.46	1.113				
					1	9	0	102.50	87.14	1.176				
		zono	prev_pgd	0.0+sg	2	0	0	79.56	44.07	1.805	140.84	116.38	1.210	0.02
					2	1	0	95.11	225.68	0.421				
					2	3	0	129.59	30.28	4.280				
					2	4	0	130.79	152.97	0.855				
					2	5	0	161.86	92.43	1.751				
					2	7	0	369.96	79.73	4.640				
					2	8	0	88.80	57.25	1.551				
					2	9	0	71.08	248.62	0.286				
		zono	prev_pgd	0.5+sg	2	0	164	79.56	36.21	2.197	140.84	87.84	1.603	0.03
					2	1	164	95.11	78.98	1.204				
					2	3	164	129.59	60.25	2.151				
					2	4	164	130.79	155.45	0.841				
					2	5	164	161.86	141.80	1.141				
					2	7	164	369.96	82.80	4.468				
					2	8	164	88.80	88.15	1.007				
					2	9	164	71.08	59.09	1.203				

Table 2: Architecture **cnn1** — merged per-perturbation table for 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** at all seeds, all τ values; perturbation **patch(1,14,14,3)**. All c_{src} groups are stacked into one table; rows keep their original c_{src} tint (green=0, yellow=1, magenta=2, ...) so groups remain visually distinct. $sp > 1$ (advstd faster) in **bold**; cells within each c_{src} block sorted by c_{tgt} . 52 cell rows. Timeout-cell gaps: Table 9.

p.type	p.size	bound.tight	varHint	τ	c.src	c.tgt	n.relax	t.std	t.adv	sp	$\bar{t}_{c,src}^{std}$	$\bar{t}_{c,src}^{adv}$	$\bar{sp}_{c,src}$	$ \bar{\delta}_{std} - \bar{\delta}_{adv} _{c,src}$
occ	3,3,5	zono	prev_pgd	0.0+sg	0	1	0	261.44	155.79	1.678	151.83	473.88	0.320	0.34
					0	2	0	79.60	59.14	1.346				
					0	3	0	191.10	113.96	1.677				
					0	4	0	367.16	3612.90	0.102				
					0	5	0	134.16	62.84	2.135				
					0	6	0	39.08	57.10	0.684				
					0	7	0	107.68	63.85	1.686				
					0	8	0	41.26	48.21	0.856				
					0	9	0	144.97	91.15	1.590				
		zono	prev_pgd	0.5+sg	0	1	166	261.44	67.56	3.870	151.83	136.22	1.115	0.04
					0	2	166	79.60	45.45	1.751				
					0	3	166	191.10	139.03	1.375				
					0	4	166	367.16	388.84	0.944				
					0	5	166	134.16	168.59	0.796				
					0	6	166	39.08	66.42	0.588				
					0	7	166	107.68	143.30	0.751				
					0	8	166	41.26	71.71	0.575				
					0	9	166	144.97	135.09	1.073				
		zono	prev_pgd	0.5+sg	1	0	166	304.94	85.45	3.569	120.05	57.06	2.104	0.03
					1	2	166	63.44	36.77	1.725				
					1	3	166	47.05	33.16	1.419				
					1	4	166	51.14	37.84	1.351				
					1	5	166	26.16	35.06	0.746				
					1	7	166	100.71	25.48	3.953				
					1	8	166	40.61	46.48	0.874				
					1	9	166	326.37	156.26	2.089				
		zono	prev_pgd	0.0+sg	1	2	0	63.44	34.03	1.864	93.64	73.17	1.280	0.04
					1	3	0	47.05	68.37	0.688				
					1	4	0	51.14	49.67	1.030				
					1	5	0	26.16	44.96	0.582				
					1	7	0	100.71	43.67	2.306				
					1	8	0	40.61	59.37	0.684				
					1	9	0	326.37	212.09	1.539				
		zono	prev_pgd	0.5+sg	2	0	166	1022.51	67.31	15.191	730.01	132.81	5.497	0.65
					2	1	166	321.37	126.22	2.546				
					2	3	166	593.19	301.00	1.971				
					2	4	166	3620.62	160.69	22.532				
					2	5	166	131.71	54.84	2.402				
					2	6	166	152.80	205.85	0.742				
					2	7	166	206.36	128.36	1.608				
					2	8	166	94.53	48.02	1.969				
					2	9	166	427.00	102.98	4.146				
		zono	prev_pgd	0.0+sg	2	0	0	1022.51	37.38	27.354	730.01	554.17	1.317	1.02
					2	1	0	321.37	197.98	1.623				
					2	3	0	593.19	431.04	1.376				
					2	4	0	3620.62	401.80	9.011				
					2	5	0	131.71	70.26	1.875				
					2	6	0	152.80	53.04	2.881				
					2	7	0	206.36	152.00	1.358				
					2	8	0	94.53	30.27	3.123				
					2	9	0	427.00	3613.73	0.118				

Table 3: Architecture **cnn1** — perturbation **occ(3,3,5)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (51 cell rows); continuation of Table 2. Column meanings: see Table 2.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
occ	5,5,5	zono	prev_pgd	0.0+sg	0	1	0	59.49	31.67	1.878	116.01	88.99	1.304	0.03
					0	2	0	58.98	44.43	1.327				
					0	3	0	42.18	61.63	0.684				
					0	4	0	536.01	364.27	1.471				
					0	5	0	110.91	57.99	1.913				
					0	6	0	33.37	47.44	0.703				
					0	7	0	56.20	58.56	0.960				
					0	8	0	57.73	45.58	1.267				
					0	9	0	89.23	89.33	0.999				
		zono	prev_pgd	0.5+sg	0	1	173	59.49	42.21	1.409	116.01	795.00	0.146	0.32
					0	2	173	58.98	57.44	1.027				
					0	3	173	42.18	58.00	0.727				
					0	4	173	536.01	3611.39	0.148				
					0	5	173	110.91	114.35	0.970				
					0	6	173	33.37	59.67	0.559				
					0	7	173	56.20	3122.22	0.018				
					0	8	173	57.73	41.84	1.380				
					0	9	173	89.23	47.84	1.865				
		zono	prev_pgd	0.5+sg	1	0	173	67.88	61.40	1.106	85.25	63.98	1.333	0.04
					1	2	173	55.31	52.45	1.055				
					1	3	173	73.18	28.98	2.525				
					1	4	173	35.85	39.42	0.909				
					1	5	173	35.61	22.92	1.554				
					1	6	173	41.49	29.14	1.424				
					1	7	173	38.38	31.18	1.231				
					1	8	173	40.33	39.20	1.029				
					1	9	173	379.24	271.09	1.399				
		zono	prev_pgd	0.0+sg	1	0	0	67.88	133.40	0.509	85.25	77.63	1.098	0.02
					1	2	0	55.31	66.19	0.836				
					1	3	0	73.18	50.82	1.440				
					1	4	0	35.85	50.58	0.709				
					1	5	0	35.61	37.22	0.957				
					1	6	0	41.49	24.40	1.700				
					1	7	0	38.38	33.07	1.161				
					1	8	0	40.33	21.97	1.836				
					1	9	0	379.24	281.06	1.349				
		zono	prev_pgd	0.0+sg	2	0	0	43.91	25.70	1.709	93.75	134.70	0.696	0.02
					2	1	0	154.12	202.88	0.760				
					2	3	0	107.60	118.87	0.905				
					2	4	0	86.23	101.67	0.848				
					2	5	0	53.12	35.07	1.515				
					2	6	0	58.42	31.74	1.841				
					2	8	0	40.99	33.72	1.216				
					2	9	0	205.61	527.98	0.389				
		zono	prev_pgd	0.5+sg	2	0	173	43.91	59.53	0.738	93.75	127.17	0.737	0.02
					2	1	173	154.12	128.49	1.199				
					2	3	173	107.60	50.39	2.135				
					2	4	173	86.23	108.42	0.795				
					2	5	173	53.12	35.96	1.477				
					2	6	173	58.42	45.49	1.284				
					2	8	173	40.99	74.80	0.548				
					2	9	173	205.61	514.30	0.400				

Table 4: Architecture **cnn1** — perturbation **occ(5,5,5)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (52 cell rows); continuation of Table 2. Column meanings: see Table 2.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\bar{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
translation	1,1	zono	prev_pgd	0.5+sg	0	1	172	1043.60	310.01	3.366	1010.11	795.77	1.269	0.03
					0	2	172	755.45	186.21	4.057				
					0	3	172	344.88	341.05	1.011				
					0	4	172	442.26	396.50	1.115				
					0	5	172	393.83	478.19	0.824				
					0	6	172	526.31	824.30	0.638				
					0	7	172	568.70	440.72	1.290				
					0	8	172	1392.84	572.91	2.431				
		zono	prev_pgd	0.0+sg	0	1	0	1043.60	399.78	2.610	1010.11	822.58	1.228	0.03
					0	2	0	755.45	526.89	1.434				
					0	3	0	344.88	249.56	1.382				
					0	4	0	442.26	419.90	1.053				
					0	5	0	393.83	343.13	1.148				
					0	6	0	526.31	241.76	2.177				
					0	7	0	568.70	483.01	1.177				
					0	8	0	1392.84	1126.92	1.236				
		zono	prev_pgd	0.0+sg	0	9	0	3623.14	3612.05	1.003	1354.66	1068.37	1.268	0.02
					1	0	0	3296.90	1380.31	2.389				
					1	2	0	3615.34	3611.66	1.001				
					1	3	0	298.04	437.28	0.682				
					1	4	0	932.29	1266.65	0.736				
					1	5	0	226.98	118.01	1.923				
					1	6	0	336.06	278.58	1.206				
					1	7	0	776.98	386.13	2.012				
		zono	prev_pgd	0.5+sg	1	0	172	3296.90	2026.11	1.627	1637.76	1299.68	1.260	0.03
					1	2	172	3615.34	3624.60	0.997				
					1	3	172	298.04	270.63	1.101				
					1	4	172	932.29	502.41	1.856				
					1	5	172	226.98	154.11	1.473				
					1	6	172	336.06	346.33	0.970				
					1	7	172	776.98	451.89	1.719				
					1	9	172	3619.49	3021.35	1.198				
		zono	prev_pgd	0.5+sg	2	0	172	820.71	488.99	1.678	1342.06	1106.28	1.213	0.03
					2	1	172	1824.67	939.35	1.942				
					2	3	172	953.53	636.48	1.498				
					2	4	172	1436.87	885.75	1.622				
					2	5	172	596.85	687.41	0.868				
					2	6	172	712.37	834.01	0.854				
					2	8	172	773.34	757.55	1.021				
					2	9	172	3618.11	3620.72	0.999				
		zono	prev_pgd	0.0+sg	2	0	0	820.71	882.20	0.930	1016.91	1015.89	1.001	0.02
					2	1	0	1824.67	1111.98	1.641				
					2	3	0	953.53	761.88	1.252				
					2	4	0	1436.87	1985.41	0.724				
					2	5	0	596.85	872.16	0.684				
					2	6	0	712.37	787.71	0.904				
					2	8	0	773.34	709.87	1.089				

Table 5: Architecture **cnn1** — perturbation **translation**(1,1), 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (48 cell rows); continuation of Table 2. Column meanings: see Table 2.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \bar{\delta}_{std} - \bar{\delta}_{adv} _{c_src}$
translation	3,1	zono	prev_pgd	0.5+sg	0	1	190	365.11	176.78	2.065	<i>711.98</i>	<i>532.83</i>	1.336	<i>0.03</i>
					0	2	190	531.41	110.69	4.801				
					0	3	190	458.41	246.37	1.861				
					0	4	190	270.41	337.89	0.800				
					0	5	190	285.88	322.52	0.886				
					0	6	190	258.24	260.24	0.992				
					0	7	190	323.42	344.66	0.938				
					0	8	190	296.48	571.62	0.519				
					0	9	190	3618.45	2424.69	1.492				
		zono	prev_pgd	0.0+sg	0	1	0	365.11	395.41	0.923	<i>711.98</i>	<i>733.19</i>	<i>0.971</i>	<i>0.02</i>
					0	2	0	531.41	221.73	2.397				
					0	3	0	458.41	269.29	1.702				
					0	4	0	270.41	491.39	0.550				
					0	5	0	285.88	444.67	0.643				
					0	6	0	258.24	386.00	0.669				
					0	7	0	323.42	302.10	1.071				
					0	8	0	296.48	469.63	0.631				
					0	9	0	3618.45	3618.52	1.000				
		zono	prev_pgd	0.0+sg	1	0	0	275.97	258.69	1.067	<i>332.52</i>	<i>277.90</i>	1.197	<i>0.08</i>
					1	2	0	433.07	271.13	1.597				
					1	3	0	285.99	130.18	2.197				
					1	4	0	111.60	166.50	0.670				
					1	5	0	501.62	126.07	3.979				
					1	6	0	249.93	171.00	1.462				
					1	7	0	245.31	276.57	0.887				
					1	8	0	264.15	526.03	0.502				
					1	9	0	625.05	574.96	1.087				
		zono	prev_pgd	0.5+sg	1	0	190	275.97	381.41	0.724	<i>332.52</i>	<i>397.12</i>	<i>0.837</i>	<i>0.09</i>
					1	2	190	433.07	408.60	1.060				
					1	3	190	285.99	572.12	0.500				
					1	4	190	111.60	143.67	0.777				
					1	5	190	501.62	176.92	2.835				
					1	6	190	249.93	461.89	0.541				
					1	7	190	245.31	664.53	0.369				
					1	8	190	264.15	350.46	0.754				
					1	9	190	625.05	414.52	1.508				
		zono	prev_pgd	0.5+sg	2	0	190	550.71	760.58	0.724	<i>1192.34</i>	<i>1012.26</i>	1.178	<i>0.05</i>
					2	1	190	938.71	567.10	1.655				
					2	3	190	1520.47	459.40	3.310				
					2	5	190	642.29	451.45	1.423				
					2	6	190	732.39	701.63	1.044				
					2	7	190	586.50	669.79	0.876				
					2	8	190	949.03	876.13	1.083				
					2	9	190	3618.64	3611.99	1.002				
		zono	prev_pgd	0.0+sg	2	0	0	550.71	628.69	0.876	<i>1192.34</i>	<i>1170.91</i>	1.018	<i>0.04</i>
					2	1	0	938.71	692.16	1.356				
					2	3	0	1520.47	646.02	2.354				
					2	5	0	642.29	1116.24	0.575				
					2	6	0	732.39	904.90	0.809				
					2	7	0	586.50	668.16	0.878				
					2	8	0	949.03	1098.18	0.864				
					2	9	0	3618.64	3612.92	1.002				

Table 6: Architecture **cnn1** — perturbation **translation(3,1)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (52 cell rows); continuation of Table 2. Column meanings: see Table 2.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$\overline{ \delta_{std} - \delta_{adv} }_{c_src}$
translation	3,3	zono	prev_pgd	0.0+sg	0	1	0	431.25	493.26	0.874	1040.99	839.20	1.240	0.03
					0	2	0	427.19	296.37	1.441				
					0	3	0	800.69	354.13	2.261				
					0	5	0	682.97	368.04	1.856				
					0	6	0	765.68	328.85	2.328				
					0	7	0	1231.36	437.49	2.815				
					0	8	0	375.90	822.23	0.457				
					0	9	0	3612.87	3613.26	1.000				
					0	1	190	431.25	395.71	1.090	1040.99	1257.72	0.828	0.18
					0	2	190	427.19	392.98	1.087				
					0	3	190	800.69	451.63	1.773				
					0	5	190	682.97	357.93	1.908				
					0	6	190	765.68	667.91	1.146				
					0	7	190	1231.36	566.56	2.173				
					0	8	190	375.90	3618.43	0.104				
					0	9	190	3612.87	3614.54	1.000				
		zono	prev_pgd	0.5+sg	1	0	190	3614.59	3624.92	0.997	2422.37	2919.97	0.830	0.14
					1	2	190	3617.57	3624.58	0.998				
					1	4	190	1086.37	3612.77	0.301				
					1	7	190	176.49	124.23	1.421				
					1	9	190	3616.82	3613.35	1.001				
					1	0	0	3614.59	3612.05	1.001	2422.37	2918.28	0.830	1.37
					1	2	0	3617.57	3612.43	1.001				
					1	4	0	1086.37	3613.26	0.301				
					1	7	0	176.49	141.74	1.245				
					1	9	0	3616.82	3611.94	1.001				
		zono	prev_pgd	0.5+sg	2	0	190	3613.68	3612.65	1.000	2840.13	2848.53	0.997	0.15
					2	1	190	3614.48	3612.80	1.000				
					2	3	190	516.59	546.15	0.946				
					2	9	190	3615.78	3622.52	0.998				
					2	0	0	3613.68	3614.24	1.000	2840.13	2913.07	0.975	0.14
					2	1	0	3614.48	3615.08	1.000				
					2	3	0	516.59	810.14	0.638				
					2	9	0	3615.78	3612.84	1.001				

Table 7: Architecture **cnn1** — perturbation **translation(3,3)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (34 cell rows); continuation of Table 2. Column meanings: see Table 2.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \bar{\delta}_{std} - \bar{\delta}_{adv} _{c_src}$
linf	0.1	zono	prev_pgd	0.0+sg	0	1	0	3620.83	3611.20	1.003	<i>3619.12</i>	<i>3611.40</i>	1.002	<i>2.20</i>
					0	2	0	3623.78	3611.38	1.003				
					0	7	0	3618.34	3611.32	1.002				
					0	8	0	3615.99	3611.69	1.001				
					0	9	0	3616.65	3611.43	1.001				
		zono	prev_pgd	0.5+sg	0	1	164	3620.83	3612.45	1.002	<i>3618.67</i>	<i>3612.42</i>	1.002	<i>1.62</i>
					0	2	164	3623.78	3611.63	1.003				
					0	6	164	3616.40	3612.79	1.001				
					0	7	164	3618.34	3612.56	1.002				
					0	8	164	3615.99	3612.98	1.001				
		zono	prev_pgd	0.5+sg	0	9	164	3616.65	3612.09	1.001				
					1	0	164	3621.18	3612.15	1.002	<i>3615.69</i>	<i>3612.32</i>	1.001	<i>6.94</i>
					1	2	164	3618.57	3612.33	1.002				
					1	3	164	3615.92	3615.24	1.000				
					1	4	164	3615.75	3611.40	1.001				
		zono	prev_pgd	0.0+sg	1	5	164	3614.47	3611.86	1.001				
					1	6	164	3613.55	3611.25	1.001				
					1	7	164	3614.64	3612.87	1.000				
					1	8	164	3613.69	3612.00	1.000				
					1	9	164	3613.41	3611.82	1.000				
		zono	prev_pgd	0.0+sg	1	0	0	3621.18	3612.06	1.003	<i>3615.69</i>	<i>3612.53</i>	1.001	<i>6.26</i>
					1	2	0	3618.57	3612.88	1.002				
					1	3	0	3615.92	3611.77	1.001				
					1	4	0	3615.75	3613.25	1.001				
					1	5	0	3614.47	3612.90	1.000				
		zono	prev_pgd	0.0+sg	1	6	0	3613.55	3612.48	1.000				
					1	7	0	3614.64	3613.34	1.000				
					1	8	0	3613.69	3611.43	1.001				
					1	9	0	3613.41	3612.64	1.000				
					2	0	0	3621.18	3612.67	1.002	<i>3617.85</i>	<i>3617.67</i>	1.000	<i>1.37</i>
		zono	prev_pgd	0.0+sg	2	1	0	3619.87	3612.33	1.002				
					2	3	0	3618.63	3620.89	0.999				
					2	5	0	3615.60	3621.80	0.998				
					2	6	0	3621.78	3613.02	1.002				
					2	7	0	3614.04	3621.74	0.998				
		zono	prev_pgd	0.5+sg	2	9	0	3613.86	3621.21	0.998				
					2	0	164	3621.18	3616.59	1.001	<i>3617.29</i>	<i>3617.55</i>	1.000	<i>1.41</i>
					2	1	164	3619.87	3619.83	1.000				
					2	3	164	3618.63	3614.24	1.001				
					2	4	164	3617.08	3619.34	0.999				
		zono	prev_pgd	0.5+sg	2	5	164	3615.60	3620.78	0.999				
					2	6	164	3621.78	3614.29	1.002				
					2	7	164	3614.04	3614.43	1.000				
					2	8	164	3613.55	3623.43	0.997				
					2	9	164	3613.86	3614.99	1.000				

Table 8: Architecture **cnn1** — perturbation **linf(0.1)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (45 cell rows); continuation of Table 2. Column meanings: see Table 2.

p_type	p_size	bound_tight	varHint	τ	c_src	n_timeout	n_advstd_tighter	avg_gap_std	avg_gap_adv	$ \overline{\text{gap}}_{\text{std}} - \overline{\text{gap}}_{\text{adv}} $
translation	1,1	zono	prev_pgd	0.5+sg	0	1	1	1.0368	0.8559	0.1809
		zono	prev_pgd	0.0+sg	0	1	1	1.0368	0.8975	0.1393
		zono	prev_pgd	0.5+sg	1	1	1	1.4000	1.2838	0.1162
		zono	prev_pgd	0.0+sg	1	1	1	1.4000	1.2953	0.1047
		zono	prev_pgd	0.5+sg	2	1	1	2.0917	1.9440	0.1478
translation	3,1	zono	prev_pgd	0.0+sg	0	1	1	0.7186	0.6274	0.0911
		zono	prev_pgd	0.5+sg	2	1	1	1.5167	1.3757	0.1409
		zono	prev_pgd	0.0+sg	2	1	1	1.5167	1.4999	0.0168
translation	3,3	zono	prev_pgd	0.5+sg	1	3	3	2.6573	2.3186	0.3386
		zono	prev_pgd	0.5+sg	2	3	3	3.4984	3.1070	0.3914
		zono	prev_pgd	0.0+sg	2	3	2	3.4984	3.4371	0.0613
		zono	prev_pgd	0.0+sg	0	1	1	1.5665	1.1806	0.3858
		zono	prev_pgd	0.5+sg	0	1	1	1.5665	1.4450	0.1214
		zono	prev_pgd	0.0+sg	1	3	1	2.6573	6.7066	4.0493
linf	0.1	zono	prev_pgd	0.5+sg	2	9	9	30.6444	27.8192	2.8252
		zono	prev_pgd	0.5+sg	1	9	8	42.7590	31.1136	11.6454
		zono	prev_pgd	0.0+sg	1	9	7	42.7590	34.1703	8.5887
		zono	prev_pgd	0.5+sg	0	6	6	20.1494	16.9137	3.2358
		zono	prev_pgd	0.0+sg	2	7	6	31.3586	31.4400	0.0814
		zono	prev_pgd	0.0+sg	0	5	1	19.0649	20.9186	1.8537

Table 9: Architecture **cn1** — gap comparison on cells where both modes hit **TIME_LIMIT**. One row per (combo, c_{src}), aggregated over all tested c_{tgt} . **n_advstd_tighter**: count where advstd’s $u - l$ gap < standard’s. **avg_gap_***: mean $u - l$ per method. **avg_gap_adv < avg_gap_std** in **bold**. Includes every combo with at least one such cell (may include combos absent from Table 2).

is the triple (**nn1_zono_bounds**, τ =**nn1_relax_threshold**, **nn1_sibling_gate**). Rows are sorted by descending win-rate, then mean sp.

7 Per-Architecture Wide Comparison: Boosting Standard Mode

The tables below summarise the per-cell wall-clock time of every standard-mode configuration produced by the boost sweep, broken out by architecture and network role (N_1 or N_2). The two baselines (**t_vaghar**, **t_Pi**) are reported directly from the **vagharNoPerturbed_*** and **vagharWithPerturbed_*** result directories, and the remaining six columns are the **nn1_zono_bounds** $\times$ **nn1_sibling_gate** $\times$ **--use_perturbed_intervals** sub-combinations whose (no, no, no) and (no, no, yes) cells reproduce the two baselines and are therefore omitted. Rows are aggregated to one entry per (arch, dataset, role, pert, p_size, c_{src}) with each displayed time being the mean over all tested c_{tgt} and Gurobi seeds.

The content between the AUTO markers below is regenerated by the same sweep invocation that fills Section 6. When the sweep finishes it overwrites the block in place; everything outside the markers (including this paragraph) is preserved.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
patch	1,14,14,3	zono	prev_pgd	0.0+sg	0	1	0	3613.40	3616.30	0.999	3617.48	3619.25	1.000	0.27
					0	2	0	3618.08	3611.16	1.002				
					0	3	0	3618.72	3624.71	0.998				
					0	4	0	3618.68	3620.86	0.999				
					0	5	0	3618.15	3618.51	1.000				
					0	6	0	3618.16	3623.01	0.999				
					0	7	0	3617.37	3621.18	0.999				
					0	8	0	3616.74	3618.40	1.000				
					0	9	0	3618.01	3619.09	1.000				
		zono	prev_pgd	0.5+sg	0	1	23	3613.40	3622.61	0.997	3617.48	3622.41	0.999	0.21
					0	2	23	3618.08	3622.09	0.999				
					0	3	23	3618.72	3620.24	1.000				
					0	4	24	3618.68	3629.03	0.997				
					0	5	23	3618.15	3620.84	0.999				
					0	6	23	3618.16	3620.09	0.999				
					0	7	23	3617.37	3620.72	0.999				
					0	8	24	3616.74	3622.79	0.998				
					0	9	23	3618.01	3623.26	0.999				
		zono	prev_pgd	0.0+sg	1	0	0	2559.91	1431.08	1.789	3500.67	3372.48	1.038	0.25
					1	2	0	3620.66	3613.17	1.002				
					1	3	0	3623.24	3615.39	1.002				
					1	4	0	3619.01	3616.05	1.001				
					1	5	0	3616.74	3615.20	1.000				
					1	6	0	3618.00	3615.03	1.001				
					1	7	0	3616.20	3615.62	1.000				
					1	8	0	3616.24	3615.62	1.000				
					1	9	0	3616.01	3615.12	1.000				
		zono	prev_pgd	0.5+sg	1	0	23	2559.91	1502.27	1.704	3500.67	3379.65	1.036	0.30
					1	2	23	3620.66	3621.53	1.000				
					1	3	23	3623.24	3615.57	1.002				
					1	4	23	3619.01	3612.10	1.002				
					1	5	23	3616.74	3613.19	1.001				
					1	6	23	3618.00	3613.04	1.001				
					1	7	23	3616.20	3612.64	1.001				
					1	8	23	3616.24	3613.44	1.001				
					1	9	23	3616.01	3613.10	1.001				
		zono	prev_pgd	0.5+sg	2	0	23	3615.42	3612.58	1.001	3616.50	3615.40	1.000	0.34
					2	1	23	3615.28	3617.11	0.999				
					2	3	23	3617.35	3619.21	0.999				
					2	4	23	3616.63	3612.97	1.001				
					2	5	23	3615.98	3611.79	1.001				
					2	6	23	3616.01	3613.02	1.001				
					2	7	23	3617.01	3616.71	1.000				
					2	8	23	3617.34	3617.45	1.000				
					2	9	23	3617.45	3617.80	1.000				
		zono	prev_pgd	0.0+sg	2	0	0	3615.42	3611.30	1.001	3616.50	3618.64	0.999	0.32
					2	1	0	3615.28	3611.51	1.001				
					2	3	0	3617.35	3612.06	1.001				
					2	4	0	3616.63	3622.72	0.998				
					2	5	0	3615.98	3616.93	1.000				
					2	6	0	3616.01	3624.31	0.998				
					2	7	0	3617.01	3623.01	0.998				
					2	8	0	3617.34	3621.82	0.999				
					2	9	0	3617.45	3624.09	0.998				

Table 10: Architecture **3x50** — merged per-perturbation table for 2 combos `zono:prev_pgd:0.0+sg`, `zono:prev_pgd:0.5+sg` at all seeds, all τ values; perturbation **patch(1,14,14,3)**. All c_{src} groups are stacked into one table; rows keep their original c_{src} tint (green=0, yellow=1, magenta=2, ...) so groups remain visually distinct. $sp > 1$ (advstd faster) in **bold**; cells within each c_{src} block sorted by c_{tgt} . 54 cell rows. Timeout-cell gaps: Table 17.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\bar{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
occ	3,3,5	zono	prev_pgd	0.5+sg	0	1	23	3617.16	3612.06	1.001	<i>3615.28</i>	<i>3611.97</i>	1.001	<i>0.03</i>
					0	2	23	3616.52	3611.83	1.001				
					0	3	23	3617.63	3611.20	1.002				
					0	4	23	3616.30	3611.66	1.001				
					0	5	23	3614.28	3611.62	1.001				
					0	6	23	3613.76	3613.91	1.000				
					0	7	23	3614.45	3611.79	1.001				
					0	8	23	3614.15	3610.50	1.001				
					0	9	23	3613.30	3613.12	1.000				
		zono	prev_pgd	0.0+sg	0	1	0	3617.16	3612.53	1.001	<i>3615.28</i>	<i>3618.11</i>	<i>0.999</i>	<i>0.01</i>
					0	2	0	3616.52	3621.39	0.999				
					0	3	0	3617.63	3611.55	1.002				
					0	4	0	3616.30	3621.24	0.999				
					0	5	0	3614.28	3617.85	0.999				
					0	6	0	3613.76	3617.80	0.999				
					0	7	0	3614.45	3618.59	0.999				
					0	8	0	3614.15	3618.95	0.999				
					0	9	0	3613.30	3623.08	0.997				
		zono	prev_pgd	0.0+sg	1	0	0	3614.65	3613.10	1.000	<i>3616.34</i>	<i>3612.91</i>	1.001	<i>0.01</i>
					1	2	0	3616.72	3612.20	1.001				
					1	3	0	3614.32	3612.08	1.001				
					1	4	0	3617.83	3612.64	1.001				
					1	5	0	3617.30	3612.07	1.001				
					1	6	0	3617.82	3613.97	1.001				
					1	7	0	3616.42	3612.67	1.001				
					1	8	0	3616.80	3613.65	1.001				
					1	9	0	3615.23	3613.80	1.000				
		zono	prev_pgd	0.5+sg	1	0	23	3614.65	3611.71	1.001	<i>3616.34</i>	<i>3619.97</i>	<i>0.999</i>	<i>0.04</i>
					1	2	23	3616.72	3621.75	0.999				
					1	3	23	3614.32	3621.32	0.998				
					1	4	23	3617.83	3620.90	0.999				
					1	5	23	3617.30	3621.40	0.999				
					1	6	23	3617.82	3622.24	0.999				
					1	7	23	3616.42	3621.38	0.999				
					1	8	23	3616.80	3617.66	1.000				
					1	9	23	3615.23	3621.36	0.998				
		zono	prev_pgd	0.0+sg	2	0	0	3616.87	3615.00	1.001	<i>3614.53</i>	<i>3618.11</i>	<i>0.999</i>	<i>0.01</i>
					2	1	0	3614.95	3619.08	0.999				
					2	3	0	3615.45	3619.04	0.999				
					2	4	0	3613.62	3619.76	0.998				
					2	5	0	3614.10	3619.25	0.999				
					2	6	0	3614.33	3617.37	0.999				
					2	7	0	3613.75	3618.19	0.999				
					2	8	0	3614.17	3617.50	0.999				
					2	9	0	3613.52	3617.80	0.999				
		zono	prev_pgd	0.5+sg	2	0	23	3616.87	3622.47	0.998	<i>3614.53</i>	<i>3620.16</i>	<i>0.998</i>	<i>0.03</i>
					2	1	23	3614.95	3622.11	0.998				
					2	3	23	3615.45	3619.42	0.999				
					2	4	23	3613.62	3620.52	0.998				
					2	5	23	3614.10	3620.22	0.998				
					2	6	23	3614.33	3618.06	0.999				
					2	7	23	3613.75	3621.19	0.998				
					2	8	23	3614.17	3620.13	0.998				
					2	9	23	3613.52	3617.29	0.999				

Table 11: Architecture **3x50** — perturbation **occ(3,3,5)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 10. Column meanings: see Table 10.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \bar{\delta}_{std} - \bar{\delta}_{adv} _{c_src}$
occ	5,5,5	zono	prev_pgd	0.0+sg	0	1	0	3615.00	3620.75	0.998	3614.69	3618.50	0.999	0.05
					0	2	0	3617.07	3614.54	1.001				
					0	3	0	3615.59	3623.60	0.998				
					0	4	0	3617.41	3624.67	0.998				
					0	5	0	3613.89	3614.49	1.000				
					0	6	0	3613.50	3619.43	0.998				
					0	7	0	3613.21	3619.96	0.998				
					0	8	0	3613.49	3612.76	1.000				
					0	9	0	3613.05	3616.30	0.999				
		zono	prev_pgd	0.5+sg	0	1	23	3615.00	3623.17	0.998	3614.69	3622.16	0.998	0.14
					0	2	23	3617.07	3624.05	0.998				
					0	3	23	3615.59	3623.18	0.998				
					0	4	23	3617.41	3621.35	0.999				
					0	5	23	3613.89	3622.05	0.998				
					0	6	23	3613.50	3621.73	0.998				
					0	7	23	3613.21	3621.58	0.998				
					0	8	23	3613.49	3621.10	0.998				
					0	9	23	3613.05	3621.22	0.998				
		zono	prev_pgd	0.0+sg	1	0	0	3623.06	3613.17	1.003	3618.42	3614.62	1.001	0.16
					1	2	0	3619.60	3615.59	1.001				
					1	3	0	3620.09	3615.05	1.001				
					1	4	0	3614.67	3614.86	1.000				
					1	5	0	3617.45	3614.85	1.001				
					1	6	0	3617.67	3614.25	1.001				
					1	8	0	3618.06	3615.00	1.001				
					1	9	0	3616.77	3614.15	1.001				
		zono	prev_pgd	0.5+sg	1	0	23	3623.06	3611.58	1.003	3618.28	3618.82	1.000	0.28
					1	2	23	3619.60	3619.58	1.000				
					1	3	23	3620.09	3620.30	1.000				
					1	4	23	3614.67	3620.14	0.998				
					1	5	23	3617.45	3620.70	0.999				
					1	6	23	3617.67	3618.83	1.000				
					1	7	23	3617.14	3619.42	0.999				
					1	8	23	3618.06	3618.55	1.000				
					1	9	23	3616.77	3620.25	0.999				
		zono	prev_pgd	0.5+sg	2	0	23	3615.93	3621.67	0.998	3617.38	3614.73	1.001	0.13
					2	1	23	3617.68	3620.59	0.999				
					2	3	23	3617.21	3613.59	1.001				
					2	4	23	3618.01	3613.73	1.001				
					2	5	23	3617.55	3612.98	1.001				
					2	6	23	3618.50	3613.09	1.001				
					2	7	23	3618.18	3612.95	1.001				
					2	8	23	3616.99	3612.05	1.001				
					2	9	23	3616.40	3611.89	1.001				
		zono	prev_pgd	0.0+sg	2	0	0	3615.93	3611.49	1.001	3617.38	3620.21	0.999	0.10
					2	1	0	3617.68	3619.02	1.000				
					2	3	0	3617.21	3622.54	0.999				
					2	4	0	3618.01	3621.45	0.999				
					2	5	0	3617.55	3620.51	0.999				
					2	6	0	3618.50	3620.94	0.999				
					2	7	0	3618.18	3621.26	0.999				
					2	8	0	3616.99	3621.92	0.999				
					2	9	0	3616.40	3622.73	0.998				

Table 12: Architecture **3x50** — perturbation **occ(5,5,5)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (53 cell rows); continuation of Table 10. Column meanings: see Table 10.

p-type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
translation	1,1	zono	prev_pgd	0.5+sg	0	1	23	3613.52	3621.61	0.998	3617.07	3617.24	1.000	0.34
					0	2	24	3617.74	3623.27	0.998				
					0	3	24	3619.72	3622.96	0.999				
					0	4	24	3617.42	3621.01	0.999				
					0	5	24	3618.73	3611.97	1.002				
					0	6	24	3615.31	3612.42	1.001				
					0	7	24	3619.08	3612.03	1.002				
					0	8	24	3615.77	3617.73	0.999				
					0	9	24	3616.33	3612.20	1.001				
		zono	prev_pgd	0.0+sg	0	1	0	3613.52	3613.05	1.000	3617.07	3620.78	0.999	0.21
					0	2	0	3617.74	3615.79	1.001				
					0	3	0	3619.72	3615.67	1.001				
					0	4	0	3617.42	3622.29	0.999				
					0	5	0	3618.73	3624.01	0.999				
					0	6	0	3615.31	3625.07	0.997				
					0	7	0	3619.08	3628.42	0.997				
					0	8	0	3615.77	3618.43	0.999				
					0	9	0	3616.33	3624.29	0.998				
		zono	prev_pgd	0.0+sg	1	0	0	2745.23	3055.52	0.898	3070.05	2496.73	1.230	0.01
					1	2	0	3616.54	3612.50	1.001				
					1	3	0	3617.19	3612.52	1.001				
					1	4	0	1263.99	613.07	2.062				
					1	5	0	3617.18	3611.40	1.002				
					1	6	0	3618.79	3612.05	1.002				
					1	7	0	2150.14	1156.04	1.860				
					1	8	0	3617.59	2182.23	1.658				
					1	9	0	3383.79	1015.25	3.333				
		zono	prev_pgd	0.5+sg	1	0	24	2745.23	2657.94	1.033	3070.05	2388.36	1.285	0.02
					1	2	24	3616.54	3615.14	1.000				
					1	3	24	3617.19	3615.10	1.001				
					1	4	24	1263.99	652.21	1.938				
					1	5	24	3617.18	3623.69	0.998				
					1	6	24	3618.79	2541.07	1.424				
					1	7	24	2150.14	1012.84	2.123				
					1	8	24	3617.59	2489.13	1.453				
					1	9	24	3383.79	1288.11	2.627				
		zono	prev_pgd	0.0+sg	2	0	0	3614.32	2586.88	1.397	3378.70	3164.74	1.068	0.24
					2	1	0	3615.11	3614.88	1.000				
					2	3	0	3615.38	3614.61	1.000				
					2	4	0	3615.18	3614.43	1.000				
					2	5	0	3614.44	3614.46	1.000				
					2	6	0	1477.25	592.70	2.492				
					2	7	0	3618.13	3614.75	1.001				
					2	8	0	3619.11	3614.94	1.001				
					2	9	0	3619.35	3614.98	1.001				
		zono	prev_pgd	0.5+sg	2	0	24	3614.32	3518.78	1.027	3378.70	3271.40	1.033	0.18
					2	1	24	3615.11	3612.17	1.001				
					2	3	24	3615.38	3612.25	1.001				
					2	4	24	3615.18	3612.19	1.001				
					2	5	24	3614.44	3612.16	1.001				
					2	6	24	1477.25	630.74	2.342				
					2	7	24	3618.13	3611.19	1.002				
					2	8	24	3619.11	3618.98	1.000				
					2	9	24	3619.35	3614.10	1.001				

Table 13: Architecture **3x50** — perturbation **translation(1,1)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 10. Column meanings: see Table 10.

p-type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
translation	3,1	zono	prev_pgd	0.5+sg	0	1	23	3616.48	3616.00	1.000	<i>2083.79</i>	<i>1137.69</i>	1.832	<i>0.06</i>
					0	2	23	2841.10	822.85	3.453				
					0	3	23	3620.30	1602.10	2.260				
					0	4	23	951.29	477.44	1.992				
					0	5	23	594.58	445.30	1.335				
					0	6	23	368.47	362.64	1.016				
					0	7	23	3618.86	1429.30	2.532				
					0	8	23	654.87	487.15	1.344				
		zono	prev_pgd	0.0+sg	0	9	23	2494.37	996.46	2.503				
					0	1	0	3616.48	3628.04	0.997	<i>2084.48</i>	<i>1295.82</i>	1.609	<i>0.06</i>
					0	2	0	2841.10	994.11	2.858				
					0	3	0	3620.30	2549.50	1.420				
					0	4	0	951.29	516.41	1.842				
					0	5	0	594.58	375.18	1.585				
					0	6	0	368.47	381.44	0.966				
					0	7	0	3618.86	1862.28	1.943				
		zono	prev_pgd	0.5+sg	0	8	0	654.87	423.76	1.545				
					0	9	0	2494.37	931.65	2.677				
					1	0	23	2446.84	359.36	6.809	<i>1049.55</i>	<i>395.96</i>	2.651	<i>0.01</i>
					1	2	23	602.86	256.60	2.349				
					1	3	23	778.37	359.48	2.165				
					1	4	23	688.38	291.95	2.358				
					1	5	23	1028.23	466.64	2.203				
					1	6	23	1934.73	546.76	3.539				
		zono	prev_pgd	0.0+sg	1	7	23	763.76	415.19	1.840				
					1	8	23	611.08	434.45	1.407				
					1	9	23	591.67	433.20	1.366				
					1	0	0	2446.84	755.88	3.237	<i>1049.55</i>	<i>616.49</i>	1.702	<i>0.02</i>
					1	2	0	602.86	550.94	1.094				
					1	3	0	778.37	489.12	1.591				
					1	4	0	688.38	671.10	1.026				
					1	5	0	1028.23	865.48	1.188				
		zono	prev_pgd	0.5+sg	1	6	0	1934.73	592.30	3.266				
					1	7	0	763.76	629.56	1.213				
					1	8	0	611.08	623.35	0.980				
					1	9	0	591.67	370.64	1.596				
					2	0	23	422.11	290.39	1.454	<i>1476.38</i>	<i>1260.99</i>	1.171	<i>0.04</i>
					2	1	23	380.72	236.12	1.612				
					2	3	23	487.03	411.73	1.183				
					2	4	23	3622.68	3613.35	1.003				
		zono	prev_pgd	0.0+sg	2	5	23	466.49	232.10	2.010				
					2	6	23	351.05	277.25	1.266				
					2	7	23	3616.61	2428.29	1.489				
					2	8	23	326.77	247.55	1.320				
					2	9	23	3613.92	3612.16	1.000				
					2	0	0	422.11	342.42	1.233	<i>1476.38</i>	<i>1428.18</i>	1.034	<i>0.02</i>
					2	1	0	380.72	391.32	0.973				
					2	3	0	487.03	429.13	1.135				
		zono	prev_pgd	0.5+sg	2	4	0	3622.68	3613.68	1.002				
					2	5	0	466.49	283.59	1.645				
					2	6	0	351.05	266.46	1.317				
					2	7	0	3616.61	3613.70	1.001				
					2	8	0	326.77	298.99	1.093				
					2	9	0	3613.92	3614.32	1.000				

Table 14: Architecture **3x50** — perturbation **translation(3,1)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 10. Column meanings: see Table 10.

p-type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
translation	3,3	zono	prev_pgd	0.0+sg	0	1	0	3615.53	3054.02	1.184	<i>1124.29</i>	<i>799.32</i>	1.407	<i>0.02</i>
					0	2	0	474.96	181.51	2.617				
					0	3	0	3616.24	2514.58	1.438				
					0	4	0	130.82	204.29	0.640				
					0	5	0	759.69	161.82	4.695				
					0	6	0	234.28	220.33	1.063				
					0	7	0	746.06	332.93	2.241				
					0	8	0	211.17	317.95	0.664				
					0	9	0	329.82	206.46	1.598				
		zono	prev_pgd	0.5+sg	0	1	23	3615.53	3226.85	1.120	<i>1124.29</i>	<i>824.20</i>	1.364	<i>0.01</i>
					0	2	23	474.96	256.57	1.851				
					0	3	23	3616.24	2190.11	1.651				
					0	4	23	130.82	100.84	1.297				
					0	5	23	759.69	367.66	2.066				
					0	6	23	234.28	409.82	0.572				
					0	7	23	746.06	373.90	1.995				
					0	8	23	211.17	251.45	0.840				
					0	9	23	329.82	240.58	1.371				
		zono	prev_pgd	0.5+sg	1	0	23	636.41	523.96	1.215	<i>1206.82</i>	<i>1067.03</i>	1.131	<i>0.07</i>
					1	2	23	593.58	170.60	3.479				
					1	3	23	571.64	221.98	2.575				
					1	4	23	357.66	277.83	1.287				
					1	5	23	3618.14	3612.22	1.002				
					1	6	23	3619.31	3622.15	0.999				
					1	7	23	497.84	256.93	1.938				
					1	8	23	367.76	358.43	1.026				
					1	9	23	599.02	559.13	1.071				
		zono	prev_pgd	0.0+sg	1	0	0	636.41	335.49	1.897	<i>1206.82</i>	<i>1083.44</i>	1.114	<i>0.08</i>
					1	2	0	593.58	317.20	1.871				
					1	3	0	571.64	448.06	1.276				
					1	4	0	357.66	311.76	1.147				
					1	5	0	3618.14	3613.93	1.001				
					1	6	0	3619.31	3613.98	1.001				
					1	7	0	497.84	453.38	1.098				
					1	8	0	367.76	283.65	1.297				
					1	9	0	599.02	373.51	1.604				
		zono	prev_pgd	0.5+sg	2	0	23	680.33	371.31	1.832	<i>855.14</i>	<i>469.42</i>	1.822	<i>0.03</i>
					2	1	23	620.24	256.52	2.418				
					2	3	23	546.26	259.31	2.107				
					2	4	23	2756.43	1439.61	1.915				
					2	5	23	550.33	498.09	1.105				
					2	6	23	711.72	335.25	2.123				
					2	7	23	676.68	293.38	2.306				
					2	8	23	482.65	340.95	1.416				
					2	9	23	671.59	430.36	1.561				
		zono	prev_pgd	0.0+sg	2	0	0	680.33	411.25	1.654	<i>855.14</i>	<i>536.13</i>	1.595	<i>0.02</i>
					2	1	0	620.24	292.27	2.122				
					2	3	0	546.26	480.04	1.138				
					2	4	0	2756.43	1941.09	1.420				
					2	5	0	550.33	223.47	2.463				
					2	6	0	711.72	322.38	2.208				
					2	7	0	676.68	398.08	1.700				
					2	8	0	482.65	440.69	1.095				
					2	9	0	671.59	315.91	2.126				

Table 15: Architecture **3x50** — perturbation **translation(3,3)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 10. Column meanings: see Table 10.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \bar{\delta}_{std} - \bar{\delta}_{adv} _{c_src}$
linf	0.1	zono	prev_pgd	0.0+sg	0	1	0	3616.22	3614.03	1.001	<i>3614.45</i>	<i>3613.99</i>	1.000	<i>0.55</i>
					0	2	0	3616.40	3614.68	1.000				
					0	3	0	3613.34	3611.84	1.000				
					0	4	0	3614.18	3611.45	1.001				
					0	5	0	3614.08	3611.72	1.001				
					0	6	0	3614.68	3611.97	1.001				
					0	7	0	3614.31	3617.20	0.999				
					0	8	0	3613.60	3616.40	0.999				
					0	9	0	3613.23	3616.64	0.999				
					0	1	21	3616.22	3621.67	0.998	<i>3614.45</i>	<i>3618.07</i>	<i>0.999</i>	<i>0.71</i>
		zono	prev_pgd	0.5+sg	0	2	21	3616.40	3617.40	1.000				
					0	3	21	3613.34	3617.91	0.999				
					0	4	21	3614.18	3614.02	1.000				
					0	5	21	3614.08	3618.35	0.999				
					0	6	21	3614.68	3619.22	0.999				
					0	7	21	3614.31	3619.18	0.999				
					0	8	21	3613.60	3616.09	0.999				
					0	9	21	3613.23	3618.80	0.998				
					1	0	21	3615.27	3613.26	1.001	<i>3614.20</i>	<i>3612.13</i>	1.001	<i>0.48</i>
					1	2	21	3618.23	3613.85	1.001				
		zono	prev_pgd	0.5+sg	1	3	21	3613.90	3611.10	1.001				
					1	4	21	3613.86	3611.61	1.001				
					1	5	21	3614.68	3611.79	1.001				
					1	6	21	3613.51	3611.54	1.001				
					1	7	21	3611.58	3611.86	1.000				
					1	8	21	3613.61	3611.91	1.000				
					1	9	21	3613.16	3612.24	1.000				
					1	0	0	3615.27	3620.22	0.999	<i>3614.14</i>	<i>3621.72</i>	<i>0.998</i>	<i>0.89</i>
					1	2	0	3618.23	3623.19	0.999				
					1	3	0	3613.90	3621.28	0.998				
		zono	prev_pgd	0.0+sg	1	4	0	3613.86	3620.81	0.998				
					1	6	0	3613.51	3621.03	0.998				
					1	7	0	3611.58	3621.55	0.997				
					1	8	0	3613.61	3623.15	0.997				
					1	9	0	3613.16	3622.54	0.997				
					2	0	0	3620.38	3612.11	1.002	<i>3617.18</i>	<i>3611.46</i>	1.002	<i>0.50</i>
					2	1	0	3621.54	3611.06	1.003				
					2	3	0	3617.93	3611.22	1.002				
					2	4	0	3617.04	3611.77	1.001				
					2	5	0	3618.97	3610.89	1.002				
		zono	prev_pgd	0.5+sg	2	6	0	3616.75	3611.51	1.001				
					2	7	0	3615.59	3611.48	1.001				
					2	8	0	3613.61	3611.45	1.001				
					2	9	0	3612.82	3611.65	1.000				
					2	0	21	3620.38	3620.61	1.000	<i>3617.18</i>	<i>3618.59</i>	<i>1.000</i>	<i>0.57</i>
					2	1	21	3621.54	3617.71	1.001				
					2	3	21	3617.93	3617.18	1.000				
					2	4	21	3617.04	3618.38	1.000				
					2	5	21	3618.97	3619.25	1.000				
					2	6	21	3616.75	3618.68	0.999				
					2	7	21	3615.59	3619.11	0.999				
					2	8	21	3613.61	3619.06	0.998				
					2	9	21	3612.82	3617.36	0.999				

Table 16: Architecture **3x50** — perturbation **linf(0.1)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (53 cell rows); continuation of Table 10. Column meanings: see Table 10.

p.type	p.size	bound_tight	varHint	τ	c_src	n_timeout	n_advstd_tighter	avg_gap_std	avg_gap_adv	$ \bar{gap}_{std} - \bar{gap}_{adv} $
patch	1,14,14,3	zono	prev_pgd	0.0+sg	0	9	8	4.2376	3.7229	0.5147
		zono	prev_pgd	0.5+sg	0	9	8	4.2376	3.8390	0.3987
		zono	prev_pgd	0.0+sg	2	9	8	5.7273	5.1648	0.5624
		zono	prev_pgd	0.5+sg	2	9	8	5.7273	5.0773	0.6500
		zono	prev_pgd	0.0+sg	1	8	7	3.0814	2.6551	0.4263
		zono	prev_pgd	0.5+sg	1	8	7	3.0814	2.5464	0.5350
occ	3,3,5	zono	prev_pgd	0.5+sg	0	9	9	0.1882	0.1220	0.0662
		zono	prev_pgd	0.5+sg	1	9	9	0.1732	0.0919	0.0813
		zono	prev_pgd	0.5+sg	2	9	9	0.1446	0.0839	0.0607
		zono	prev_pgd	0.0+sg	2	9	6	0.1446	0.1380	0.0066
		zono	prev_pgd	0.0+sg	1	9	5	0.1732	0.1745	0.0012
		zono	prev_pgd	0.0+sg	0	9	4	0.1882	0.1928	0.0046
occ	5,5,5	zono	prev_pgd	0.5+sg	0	9	9	1.3688	1.0800	0.2888
		zono	prev_pgd	0.5+sg	1	9	9	1.4283	0.8701	0.5582
		zono	prev_pgd	0.0+sg	1	8	8	1.4501	1.1343	0.3159
		zono	prev_pgd	0.5+sg	2	9	8	0.9870	0.7399	0.2471
		zono	prev_pgd	0.0+sg	2	9	7	0.9870	0.9130	0.0740
		zono	prev_pgd	0.0+sg	0	9	3	1.3688	1.3461	0.0227
translation	1,1	zono	prev_pgd	0.5+sg	0	9	9	1.9030	1.2254	0.6776
		zono	prev_pgd	0.0+sg	0	9	9	1.9030	1.4746	0.4284
		zono	prev_pgd	0.5+sg	2	7	7	1.6704	1.2286	0.4418
		zono	prev_pgd	0.0+sg	2	7	7	1.6704	1.0857	0.5846
		zono	prev_pgd	0.0+sg	1	4	4	0.2851	0.2513	0.0337
		zono	prev_pgd	0.5+sg	1	3	3	0.3312	0.2455	0.0857
translation	3,1	zono	prev_pgd	0.0+sg	2	3	2	0.5117	0.4969	0.0148
		zono	prev_pgd	0.5+sg	2	2	2	0.6616	0.3908	0.2708
		zono	prev_pgd	0.0+sg	0	1	1	1.8819	1.1351	0.7468
		zono	prev_pgd	0.5+sg	0	1	1	1.8819	1.2142	0.6676
translation	3,3	zono	prev_pgd	0.5+sg	1	2	2	0.7587	0.2054	0.5534
		zono	prev_pgd	0.0+sg	1	2	2	0.7587	0.1495	0.6092
linf	0.1	zono	prev_pgd	0.5+sg	0	9	9	9.5874	8.1594	1.4280
		zono	prev_pgd	0.0+sg	0	9	8	9.5874	8.5966	0.9907
		zono	prev_pgd	0.5+sg	1	9	8	4.0109	3.0644	0.9465
		zono	prev_pgd	0.5+sg	2	9	8	9.1106	8.0544	1.0562
		zono	prev_pgd	0.0+sg	2	9	8	9.1106	8.2197	0.8909
		zono	prev_pgd	0.0+sg	1	8	5	4.0697	4.8788	0.8090

Table 17: Architecture **3x50** — gap comparison on cells where both modes hit `TIME_LIMIT`. One row per (combo, c_{src}), aggregated over all tested c_{tgt} . `n_advstd_tighter`: count where advstd’s $u - l$ gap < standard’s. `avg_gap_*`: mean $u - l$ per method. `avg_gap_adv < avg_gap_std` in **bold**. Includes every combo with at least one such cell (may include combos absent from Table 10).

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
patch	1,14,14,3	zono	prev_pgd	0.5+sg	0	1	0	17.14	20.79	0.824	16.05	18.80	0.854	0.01
					0	2	0	21.33	20.03	1.065				
					0	3	0	14.75	19.82	0.744				
					0	4	0	15.24	18.42	0.827				
					0	5	0	16.13	19.14	0.843				
					0	6	0	13.07	20.07	0.651				
					0	7	0	13.37	19.97	0.670				
					0	8	0	13.05	15.65	0.834				
					0	9	0	20.34	15.28	1.331				
		zono	prev_pgd	0.0+sg	0	1	0	17.14	19.51	0.879	16.05	19.18	0.837	0.01
					0	2	0	21.33	18.69	1.141				
					0	3	0	14.75	19.93	0.740				
					0	4	0	15.24	19.34	0.788				
					0	5	0	16.13	17.98	0.897				
					0	6	0	13.07	22.83	0.572				
					0	7	0	13.37	20.00	0.668				
					0	8	0	13.05	19.93	0.655				
					0	9	0	20.34	14.39	1.413				
		zono	prev_pgd	0.5+sg	1	0	0	19.11	19.84	0.963	21.66	16.93	1.279	0.02
					1	2	0	12.95	19.76	0.655				
					1	3	0	22.55	17.43	1.294				
					1	4	0	23.72	17.73	1.338				
					1	5	0	26.84	18.19	1.476				
					1	6	0	25.81	15.11	1.708				
					1	7	0	26.70	14.57	1.833				
					1	8	0	22.57	15.67	1.440				
					1	9	0	14.65	14.06	1.042				
		zono	prev_pgd	0.0+sg	1	0	0	19.11	18.16	1.052	21.66	17.84	1.214	0.02
					1	2	0	12.95	21.00	0.617				
					1	3	0	22.55	17.81	1.266				
					1	4	0	23.72	17.99	1.319				
					1	5	0	26.84	18.87	1.422				
					1	6	0	25.81	18.22	1.417				
					1	7	0	26.70	20.32	1.314				
					1	8	0	22.57	14.29	1.579				
					1	9	0	14.65	13.89	1.055				
		zono	prev_pgd	0.5+sg	2	0	0	16.30	20.42	0.798	17.44	16.94	1.030	0.01
					2	1	0	12.37	16.90	0.732				
					2	3	0	19.22	18.19	1.057				
					2	4	0	16.27	19.06	0.854				
					2	5	0	15.83	13.43	1.179				
					2	6	0	15.99	14.01	1.141				
					2	7	0	18.06	15.79	1.144				
					2	8	0	26.91	14.44	1.864				
					2	9	0	16.02	20.19	0.793				
		zono	prev_pgd	0.0+sg	2	0	0	16.30	20.45	0.797	17.44	18.97	0.919	0.01
					2	1	0	12.37	17.11	0.723				
					2	3	0	19.22	17.87	1.076				
					2	4	0	16.27	20.57	0.791				
					2	5	0	15.83	18.98	0.834				
					2	6	0	15.99	23.12	0.692				
					2	7	0	18.06	18.19	0.993				
					2	8	0	26.91	16.79	1.603				
					2	9	0	16.02	17.69	0.906				

Table 18: Architecture **3x10** — merged per-perturbation table for 2 combos `zono:prev_pgd:0.0+sg`, `zono:prev_pgd:0.5+sg` at all seeds, all τ values; perturbation `patch(1,14,14,3)`. All c_{src} groups are stacked into one table; rows keep their original c_{src} tint (green=0, yellow=1, magenta=2, ...) so groups remain visually distinct. $sp > 1$ (advstd faster) in **bold**; cells within each c_{src} block sorted by c_{tgt} . 54 cell rows.

p.type	p.size	bound.tight	varHint	τ	c.src	c.tgt	n.relax	t.std	t.adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
occ	3,3,5	zono	prev_pgd	0.5+sg	0	1	0	21.45	17.45	1.229	16.60	16.51	1.005	0.00
					0	2	0	20.52	18.47	1.111				
					0	3	0	14.59	16.80	0.868				
					0	4	0	16.49	15.77	1.046				
					0	5	0	14.70	15.12	0.972				
					0	6	0	13.77	16.08	0.856				
					0	7	0	15.26	16.52	0.924				
					0	8	0	13.88	17.49	0.794				
					0	9	0	18.70	14.86	1.258				
		zono	prev_pgd	0.0+sg	0	1	0	21.45	18.96	1.131	16.60	16.84	0.986	0.00
					0	2	0	20.52	21.32	0.962				
					0	3	0	14.59	16.93	0.862				
					0	4	0	16.49	17.14	0.962				
					0	5	0	14.70	17.29	0.850				
					0	6	0	13.77	15.66	0.879				
					0	7	0	15.26	14.15	1.078				
					0	8	0	13.88	15.10	0.919				
					0	9	0	18.70	14.99	1.247				
		zono	prev_pgd	0.5+sg	1	0	0	20.57	11.60	1.773	21.67	12.97	1.671	0.02
					1	2	0	19.27	14.71	1.310				
					1	3	0	26.38	12.51	2.109				
					1	4	0	22.64	11.30	2.004				
					1	5	0	26.24	12.79	2.052				
					1	6	0	24.21	13.52	1.791				
					1	7	0	14.84	13.25	1.120				
					1	8	0	24.15	12.33	1.959				
					1	9	0	16.72	14.72	1.136				
		zono	prev_pgd	0.0+sg	1	0	0	20.57	15.99	1.286	21.67	14.99	1.446	0.02
					1	2	0	19.27	17.27	1.116				
					1	3	0	26.38	18.67	1.413				
					1	4	0	22.64	14.33	1.580				
					1	5	0	26.24	14.05	1.868				
					1	6	0	24.21	13.41	1.805				
					1	7	0	14.84	13.50	1.099				
					1	8	0	24.15	12.81	1.885				
					1	9	0	16.72	14.85	1.126				
		zono	prev_pgd	0.5+sg	2	0	0	20.11	12.57	1.600	15.14	13.23	1.145	0.02
					2	1	0	19.36	14.16	1.367				
					2	3	0	16.57	13.51	1.226				
					2	4	0	12.81	12.61	1.016				
					2	5	0	16.13	12.97	1.244				
					2	6	0	12.93	13.17	0.982				
					2	7	0	13.80	13.12	1.052				
					2	8	0	12.05	14.08	0.856				
					2	9	0	12.49	12.84	0.973				
		zono	prev_pgd	0.0+sg	2	0	0	20.11	14.73	1.365	15.14	14.30	1.059	0.02
					2	1	0	19.36	19.13	1.012				
					2	3	0	16.57	19.74	0.839				
					2	4	0	12.81	15.58	0.822				
					2	5	0	16.13	11.46	1.408				
					2	6	0	12.93	12.05	1.073				
					2	7	0	13.80	11.91	1.159				
					2	8	0	12.05	12.55	0.960				
					2	9	0	12.49	11.54	1.082				

Table 19: Architecture **3x10** — perturbation **occ(3,3,5)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 18. Column meanings: see Table 18.

p.type	p.size	bound.tight	varHint	τ	c.src	c.tgt	n.relax	t.std	t.adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\overline{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
occ	5,5,5	zono	prev_pgd	0.5+sg	0	1	0	23.57	17.23	1.368	<i>18.86</i>	<i>17.26</i>	1.093	<i>0.01</i>
					0	2	0	25.54	18.15	1.407				
					0	3	0	21.76	20.26	1.074				
					0	4	0	15.18	17.71	0.857				
					0	5	0	15.59	17.86	0.873				
					0	6	0	14.17	15.65	0.905				
					0	7	0	17.19	16.09	1.068				
					0	8	0	14.13	17.96	0.787				
					0	9	0	22.61	14.39	1.571				
		zono	prev_pgd	0.0+sg	0	1	0	23.57	17.22	1.369	<i>18.86</i>	<i>18.81</i>	1.003	<i>0.01</i>
					0	2	0	25.54	20.15	1.267				
					0	3	0	21.76	21.41	1.016				
					0	4	0	15.18	18.98	0.800				
					0	5	0	15.59	20.59	0.757				
					0	6	0	14.17	19.04	0.744				
					0	7	0	17.19	17.34	0.991				
					0	8	0	14.13	18.36	0.770				
					0	9	0	22.61	16.16	1.399				
		zono	prev_pgd	0.5+sg	1	0	0	19.80	13.61	1.455	<i>15.82</i>	<i>13.01</i>	1.216	<i>0.02</i>
					1	2	0	12.88	11.73	1.098				
					1	3	0	16.02	12.64	1.267				
					1	4	0	14.21	11.16	1.273				
					1	5	0	16.39	16.01	1.024				
					1	6	0	17.03	11.81	1.442				
					1	7	0	16.60	14.63	1.135				
					1	8	0	15.69	12.98	1.209				
					1	9	0	13.80	12.56	1.099				
		zono	prev_pgd	0.0+sg	1	0	0	19.80	15.06	1.315	<i>15.82</i>	<i>15.26</i>	1.037	<i>0.02</i>
					1	2	0	12.88	18.57	0.694				
					1	3	0	16.02	17.63	0.909				
					1	4	0	14.21	14.19	1.001				
					1	5	0	16.39	15.27	1.073				
					1	6	0	17.03	13.61	1.251				
					1	7	0	16.60	14.59	1.138				
					1	8	0	15.69	15.26	1.028				
					1	9	0	13.80	13.19	1.046				
		zono	prev_pgd	0.0+sg	2	0	0	20.38	17.52	1.163	<i>18.69</i>	<i>17.61</i>	1.061	<i>0.02</i>
					2	1	0	20.09	19.96	1.007				
					2	3	0	21.28	16.50	1.290				
					2	4	0	17.96	17.85	1.006				
					2	5	0	16.39	15.72	1.043				
					2	6	0	15.98	18.20	0.878				
					2	7	0	19.00	22.19	0.856				
					2	8	0	21.74	17.31	1.256				
					2	9	0	15.39	13.26	1.161				
		zono	prev_pgd	0.5+sg	2	0	0	20.38	16.29	1.251	<i>18.69</i>	<i>18.01</i>	1.038	<i>0.02</i>
					2	1	0	20.09	19.43	1.034				
					2	3	0	21.28	17.11	1.244				
					2	4	0	17.96	18.04	0.996				
					2	5	0	16.39	15.47	1.059				
					2	6	0	15.98	19.08	0.838				
					2	7	0	19.00	21.96	0.865				
					2	8	0	21.74	17.54	1.239				
					2	9	0	15.39	17.21	0.894				

Table 20: Architecture **3x10** — perturbation **occ(5,5,5)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 18. Column meanings: see Table 18.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\bar{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
translation	1,1	zono	prev_pgd	0.0+sg	0	1	0	20.98	19.35	1.084	<i>17.81</i>	<i>15.93</i>	1.118	<i>0.01</i>
					0	2	0	21.15	18.06	1.171				
					0	3	0	13.93	18.02	0.773				
					0	4	0	15.18	14.16	1.072				
					0	5	0	15.40	13.80	1.116				
					0	6	0	18.67	13.43	1.390				
					0	7	0	20.37	14.41	1.414				
					0	8	0	16.12	15.46	1.043				
					0	9	0	18.52	16.67	1.111				
	zono	prev_pgd	0.5+sg	0	1	0	20.98	19.31	1.086	<i>17.81</i>	<i>16.25</i>	1.097	<i>0.01</i>	
				0	2	0	21.15	18.57	1.139					
				0	3	0	13.93	18.65	0.747					
				0	4	0	15.18	15.91	0.954					
				0	5	0	15.40	14.76	1.043					
				0	6	0	18.67	13.87	1.346					
				0	7	0	20.37	14.18	1.437					
				0	8	0	16.12	14.36	1.123					
				0	9	0	18.52	16.60	1.116					
	zono	prev_pgd	0.0+sg	1	0	0	20.62	9.85	2.093	<i>20.71</i>	<i>12.24</i>	1.692	<i>0.01</i>	
				1	2	0	16.58	10.44	1.588					
				1	3	0	22.44	10.24	2.191					
				1	4	0	22.97	10.99	2.090					
				1	5	0	21.27	12.26	1.735					
				1	6	0	21.10	13.34	1.582					
				1	7	0	22.94	14.09	1.628					
				1	8	0	22.62	14.35	1.576					
				1	9	0	15.86	14.62	1.085					
	zono	prev_pgd	0.5+sg	1	0	0	20.62	10.36	1.990	<i>20.71</i>	<i>12.11</i>	1.710	<i>0.01</i>	
				1	2	0	16.58	11.25	1.474					
				1	3	0	22.44	11.55	1.943					
				1	4	0	22.97	13.60	1.689					
				1	5	0	21.27	12.34	1.724					
				1	6	0	21.10	12.59	1.676					
				1	7	0	22.94	14.13	1.623					
				1	8	0	22.62	12.11	1.868					
				1	9	0	15.86	11.07	1.433					
	zono	prev_pgd	0.0+sg	2	0	0	19.30	9.73	1.984	<i>14.19</i>	<i>11.84</i>	1.199	<i>0.06</i>	
				2	1	0	14.24	10.33	1.379					
				2	3	0	13.76	12.99	1.059					
				2	4	0	13.78	11.28	1.222					
				2	5	0	13.81	11.77	1.173					
				2	6	0	13.54	12.38	1.094					
				2	7	0	14.15	12.63	1.120					
				2	8	0	13.56	12.48	1.087					
				2	9	0	11.59	12.98	0.893					
	zono	prev_pgd	0.5+sg	2	0	0	19.30	10.29	1.876	<i>14.19</i>	<i>12.14</i>	1.169	<i>0.06</i>	
				2	1	0	14.24	14.03	1.015					
				2	3	0	13.76	13.93	0.988					
				2	4	0	13.78	14.91	0.924					
				2	5	0	13.81	14.44	0.956					
				2	6	0	13.54	12.06	1.123					
				2	7	0	14.15	9.88	1.432					
				2	8	0	13.56	9.80	1.384					
				2	9	0	11.59	9.95	1.165					

Table 21: Architecture **3x10** — perturbation **translation(1,1)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 18. Column meanings: see Table 18.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\bar{sp}_{c_src}$	$ \bar{\delta}_{std} - \bar{\delta}_{adv} _{c_src}$
translation	3,1	zono	prev_pgd	0.0+sg	0	1	0	21.16	18.08	1.170	15.30	15.17	1.008	0.01
					0	2	0	19.76	14.33	1.379				
					0	3	0	13.40	14.38	0.932				
					0	4	0	14.38	15.84	0.908				
					0	5	0	13.93	14.30	0.974				
					0	6	0	11.80	15.13	0.780				
					0	7	0	11.82	15.35	0.770				
					0	8	0	12.12	14.68	0.826				
					0	9	0	19.29	14.45	1.335				
		zono	prev_pgd	0.5+sg	0	1	0	21.16	20.10	1.053	15.30	16.68	0.917	0.01
					0	2	0	19.76	19.06	1.037				
					0	3	0	13.40	19.39	0.691				
					0	4	0	14.38	16.93	0.849				
					0	5	0	13.93	15.94	0.874				
					0	6	0	11.80	15.45	0.764				
					0	7	0	11.82	14.50	0.815				
					0	8	0	12.12	13.89	0.873				
					0	9	0	19.29	14.83	1.301				
		zono	prev_pgd	0.5+sg	1	0	0	18.86	9.83	1.919	20.19	12.17	1.659	0.04
					1	2	0	16.73	10.59	1.580				
					1	3	0	23.91	11.28	2.120				
					1	4	0	22.60	13.70	1.650				
					1	5	0	15.45	13.65	1.132				
					1	6	0	23.56	13.73	1.716				
					1	7	0	24.44	11.33	2.157				
					1	8	0	19.60	12.10	1.620				
					1	9	0	16.56	13.34	1.241				
		zono	prev_pgd	0.0+sg	1	0	0	18.86	9.78	1.928	20.19	12.31	1.640	0.04
					1	2	0	16.73	11.74	1.425				
					1	3	0	23.91	10.76	2.222				
					1	4	0	22.60	11.37	1.988				
					1	5	0	15.45	11.35	1.361				
					1	6	0	23.56	13.02	1.810				
					1	7	0	24.44	14.61	1.673				
					1	8	0	19.60	14.00	1.400				
					1	9	0	16.56	14.17	1.169				
		zono	prev_pgd	0.5+sg	2	0	0	18.70	14.24	1.313	13.92	13.07	1.064	0.07
					2	1	0	11.52	11.37	1.013				
					2	3	0	13.58	11.94	1.137				
					2	4	0	14.04	11.87	1.183				
					2	5	0	13.60	15.59	0.872				
					2	6	0	14.04	12.04	1.166				
					2	7	0	15.10	11.90	1.269				
					2	8	0	13.51	11.63	1.162				
					2	9	0	11.16	17.09	0.653				
		zono	prev_pgd	0.0+sg	2	0	0	18.70	15.37	1.217	13.92	14.73	0.945	0.07
					2	1	0	11.52	13.60	0.847				
					2	3	0	13.58	12.90	1.053				
					2	4	0	14.04	12.79	1.098				
					2	5	0	13.60	14.94	0.910				
					2	6	0	14.04	14.63	0.960				
					2	7	0	15.10	15.91	0.949				
					2	8	0	13.51	18.28	0.739				
					2	9	0	11.16	14.17	0.788				

Table 22: Architecture **3x10** — perturbation **translation(3,1)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 18. Column meanings: see Table 18.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\bar{sp}_{c_src}$	$ \bar{\delta}_{std} - \bar{\delta}_{adv} _{c_src}$
translation	3,3	zono	prev_pgd	0.5+sg	0	1	0	20.13	13.33	1.510	15.27	15.60	0.978	0.02
					0	2	0	21.34	13.09	1.630				
					0	3	0	12.78	13.89	0.920				
					0	4	0	14.57	15.73	0.926				
					0	5	0	14.10	17.22	0.819				
					0	6	0	10.86	16.49	0.659				
					0	7	0	11.99	16.10	0.745				
					0	8	0	12.37	16.89	0.732				
					0	9	0	19.27	17.70	1.089				
		zono	prev_pgd	0.0+sg	0	1	0	20.13	19.54	1.030	15.27	16.36	0.933	0.02
					0	2	0	21.34	17.81	1.198				
					0	3	0	12.78	18.51	0.690				
					0	4	0	14.57	15.84	0.920				
					0	5	0	14.10	14.82	0.951				
					0	6	0	10.86	14.51	0.748				
					0	7	0	11.99	16.35	0.733				
					0	8	0	12.37	14.70	0.841				
					0	9	0	19.27	15.17	1.270				
		zono	prev_pgd	0.0+sg	1	0	0	19.27	9.84	1.958	14.99	11.86	1.264	0.02
					1	2	0	17.32	10.02	1.729				
					1	3	0	14.78	10.34	1.429				
					1	4	0	13.53	10.42	1.298				
					1	5	0	13.74	11.07	1.241				
					1	6	0	12.78	16.24	0.787				
					1	7	0	14.15	14.02	1.009				
					1	8	0	13.03	11.45	1.138				
					1	9	0	16.29	13.33	1.222				
		zono	prev_pgd	0.5+sg	1	0	0	19.27	11.77	1.637	14.99	12.55	1.194	0.02
					1	2	0	17.32	13.25	1.307				
					1	3	0	14.78	10.92	1.353				
					1	4	0	13.53	11.03	1.227				
					1	5	0	13.74	11.97	1.148				
					1	6	0	12.78	16.08	0.795				
					1	7	0	14.15	12.64	1.119				
					1	8	0	13.03	13.39	0.973				
					1	9	0	16.29	11.89	1.370				
		zono	prev_pgd	0.0+sg	2	0	0	18.68	11.94	1.564	14.39	13.76	1.045	0.05
					2	1	0	16.17	12.06	1.341				
					2	3	0	13.92	14.06	0.990				
					2	4	0	13.93	12.24	1.138				
					2	5	0	13.35	11.14	1.198				
					2	6	0	14.39	13.92	1.034				
					2	7	0	13.92	13.95	0.998				
					2	8	0	13.81	15.88	0.870				
					2	9	0	11.30	18.66	0.606				
		zono	prev_pgd	0.5+sg	2	0	0	18.68	12.86	1.453	14.39	13.92	1.034	0.05
					2	1	0	16.17	12.99	1.245				
					2	3	0	13.92	14.88	0.935				
					2	4	0	13.93	12.53	1.112				
					2	5	0	13.35	11.21	1.191				
					2	6	0	14.39	14.06	1.023				
					2	7	0	13.92	13.27	1.049				
					2	8	0	13.81	16.33	0.846				
					2	9	0	11.30	17.12	0.660				

Table 23: Architecture **3x10** — perturbation **translation(3,3)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 18. Column meanings: see Table 18.

p_type	p_size	bound_tight	varHint	τ	c_src	c_tgt	n_relax	t_std	t_adv	sp	$\bar{t}_{c_src}^{std}$	$\bar{t}_{c_src}^{adv}$	$\bar{sp}_{c_src}$	$ \delta_{std} - \delta_{adv} _{c_src}$
linf	0.1	zono	prev_pgd	0.5+sg	0	1	0	24.23	17.45	1.389	<i>21.09</i>	<i>17.52</i>	1.204	<i>0.04</i>
					0	2	0	23.54	19.98	1.178				
					0	3	0	21.68	20.55	1.055				
					0	4	0	21.14	18.69	1.131				
					0	5	0	28.15	16.90	1.666				
					0	6	0	19.07	16.04	1.189				
					0	7	0	16.40	15.09	1.087				
					0	8	0	17.09	17.00	1.005				
					0	9	0	18.54	15.98	1.160				
		zono	prev_pgd	0.0+sg	0	1	0	24.23	15.82	1.532	<i>21.09</i>	<i>17.79</i>	1.186	<i>0.04</i>
					0	2	0	23.54	17.91	1.314				
					0	3	0	21.68	17.82	1.217				
					0	4	0	21.14	17.92	1.180				
					0	5	0	28.15	19.90	1.415				
					0	6	0	19.07	18.87	1.011				
					0	7	0	16.40	16.75	0.979				
					0	8	0	17.09	17.43	0.980				
					0	9	0	18.54	17.68	1.049				
		zono	prev_pgd	0.0+sg	1	0	0	22.43	16.08	1.395	<i>15.99</i>	<i>15.77</i>	1.014	<i>0.05</i>
					1	2	0	20.13	16.82	1.197				
					1	3	0	19.97	19.09	1.046				
					1	4	0	13.25	17.06	0.777				
					1	5	0	14.60	15.80	0.924				
					1	6	0	12.79	13.99	0.914				
					1	7	0	14.48	13.48	1.074				
					1	8	0	13.58	15.70	0.865				
					1	9	0	12.66	13.90	0.911				
		zono	prev_pgd	0.5+sg	1	0	0	22.43	18.57	1.208	<i>15.99</i>	<i>16.68</i>	<i>0.959</i>	<i>0.05</i>
					1	2	0	20.13	19.32	1.042				
					1	3	0	19.97	20.61	0.969				
					1	4	0	13.25	14.82	0.894				
					1	5	0	14.60	15.64	0.934				
					1	6	0	12.79	14.77	0.866				
					1	7	0	14.48	16.21	0.893				
					1	8	0	13.58	15.85	0.857				
					1	9	0	12.66	14.32	0.884				
		zono	prev_pgd	0.0+sg	2	0	0	21.33	14.11	1.512	<i>15.26</i>	<i>15.33</i>	<i>0.996</i>	<i>0.05</i>
					2	1	0	18.17	16.70	1.088				
					2	3	0	17.14	15.72	1.090				
					2	4	0	14.09	16.21	0.869				
					2	5	0	14.45	14.13	1.023				
					2	6	0	13.69	17.29	0.792				
					2	7	0	14.15	15.03	0.941				
					2	8	0	11.64	12.07	0.964				
					2	9	0	12.69	16.69	0.760				
		zono	prev_pgd	0.5+sg	2	0	0	21.33	19.90	1.072	<i>15.26</i>	<i>19.35</i>	<i>0.789</i>	<i>0.04</i>
					2	1	0	18.17	20.27	0.896				
					2	3	0	17.14	21.66	0.791				
					2	4	0	14.09	21.36	0.660				
					2	5	0	14.45	17.15	0.843				
					2	6	0	13.69	21.98	0.623				
					2	7	0	14.15	19.64	0.720				
					2	8	0	11.64	14.66	0.794				
					2	9	0	12.69	17.51	0.725				

Table 24: Architecture **3x10** — perturbation **linf(0.1)**, 2 combos **zono:prev_pgd:0.0+sg**, **zono:prev_pgd:0.5+sg** (54 cell rows); continuation of Table 18. Column meanings: see Table 18.

#	combo	$\overline{sp}$	n_wins	n_cells	win_rate
1	N1, zono, $\tau=0.0$	1.389	28	32	87.5%
2	N1, no-zono, $\tau=0.0$	1.353	27	32	84.4%
3	N1, no-zono, $\tau=0.5$	1.295	27	32	84.4%
4	N1, no-zono, $\tau=0.5$, SibGate	1.335	26	32	81.2%
5	N1, zono, $\tau=0.0$, SibGate	1.288	26	32	81.2%
6	N1, zono, $\tau=0.5$	1.267	25	32	78.1%
7	N1, zono, $\tau=0.5$, SibGate	1.246	24	32	75.0%
8	N1, no-zono, $\tau=0.0$, SibGate	1.223	24	32	75.0%
9	N2, no-zono, $\tau=0.5$	1.190	76	116	65.5%
10	N2, zono, $\tau=0.0$, SibGate	1.152	77	119	64.7%
11	N2, no-zono, $\tau=0.5$, SibGate	1.160	73	118	61.9%
12	N2, no-zono, $\tau=0.0$, SibGate	1.150	73	118	61.9%
13	N2, zono, $\tau=0.5$, SibGate	1.148	73	119	61.3%
14	N2, no-zono, $\tau=0.0$	1.167	71	116	61.2%
15	N2, zono, $\tau=0.5$	1.162	72	119	60.5%
16	N2, zono, $\tau=0.0$	1.134	69	119	58.0%

Table 25: Overall nn1-boost combination ranking, aggregated over every (arch, pert, size, c_{src} , c_{tgt}) cell. $\overline{sp}$: mean per-cell $sp = t_{base}/t_{boost}$ vs the with-perturbed-intervals baseline. **n_wins**: cells with $sp > 1$. Sorted by descending **win_rate**, then $\overline{sp}$.

model	pert (size)	c_{src}	t_{vagher}	$t_z^{\tau=0}$	$t_{SG}^{\tau=0}$	$t_{z+SG}^{\tau=0}$	t_{PI}	$t_{PI}^{\tau=0.5}$	$t_{z+PI}^{\tau=0}$	$t_{z+PI}^{\tau=0.5}$	$t_{SG+PI}^{\tau=0}$	$t_{SG+PI}^{\tau=0.5}$	$t_{z+SG+PI}^{\tau=0}$	$t_{z+SG+PI}^{\tau=0.5}$	$t_{N2}^{adv,\tau=0.0}$	$t_{N2}^{adv,\tau=0.5}$
cnn1 mnist N1	brightness (0.25)	0	18.1	—	—	—	—	—	—	—	—	—	—	—	—	—
	linf (0.1)	0	—	—	—	—	—	60.3	—	60.3	—	60.3	—	60.2	—	—
	occ (1,1,5)	0	5.3	—	—	—	3.5	—	—	—	—	—	—	—	—	—
		1	5.5	—	—	—	0.8	—	—	—	—	—	—	—	—	—
	occ (3,3,5)	0	9.4	—	—	—	3.9	2.0	—	1.8	—	1.8	—	2.0	—	—
		0	20.6	—	—	—	4.0	6.6	—	5.3	—	5.0	—	5.5	—	—
	patch (1,14,14,3)	0	—	—	—	—	2.0	8.3	—	8.3	—	8.2	—	8.4	—	—
		1	—	—	—	—	1.2	—	—	—	—	—	—	—	—	—
	rotation (10)	0	—	—	—	—	5.6	—	—	—	—	—	—	—	—	—
		0	5.7	—	—	—	5.9	14.1	—	14.7	—	14.2	—	15.0	—	—
	translation (1,1)	1	8.2	—	—	—	9.8	—	—	—	—	—	—	—	—	—
		0	4.6	—	—	—	5.0	—	—	—	—	—	—	—	—	—
	translation (1,3)	1	—	—	—	—	9.9	—	—	—	—	—	—	—	—	—
		0	7.4	—	—	—	8.2	13.0	—	12.2	—	11.3	—	12.3	—	—
	translation (3,1)	1	6.7	—	—	—	7.5	—	—	—	—	—	—	—	—	—
		0	7.2	—	—	—	7.5	18.3	—	19.2	—	20.0	—	18.7	—	—
	translation (3,3)	1	—	—	—	—	23.4	—	—	—	—	—	—	—	—	—
cnn1 mnist N2	contrast (1.5)	1	—	—	—	—	18.3	—	—	—	—	—	—	—	—	—
	linf (0.1)	0	—	—	—	—	60.3	—	—	60.3	—	60.3	—	60.3	60.2	60.2
		1	—	—	—	—	60.3	—	—	—	—	—	—	—	60.2	60.2
		2	—	—	—	—	60.3	—	—	—	—	—	—	—	60.3	60.3
	occ (1,1,5)	0	7.8	—	—	—	2.5	—	—	—	—	—	—	—	—	—
		1	1.9	—	—	—	1.4	—	—	—	—	—	—	—	—	—
		0	16.4	—	—	—	2.6	1.9	—	2.2	—	2.2	—	2.1	4.7	1.9
	occ (3,3,5)	1	—	—	—	—	2.0	—	—	—	—	—	—	—	1.2	1.0
		2	—	—	—	—	12.2	—	—	—	—	—	—	—	9.2	2.2
		0	6.5	—	—	—	2.2	7.5	—	6.7	—	7.0	—	6.9	1.4	7.8
	occ (5,5,5)	1	—	—	—	—	1.4	—	—	—	—	—	—	—	1.3	1.1
		2	—	—	—	—	1.6	—	—	—	—	—	—	—	2.2	2.1
		0	11.9	—	—	—	1.7	3.3	—	3.7	—	3.6	—	3.6	1.2	1.1
	patch (1,14,14,3)	1	—	—	—	—	1.5	—	—	—	—	—	—	—	6.0	0.8
		2	—	—	—	—	2.3	—	—	—	—	—	—	—	1.9	1.5
		0	—	—	—	—	5.7	—	—	—	—	—	—	—	—	—
	rotation (10)	0	6.4	—	—	—	14.7	22.2	—	19.8	—	21.4	—	22.2	12.9	12.4
		1	5.5	—	—	—	23.1	—	—	—	—	—	—	—	17.8	21.7
		2	—	—	—	—	22.4	—	—	—	—	—	—	—	16.9	18.4
	translation (1,1)	0	4.9	—	—	—	6.2	—	—	—	—	—	—	—	—	—
		1	—	—	—	—	14.2	—	—	—	—	—	—	—	—	—
		2	—	—	—	—	9.2	—	—	—	—	—	—	—	—	—
	translation (1,3)	0	6.5	—	—	—	11.2	14.1	—	15.0	—	13.8	—	14.8	11.7	8.0
		1	2.5	—	—	—	4.9	—	—	—	—	—	—	—	4.6	6.6
		2	—	—	—	—	19.9	—	—	—	—	—	—	—	19.5	16.9
	translation (3,1)	0	9.3	—	—	—	16.2	19.1	—	19.0	—	19.0	—	18.4	12.9	20.0
		1	—	—	—	—	35.7	—	—	—	—	—	—	—	48.6	48.7
		2	—	—	—	—	47.3	—	—	—	—	—	—	—	48.6	47.5

Table 26: Architecture **cnn1** (dataset mnist) — per-cell wall-clock time (minutes), mean over c_{tgt} and Gurobi seed. The two baselines t_{vagher} (PI=off, no boosts) and t_{PI} (PI=on, no boosts) each pool together the standalone vaghar baseline runs (`vagharNoPerturbed_*` / `vagharWithPerturbed_*`) and the `stdBoost` cells with all options off ($z = F, SG = F, \tau = 0$ and matching PI value), since those cells encode an identical MIP and only differ in Gurobi seed. The remaining columns are the actual boosts. In the column labels, z stands for `nn1.zono.bounds`, SG for `nn1.sibling_gate`, PI for `--use_perturbed_intervals`, and the τ superscript is `nn1.relax_threshold`. Combinations the sweep filter excludes ((PI=off, $\tau > 0$); see `run_relaxation_sweep.py:2854`) appear as —. The two rightmost columns $t_{N2}^{adv,\tau=0}$ and $t_{N2}^{adv,\tau=0.5}$ report the advstd N2 transfer-mode runs for the two `zono:prev_pgd:0.0+sg` and `zono:prev_pgd:0.5+sg` combinations (loaded directly from the per-cell advstd CSVs that back Section 5); these rows only populate on the N_2 block since advstd is a transfer technique. The fastest entry in each row is in **bold**.

model	pert (size)	c_{src}	t_{vaghar}	$t_z^{\tau=0}$	$t_{SG}^{\tau=0}$	$t_{z+SG}^{\tau=0}$	t_{PI}	$t_{PI}^{\tau=0.5}$	$t_{z+PI}^{\tau=0}$	$t_{z+PI}^{\tau=0.5}$	$t_{SG+PI}^{\tau=0}$	$t_{SG+PI}^{\tau=0.5}$	$t_{z+SG+PI}^{\tau=0}$	$t_{z+SG+PI}^{\tau=0.5}$	$t_{N2}^{adv,\tau=0.0}$	$t_{N2}^{adv,\tau=0.5}$
3x50 mnist N2	linf (0.1)	0	—	—	—	—	60.2	—	—	—	—	—	—	—	60.2	60.3
		1	—	—	—	—	60.2	—	—	—	—	—	—	—	60.4	60.2
		2	—	—	—	—	60.3	—	—	—	—	—	—	—	60.2	60.3
	occ (3,3,5)	0	—	—	—	—	60.3	—	—	—	—	—	—	—	60.3	60.2
		1	—	—	—	—	60.3	—	—	—	—	—	—	—	60.2	60.3
		2	—	—	—	—	60.2	—	—	—	—	—	—	—	60.3	60.3
	occ (5,5,5)	0	—	—	—	—	60.2	—	—	—	—	—	—	—	60.3	60.3
		1	—	—	—	—	60.3	—	—	—	—	—	—	—	60.2	60.3
		2	—	—	—	—	60.3	—	—	—	—	—	—	—	60.3	60.2
	patch (1,14,14,3)	0	—	—	—	—	60.3	—	—	—	—	—	—	—	60.3	60.4
		1	—	—	—	—	58.3	—	—	—	—	—	—	—	56.2	56.3
		2	—	—	—	—	60.3	—	—	—	—	—	—	—	60.3	60.3
	translation (1,1)	0	—	—	—	—	60.3	—	—	—	—	—	—	—	60.3	60.3
		1	—	—	—	—	51.2	—	—	—	—	—	—	—	41.6	39.8
		2	—	—	—	—	56.3	—	—	—	—	—	—	—	52.7	54.5
	translation (3,1)	0	—	—	—	—	34.7	—	—	—	—	—	—	—	19.7	17.7
		1	—	—	—	—	17.5	—	—	—	—	—	—	—	10.3	6.6
		2	—	—	—	—	24.6	—	—	—	—	—	—	—	23.8	21.0
	translation (3,3)	0	—	—	—	—	18.7	—	—	—	—	—	—	—	11.7	10.8
		1	—	—	—	—	20.1	—	—	—	—	—	—	—	18.1	17.8
		2	—	—	—	—	14.3	—	—	—	—	—	—	—	8.9	7.8

Table 27: Architecture **3x50** (dataset mnist) — per-cell wall-clock time (minutes), mean over c_{tgt} and Gurobi seed. The two baselines t_{vaghar} (PI=off, no boosts) and t_{PI} (PI=on, no boosts) each pool together the standalone vaghar baseline runs (`vagharNoPerturbed.*` / `vagharWithPerturbed.*`) and the `stdBoost` cells with all options off ($z = F, SG = F, \tau = 0$ and matching PI value), since those cells encode an identical MIP and only differ in Gurobi seed. The remaining columns are the actual boosts. In the column labels, z stands for `nn1_zono_bounds`, SG for `nn1_sibling_gate`, PI for `--use_perturbed_intervals`, and the τ superscript is `nn1_relax_threshold`. Combinations the sweep filter excludes ((PI=off, $\tau > 0$); see `run_relaxation_sweep.py:2854`) appear as —. The two rightmost columns $t_{N2}^{adv,\tau=0}$ and $t_{N2}^{adv,\tau=0.5}$ report the advstd N2 transfer-mode runs for the two `zono:prev_pgd:0.0+sg` and `zono:prev_pgd:0.5+sg` combinations (loaded directly from the per-cell advstd CSVs that back Section 5); these rows only populate on the N_2 block since advstd is a transfer technique. The fastest entry in each row is in **bold**.

model	pert (size)	c_{src}	t_{vagher}	$t_z^{\tau=0}$	$t_{SG}^{\tau=0}$	$t_{z+SG}^{\tau=0}$	t_{PI}	$t_{PI}^{\tau=0.5}$	$t_{z+PI}^{\tau=0}$	$t_{z+PI}^{\tau=0.5}$	$t_{SG+PI}^{\tau=0}$	$t_{SG+PI}^{\tau=0.5}$	$t_{z+SG+PI}^{\tau=0}$	$t_{z+SG+PI}^{\tau=0.5}$	$t_{N2}^{adv,\tau=0.0}$	$t_{N2}^{adv,\tau=0.5}$
3x10 mnist N1	linf (0.1)	0	—	—	—	—	—	0.3	—	—	—	—	—	—	—	—
	occ (1,1,5)	0	—	—	—	—	0.3	—	—	—	—	—	—	—	—	—
	occ (3,3,5)	0	—	—	—	—	0.3	0.3	—	0.3	—	0.2	—	0.3	—	—
	occ (5,5,5)	0	—	—	—	—	0.3	0.2	—	0.2	—	0.3	—	0.2	—	—
	patch (1,14,14,3)	0	—	—	—	—	0.3	0.3	—	0.3	—	0.3	—	0.3	—	—
	rotation (10)	0	—	—	—	—	0.5	—	—	—	—	—	—	—	—	—
	translation (1,1)	0	—	—	—	—	0.3	0.2	—	0.2	—	0.2	—	0.2	—	—
	translation (1,3)	0	—	—	—	—	0.3	—	—	—	—	—	—	—	—	—
	translation (3,1)	0	—	—	—	—	0.3	0.2	—	0.2	—	0.2	—	0.3	—	—
	translation (3,3)	0	—	—	—	—	0.3	0.2	—	0.2	—	0.2	—	0.2	—	—
3x10 mnist N2	brightness (0.25)	0	—	—	—	—	0.4	—	—	—	—	—	—	—	—	—
	contrast (1.5)	1	—	—	—	—	1.3	—	—	—	—	—	—	—	—	—
	linf (0.05)	0	—	—	—	—	0.3	—	—	—	—	—	—	—	—	—
	linf (0.1)	0	—	—	—	—	0.4	0.2	—	0.3	—	0.2	—	0.3	0.3	0.3
		1	—	—	—	—	0.3	—	—	—	—	—	—	—	0.3	0.3
		2	—	—	—	—	0.3	—	—	—	—	—	—	—	0.3	0.3
		2	—	—	—	—	0.3	—	—	—	—	—	—	—	—	—
	occ (1,1,5)	0	—	—	—	—	0.3	—	—	—	—	—	—	—	—	—
		0	—	—	—	—	0.3	0.2	—	0.2	—	0.2	—	0.2	0.3	0.2
		1	—	—	—	—	0.4	—	—	—	—	—	—	—	0.2	0.2
		2	—	—	—	—	0.3	—	—	—	—	—	—	—	0.2	0.2
	occ (5,5,5)	0	—	—	—	—	0.3	0.2	—	0.3	—	0.3	—	0.3	0.3	0.2
		1	—	—	—	—	0.3	—	—	—	—	—	—	—	0.3	0.2
		2	—	—	—	—	0.3	—	—	—	—	—	—	—	0.3	0.3
		2	—	—	—	—	0.3	—	—	—	—	—	—	—	0.3	0.3
	patch (1,14,14,3)	0	—	—	—	—	0.3	0.3	—	0.3	—	0.3	—	0.3	0.3	0.3
		1	—	—	—	—	0.4	—	—	—	—	—	—	—	0.3	0.3
		2	—	—	—	—	0.3	—	—	—	—	—	—	—	0.3	0.3
		2	—	—	—	—	0.5	—	—	—	—	—	—	—	—	—
	rotation (10)	0	—	—	—	—	0.3	0.2	—	0.2	—	0.2	—	0.2	0.2	0.2
		1	—	—	—	—	0.3	—	—	—	—	—	—	—	0.2	0.2
		2	—	—	—	—	0.2	—	—	—	—	—	—	—	0.2	0.2
		2	—	—	—	—	0.2	—	—	—	—	—	—	—	—	—
	translation (1,3)	0	—	—	—	—	0.2	—	—	—	—	—	—	—	—	—
		1	—	—	—	—	0.4	—	—	—	—	—	—	—	—	—
		2	—	—	—	—	0.2	—	—	—	—	—	—	—	—	—
		2	—	—	—	—	0.2	—	—	—	—	—	—	—	—	—
	translation (3,1)	0	—	—	—	—	0.3	0.2	—	0.2	—	0.2	—	0.2	0.2	0.2
		1	—	—	—	—	0.3	—	—	—	—	—	—	—	0.2	0.2
		2	—	—	—	—	0.2	—	—	—	—	—	—	—	0.2	0.2
		2	—	—	—	—	0.2	—	—	—	—	—	—	—	0.2	0.2
	translation (3,3)	0	—	—	—	—	0.3	0.2	—	0.2	—	0.2	—	0.2	0.2	0.2
		1	—	—	—	—	0.2	—	—	—	—	—	—	—	0.2	0.2
		2	—	—	—	—	0.2	—	—	—	—	—	—	—	0.2	0.2
		2	—	—	—	—	0.2	—	—	—	—	—	—	—	0.2	0.2

Table 28: Architecture **3x10** (dataset mnist) — per-cell wall-clock time (minutes), mean over c_{tgt} and Gurobi seed. The two baselines t_{vagher} (PI=off, no boosts) and t_{PI} (PI=on, no boosts) each pool together the standalone vaghar baseline runs (`vagharNoPerturbed.*` / `vagharWithPerturbed.*`) and the `stdBoost` cells with all options off ($z = F, SG = F, \tau = 0$ and matching PI value), since those cells encode an identical MIP and only differ in Gurobi seed. The remaining columns are the actual boosts. In the column labels, z stands for `nn1_zono_bounds`, SG for `nn1_sibling_gate`, PI for `--use_perturbed_intervals`, and the τ superscript is `nn1_relax_threshold`. Combinations the sweep filter excludes ((PI=off, $\tau > 0$); see `run_relaxation_sweep.py:2854`) appear as —. The two rightmost columns $t_{N2}^{adv,\tau=0}$ and $t_{N2}^{adv,\tau=0.5}$ report the advstd N2 transfer-mode runs for the two `zono:prev_pgd:0.0+sg` and `zono:prev_pgd:0.5+sg` combinations (loaded directly from the per-cell advstd CSVs that back Section 5); these rows only populate on the N_2 block since advstd is a transfer technique. The fastest entry in each row is in **bold**.